

Claude Code 완벽 입문

터미널에서 시작하는 AI 코딩의 새로운 패러다임

저자: @jaden__ai

Claude Code를 시작하는 개발자를 위한 실전 가이드

목차

머리말

Part 1: Claude Code와의 첫 만남

AI 코딩 도구의 배경을 이해하고, Claude Code를 설치하여 첫 대화를 나누며, 기본적인 사용법을 익힙니다.

- **Chapter 1: AI 코딩 도구의 시대**

- 1.1 자동완성에서 에이전트로: AI 코딩 도구의 진화
- 1.2 Claude Code란 무엇인가
- 1.3 왜 Claude Code인가: 구체적 사례로 보는 가치
- 1.4 경쟁 도구와의 비교
- 1.5 이 책의 로드맵
- 1.6 정리

- **Chapter 2: 설치와 첫 대화**

- 2.1 시스템 요구사항 확인
- 2.2 설치 방법 선택
- 2.3 인증과 계정 설정
- 2.4 첫 세션 시작하기
- 2.5 권한 시스템 이해하기
- 2.6 초기 프롬프트와 비대화형 모드
- 2.7 업데이트와 버전 관리
- 2.8 문제 발생 시 진단
- 2.9 실습: 첫 대화 나눠보기
- 2.10 정리

- **Chapter 3: 기본기 다지기**

- 3.1 대화형 모드 vs 비대화형 모드
- 3.2 효과적인 프롬프트 작성법
- 3.3 핵심 슬래시 명령어
- 3.4 모델 선택 전략
- 3.5 노력 수준 (Effort Level)
- 3.6 비용 이해와 관리
- 3.7 세션 관리 고급 기법
- 3.8 파이프라인과 자동화 활용
- 3.9 실습: 기본기 종합 연습
- 3.10 정리

Part 2: 핵심 기능 마스터

Claude Code의 4가지 핵심 역량(코드 읽기, 코드 쓰기, Git 연동, 프로젝트 메모리)을 깊이 있게 학습합니다.

- **Chapter 4: 코드 읽기와 이해**

- 4.1 프로젝트 전체 구조 파악하기
- 4.2 파일 및 함수 단위 분석
- 4.3 의존성 추적
- 4.4 아키텍처 시각화
- 4.5 코드 리뷰 요청
- 4.6 코드 검색 도구 이해하기
- 4.7 효과적인 코드 읽기를 위한 팁
- 정리

- **Chapter 5: 코드 작성과 편집**

- 5.1 새 파일 생성하기
- 5.2 기존 코드 수정하기
- 5.3 테스트 코드 작성
- 5.4 리팩토링
- 5.5 멀티 파일 변경과 Plan 모드
- 5.6 체크포인트와 되돌리기
- 5.7 JavaScript에서 TypeScript로 변환
- 5.8 효과적인 코드 작성 요청 팁
- 정리

- **Chapter 6: Git과 함께하기**

- 6.1 자동 커밋
- 6.2 브랜치 관리
- 6.3 PR 생성
- 6.4 PR 코멘트 처리
- 6.5 머지 충돌 해결
- 6.6 Git 히스토리 분석
- 6.7 파이프라인을 활용한 코드 리뷰
- 6.8 Git Worktree로 병렬 작업하기
- 정리

- **Chapter 7: CLAUDE.md로 프로젝트 기억시키기**

- 7.1 왜 CLAUDE.md가 필요한가
- 7.2 CLAUDE.md 파일의 위치와 범위
- 7.3 효과적인 CLAUDE.md 작성법
- 7.4 `/init` 으로 자동 생성하기
- 7.5 Rules 시스템: 규칙의 모듈화
- 7.6 외부 파일 임포트
- 7.7 Auto Memory: Claude의 자동 학습
- 7.8 CLAUDE.md 실전 운영 가이드
- 정리

Part 3: 실전 프로젝트

실제 개발 시나리오(신규 웹앱 구축, 레거시 리팩토링, 디버깅)에서 Claude Code를 처음부터 끝까지 활용합니다.

- **Chapter 8: 웹 앱 만들기 -- 처음부터 끝까지**

- 8.1 프로젝트 설계: Plan 모드로 큰 그림 그리기
- 8.2 프로젝트 초기화: 기반 구조 잡기
- 8.3 백엔드 API 구축: 데이터베이스와 라우트
- 8.4 프론트엔드 UI 구현: 컴포넌트와 상태 관리
- 8.5 테스트 작성: 안정성 확보
- 8.6 마무리: 커밋과 배포 준비
- 8.7 되돌아보기: Claude Code와의 협업 패턴
- 정리

- **Chapter 9: 레거시 코드 리팩토링**

- 9.1 레거시 코드 분석: 무엇이 문제인가
- 9.2 1단계: 테스트 안전망 구축
- 9.3 2단계: 보안 취약점 수정
- 9.4 3단계: 구조 분해 -- 거대 함수 쪼개기
- 9.5 4단계: 모던 문법 전환 -- 콜백에서 async/await로
- 9.6 5단계: 최종 정리
- 9.7 전후 비교: 리팩토링의 성과
- 정리

- **Chapter 10: 버그 사냥과 디버깅**

- 10.1 에러 메시지 기반 디버깅: 첫 번째 단서 잡기
- 10.2 git diff로 근본 원인 파악
- 10.3 로그 파이프라인 분석: 실시간 감시
- 10.4 재현 테스트 작성: 버그를 가두다
- 10.5 성능 디버깅: 느린 API 응답 추적
- 10.6 디버깅 워크플로 정리
- 10.7 Claude Code와 모니터링 도구 연동
- 10.8 디버깅 치트시트: 상황별 프롬프트 모음
- 10.9 실시간 모니터링 파이프라인 구축
- 정리

Part 4: 고급 활용과 확장

Claude Code의 설정을 심화 커스터마이징하고, MCP/Hooks로 기능을 확장하며, IDE 연동과 팀 협업 전략을 수립합니다.

- **Chapter 11: 설정 심화**

- 11.1 설정 파일 체계 이해하기
- 11.2 settings.json 구조 상세
- 11.3 권한 모드 (Permission Mode)
- 11.4 권한 규칙 심화
- 11.5 샌드박스 설정
- 11.6 팀 설정과 개인 설정 분리
- 11.7 조직 전체 관리형 설정 (Managed Settings)
- 11.8 설정 디버깅
- 정리

- **Chapter 12: MCP 서버로 능력 확장**

- 12.1 MCP란 무엇인가
- 12.2 MCP 서버 설치 방법
- 12.3 MCP 관리 명령어
- 12.4 MCP 설치 범위 (Scope)
- 12.5 팀 공유용 .mcp.json
- 12.6 실전 MCP 활용 예시
- 12.7 Claude Code를 MCP 서버로 사용하기
- 12.8 Tool Search로 컨텍스트 최적화
- 정리

- **Chapter 13: Hooks로 워크플로우 자동화**

- 13.1 Hooks 시스템 개요
- 13.2 Hook 이벤트 종류
- 13.3 Hook 설정 구조
- 13.4 command Hook 상세
- 13.5 http Hook 상세
- 13.6 prompt Hook 상세
- 13.7 agent Hook 상세
- 13.8 실전 Hook 레시피 모음
- 13.9 Hook 설정 위치와 디버깅
- 정리

- **Chapter 14: IDE 연동과 팀 협업**

- 14.1 VS Code 확장
- 14.2 JetBrains 플러그인
- 14.3 데스크톱 앱과 웹 브라우저
- 14.4 팀 설정 표준화
- 14.5 Git Worktree 병렬 작업
- 14.6 멀티 에이전트 팀
- 14.7 CI/CD 통합
- 14.8 Chrome 연동과 원격 제어
- 정리

부록

- **부록 A:** 슬래시 명령어 전체 레퍼런스
- **부록 B:** CLI 플래그 레퍼런스
- **부록 C:** 트러블슈팅 가이드
- **부록 D:** 유용한 CLAUDE.md 템플릿 모음

맺음말

머리말

| 이 책을 쓴 이유

2025년, 소프트웨어 개발의 풍경이 근본적으로 바뀌기 시작했습니다. AI가 코드 한두 줄을 자동완성하던 시대를 지나, 이제는 "인증 모듈의 테스트를 작성하고, 실행하고, 실패하면 수정해줘"라고 말하면 AI가 스스로 파일을 읽고, 코드를 작성하고, 테스트를 실행하고, 결과를 보고하는 시대가 되었습니다. 이것이 바로 ****에이전틱 코딩(Agentic Coding)****이며, Claude Code는 이 패러다임을 선도하는 도구입니다.

하지만 강력한 도구일수록 제대로 사용하는 법을 아는 것이 중요합니다. Claude Code를 설치하고 "뭔가 만들어줘"라고 말하는 것과, 프로젝트의 맥락을 정확히 전달하고, 적절한 모델과 모드를 선택하며, 팀 전체가 일관된 규칙 아래 협업하는 것 사이에는 큰 생산성 차이가 있습니다.

이 책은 그 차이를 메우기 위해 쓰였습니다. Claude Code를 처음 접하는 개발자가 설치부터 시작하여, 핵심 기능을 마스터하고, 실전 프로젝트에 적용하며, 팀 전체의 개발 워크플로를 혁신하기까지의 전체 여정을 안내합니다.

| 이 책의 구성

이 책은 네 개의 파트로 구성되어 있습니다.

Part 1: Claude Code와의 첫 만남 (Chapter 1~3)에서는 AI 코딩 도구의 배경을 이해하고, Claude Code를 설치하고, 기본적인 대화를 나누며, 모델 선택, 비용 관리, 세션 관리 등 효율적 사용의 기본기를 다집니다.

Part 2: 핵심 기능 마스터 (Chapter 4~7)에서는 코드 읽기, 코드 쓰기, Git 연동, 프로젝트 메모리(CLAUDE.md)라는 네 가지 핵심 역량을 깊이 있게 학습합니다. 이 네 가지가 Claude Code를 일상적인 개발에 통합하는 기반입니다.

Part 3: 실전 프로젝트 (Chapter 8~10)에서는 빈 디렉터리에서 웹 앱을 구축하고, 레거시 코드를 단계적으로 리팩토링하며, 긴급한 프로덕션 버그를 추적하여 해결하는 실제 개발 시나리오를 체험합니다.

Part 4: 고급 활용과 확장 (Chapter 11~14)에서는 설정 심화, MCP를 통한 외부 서비스 연동, Hooks를 활용한 워크플로 자동화, IDE 연동과 팀 협업 전략 등 고급 주제를 다룹니다.

각 챕터는 학습 목표로 시작하여, 개념 설명, 실제 대화 예시, 실습, 그리고 정리로 마무리됩니다. 순서대로 읽는 것을 권장하지만, 이미 기본기가 있는 분은 관심 있는 챕터부터 읽어도 됩니다.

| 이 책을 읽는 방법

- **입문자**: Part 1부터 순서대로 읽으며, 각 실습을 직접 따라 해보세요.
- **경험자**: Part 1을 빠르게 훑고, Part 2의 관심 챕터부터 시작하세요.
- **팀 리드**: Part 4의 설정 표준화와 CI/CD 통합을 먼저 읽고, 팀 도입 전략을 세우세요.

- **레퍼런스:** 부록의 명령어/플래그/설정 레퍼런스를 수시로 참고하세요.

자, 이제 터미널을 열고 Claude Code와의 첫 만남을 시작합시다.

Chapter 1: AI 코딩 도구의 시대

학습 목표

이 챕터를 마치면 다음을 할 수 있습니다:

1. AI 코딩 도구의 발전 흐름과 에이전틱 코딩(Agentic Coding)의 개념을 이해합니다
2. Claude Code가 기존 도구(GitHub Copilot, Cursor)와 어떻게 다른지 설명할 수 있습니다
3. Claude Code의 핵심 철학과 아키텍처를 파악합니다

1.1 자동완성에서 에이전트로: AI 코딩 도구의 진화

프로그래밍의 역사는 곧 "더 적게 타이핑하고, 더 많이 생각하기"의 역사입니다. 어셈블리에서 고급 언어로, 고급 언어에서 프레임워크로, 프레임워크에서 코드 생성기로. 그리고 이제, AI가 그 다음 단계를 열었습니다.

AI 코딩 도구의 발전은 크게 세 세대로 나눌 수 있습니다.

1세대: 인라인 자동완성 (2021~2022)

GitHub Copilot이 대표적입니다. 코드를 작성하는 도중 다음 줄을 예측하여 제안하는 방식입니다. 마치 스마트폰 키보드의 자동완성이 문장 단위로 확장된 것과 비슷합니다.

```
# 함수 이름과 주석만 작성하면
def calculate_fibonacci(n):
    """n번째 피보나치 수를 반환합니다."""
    # Copilot이 나머지를 자동완성합니다
    if n <= 1:
        return n
    return calculate_fibonacci(n - 1) + calculate_fibonacci(n - 2)
```

이 방식은 반복적인 패턴 코딩에서 놀라운 속도 향상을 가져왔지만, 한계도 분명했습니다. 한 번에 몇 줄의 코드만 제안할 수 있었고, 프로젝트 전체의 맥락을 이해하지 못했습니다.

2세대: IDE 통합 어시스턴트 (2023~2024)

Cursor, GitHub Copilot Chat 등이 등장하면서 AI가 "대화 상대"가 되었습니다. 채팅 창에서 질문을 하고, 여러 파일에 걸친 변경을 제안받을 수 있게 되었습니다. IDE 안에서 코드를 보며 AI와 토론하는 경험은 혁신적이었습니다.

하지만 여전히 개발자가 주도권을 쥐고 있었습니다. AI가 "이렇게 바꾸면 어떨까요?"라고 제안하면, 개발자가 하나하나 검토하고 적용해야 했습니다. 테스트를 돌리거나, 빌드를 실행하거나, git에 커밋하는 것은 여전히 개발자의 몫이었습니다.

3세대: 에이전틱 코딩 (2025~현재)

그리고 지금, 우리는 3세대에 와 있습니다. **에이전틱 코딩(Agentic Coding)**의 시대입니다.

에이전틱 코딩이란 무엇일까요? 한마디로 말하면, AI가 "자율적으로 연쇄 작업을 수행하는 것"입니다. 개발자가 "인증 모듈의 테스트를 작성하고, 실행하고, 실패하면 수정해줘"라고 지시하면, AI가 다음 과정을 스스로 수행합니다:

1. 인증 모듈의 코드를 읽고 구조를 파악합니다
2. 적절한 테스트 파일을 생성합니다
3. 테스트를 실행합니다
4. 실패한 테스트가 있으면 원인을 분석합니다
5. 코드를 수정하고 다시 테스트를 실행합니다
6. 모든 테스트가 통과하면 결과를 보고합니다

이것이 바로 Claude Code가 하는 일입니다.

1.2 Claude Code란 무엇인가

Claude Code는 Anthropic이 개발한 **에이전틱 코딩 도구**입니다. 터미널(Terminal)에서 동작하며, 코드베이스를 이해하고, 파일을 편집하고, 명령어를 실행하고, 다양한 개발 도구와 통합되는 AI 기반 코딩 어시스턴트입니다.

좀 더 구체적으로 말하면, Claude Code는 다음과 같은 작업을 자연어 명령 하나로 수행할 수 있습니다:

- **기능 구현**: "사용자 프로필 페이지를 만들어줘"
- **버그 수정**: "로그인 시 500 에러가 발생하는 문제를 해결해줘"
- **테스트 작성**: "auth 모듈의 단위 테스트를 작성하고 실행해줘"
- **코드 리뷰**: "이 PR의 변경 사항을 보안 관점에서 검토해줘"
- **Git 작업**: "변경 사항을 의미 있는 메시지로 커밋하고 PR을 만들어줘"
- **리팩토링**: "이 파일의 콜백 지옥을 async/await로 변환해줘"
- **코드 설명**: "이 결제 처리 파이프라인의 흐름을 설명해줘"

Claude Code의 네 가지 철학

Claude Code가 다른 도구와 차별화되는 핵심 철학이 있습니다.

1. 에이전틱(Agentic)

지시를 받으면 파일 읽기, 쓰기, 명령 실행, 테스트, git 커밋 등을 자율적으로 연쇄 수행합니다. 개발자가 매 단계마다 개입할 필요가 없습니다.

2. 터미널 네이티브(Terminal Native)

Unix 철학을 따릅니다. 파이프라인(Pipeline)과 스크립팅이 가능하며, 다른 터미널 도구들과 자연스럽게 조합됩니다.

```
# git diff의 출력을 Claude Code에게 넘겨 보안 리뷰 요청
git diff main --name-only | claude -p "이 변경 파일들의 보안 이슈를 검토해줘"
```

3. 코드베이스 인식(Codebase Aware)

전체 프로젝트 구조를 이해합니다. 단일 파일이 아니라 프로젝트 전체의 맥락에서 작업합니다. 최대 1백만 토큰(약 75만 단어)의 컨텍스트 윈도우(Context Window)를 활용하여 대규모 코드베이스도 다룰 수 있습니다.

4. 자연어 인터페이스(Natural Language Interface)

모든 작업을 자연어 명령으로 수행합니다. 특별한 문법이나 DSL(Domain Specific Language)을 배울 필요가 없습니다. 동료 개발자에게 부탁하듯 말하면 됩니다.

1.3 왜 Claude Code인가: 구체적 사례로 보는 가치

이론적인 설명보다 실제 사례를 통해 Claude Code의 가치를 느껴보겠습니다.

사례 1: 새벽 2시의 긴급 버그 수정

운영 중인 서비스에서 결제 오류가 발생했습니다. Sentry 알림이 울리고, 슬랙에는 고객 불만이 쌓이고 있습니다. 에러 로그를 확인해보니 이런 메시지가 보입니다:

```
TypeError: Cannot read properties of undefined (reading 'amount')
    at processPayment (src/payments/processor.js:47:23)
    at handleCheckout (src/routes/checkout.js:112:5)
```

기존 방식: 에러가 발생한 파일을 열고, 47번째 줄을 확인하고, 관련 함수를 추적하며 원인을 파악합니다. 수정 후 테스트를 실행하고, 커밋하고, 배포합니다. 숙련된 개발자도 30분~1시간은 걸립니다.

Claude Code 방식:

사용자: processPayment 함수에서 TypeError가 발생합니다.
"Cannot read properties of undefined (reading 'amount')"
에러가 발생한 위치는 src/payments/processor.js:47 입니다.
원인을 찾아서 수정해주세요.

Claude Code는 해당 파일을 읽고, 호출 경로를 추적하고, 입력 데이터의 유효성 검사가 누락된 것을 발견합니다. null 체크를 추가하고, 관련 테스트를 작성하고 실행하여 통과를 확인한 뒤 결과를 보고합니다. 전체 과정이 5분 이내에 완료됩니다.

사례 2: 레거시 코드 이해하기

팀에 새로 합류했는데, 3년간 10명이 넘는 개발자가 쓴 인증 모듈을 파악해야 합니다. 문서는 오래전에 업데이트가 멈췄고, 코드에는 "TODO: 나중에 정리" 주석이 곳곳에 있습니다.

사용자: src/auth/ 디렉터리의 전체 구조를 분석하고,
로그인 요청이 들어왔을 때의 처리 흐름을
단계별로 설명해줘.
각 단계에서 어떤 파일의 어떤 함수가
호출되는지 포함해줘.

Claude Code는 디렉터리 구조를 탐색하고, 진입점(Entry Point)부터 데이터베이스 쿼리까지의 전체 호출 체인을 분석하여, 이해하기 쉬운 형태로 정리해줍니다. 신규 팀원이 코드를 파악하는 데 걸리는 시간이 며칠에서 몇 시간으로 단축됩니다.

사례 3: 반복 작업 자동화

프로젝트의 모든 JavaScript 파일을 TypeScript로 변환해야 합니다. 파일이 50개가 넘고, 각 파일마다 타입을 추론하고, import 구문을 수정하고, tsconfig.json을 설정해야 합니다.

사용자: src/ 디렉터리의 모든 .js 파일을 .ts로 변환해줘.
기존 코드에서 타입을 최대한 추론하고,
any 타입은 최소한으로 사용해줘.
변환 후 tsc로 타입 체크를 실행해서
에러가 있으면 수정해줘.

이런 대규모 반복 작업에서 Claude Code는 압도적인 생산성을 발휘합니다.

1.4 경쟁 도구와의 비교

2026년 현재, AI 코딩 도구 시장의 주요 플레이어를 비교해보겠습니다.

비교 요약표

특성	Claude Code	Cursor	GitHub Copilot
유형	에이전틱 CLI 도구	IDE (VS Code 포크)	IDE 확장
접근법	자율적 작업 실행	IDE 통합 어시스턴트	인라인 자동완성
최대 컨텍스트	최대 1M 토큰	제한적	제한적
자동화 수준	최고 (완전 자율)	중간 (Composer)	낮음 (패턴 기반)
실행 환경	터미널, IDE, 웹, 데스크톱	Cursor IDE	VS Code 등 IDE

GitHub Copilot

GitHub Copilot은 AI 코딩 도구의 대중화를 이끈 선구자입니다. Microsoft와 GitHub의 방대한 생태계 위에 구축되어 있어 기업 환경에서의 신뢰도가 높습니다.

강점:

- 인라인 자동완성의 속도와 정확도가 뛰어납니다
- 기업 환경에서의 보안과 규정 준수에 강합니다
- Visual Studio, VS Code 등 Microsoft 생태계와의 통합이 매끄럽습니다

한계:

- 복잡한 멀티 파일 작업에서는 제한적입니다
- 자율적인 작업 실행 능력이 부족합니다
- 컨텍스트 윈도우가 상대적으로 작습니다

적합한 사용자: 일상적인 코딩 속도 향상이 목적이거나, 기업 규정 준수가 중요한 환경의 개발자

Cursor

Cursor는 VS Code를 포크(Fork)하여 AI 기능을 깊이 통합한 IDE입니다. Composer 기능으로 여러 파일에 걸친 변경을 제안할 수 있으며, 다양한 AI 모델(Claude, GPT-4 등)을 선택할 수 있는 유연성이 특징입니다.

강점:

- Composer로 멀티 파일 변경이 가능합니다
- 모델을 자유롭게 선택할 수 있습니다
- IDE의 시각적 인터페이스가 직관적입니다

한계:

- Cursor IDE에 종속적입니다 (다른 IDE에서는 사용 불가)
- 별도의 구독이 필요합니다
- 자율적 작업 실행은 제한적입니다

적합한 사용자: 시각적 인터페이스를 선호하고, 일상적인 개발에서 AI 어시스턴트를 활용하고 싶은 개발자

Claude Code

Claude Code는 에이전틱 코딩의 대표주자입니다. 터미널에서 동작하며, 지시를 받으면 파일 읽기/쓰기, 명령 실행, 테스트, 커밋까지 자율적으로 연쇄 수행합니다.

강점:

- 복잡한 문제 해결과 자율적 작업 실행 능력이 최고 수준입니다
- 최대 1백만 토큰의 컨텍스트 윈도우로 대규모 코드베이스를 다룰 수 있습니다
- 터미널 네이티브로 파이프라인, 스크립팅, CI/CD 통합이 자유롭습니다
- 터미널뿐 아니라 VS Code 확장, 데스크톱 앱, 웹 브라우저에서도 사용할 수 있습니다

한계:

- 기본 인터페이스가 터미널이라 시각적 UI에 익숙한 사용자에게는 진입장벽이 있습니다 (VS Code 확장으로 보완 가능)
- 자율적 작업 수행 시 비용이 상대적으로 높을 수 있습니다

적합한 사용자: 복잡한 아키텍처 결정, 자율적 작업 실행, 대규모 코드베이스 분석이 필요한 개발자

시장 현황 (2026년 초 기준)

개발자 선호도 조사에서 Claude Code는 "가장 좋아하는 AI 코딩 도구"로 **46%**의 선택을 받았습니다. Cursor가 19%, GitHub Copilot이 9%로 뒤를 이었습니다. 에이전틱 코딩에 대한 개발자들의 높은 관심을 반영하는 결과입니다.

실용적 추천: 두 가지 도구의 병행

실제로 많은 개발자들은 단일 도구만 사용하지 않습니다. 최적의 조합은 상황에 따라 도구를 구분하여 사용하는 것입니다:

- **Claude Code:** 자율적 작업 실행, 복잡한 문제 해결, 대규모 리팩토링, 디버깅
- **Copilot 또는 Cursor:** 인라인 자동완성, 코딩 중 빠른 질문, 시각적 diff 검토

이 책에서는 Claude Code에 집중하지만, 다른 도구와의 병행 사용을 배제하지 않습니다. 도구는 목적에 맞게 선택하는 것이 중요합니다.

| 1.5 이 책의 로드맵

이 책은 네 개의 파트(Part)로 구성되어 있습니다.

Part 1: Claude Code와의 첫 만남 (지금 읽고 있는 부분)

- AI 코딩 도구의 배경을 이해하고, Claude Code를 설치하고, 기본적인 사용법을 익힙니다.

Part 2: 핵심 기능 마스터

- 코드 읽기, 코드 쓰기, Git 연동, 프로젝트 메모리(CLAUDE.md) 등 네 가지 핵심 역량을 깊이 있게 학습합니다.

Part 3: 실전 프로젝트

- 웹 앱 구축, 레거시 코드 리팩토링, 버그 사냥과 디버깅 등 실제 개발 시나리오에서 Claude Code를 활용합니다.

Part 4: 고급 활용과 확장

- 설정 심화, MCP 서버 연동, Hooks 자동화, IDE 연동과 팀 협업 등 고급 주제를 다룹니다.

각 챕터는 학습 목표로 시작하여, 개념 설명, 실제 대화 예시, 실습, 그리고 정리로 마무리됩니다. 순서대로 읽는 것을 권장하지만, 이미 기본기가 있는 분은 관심 있는 챕터부터 읽어도 됩니다.

| 1.6 정리

이 챕터에서 배운 내용을 정리합니다.

- **AI 코딩 도구는 세 세대**를 거쳐 진화했습니다: 인라인 자동완성 → IDE 통합 어시스턴트 → 에이전틱 코딩
- **에이전틱 코딩**은 AI가 지시를 받으면 파일 읽기/쓰기, 명령 실행, 테스트, 커밋 등을 자율적으로 연쇄 수행하는 패러다임입니다
- **Claude Code**는 Anthropic이 만든 에이전틱 코딩 도구로, 터미널 네이티브, 코드베이스 인식, 자연어 인터페이스를 핵심 철학으로 합니다
- 경쟁 도구 대비 Claude Code의 강점은 **자율적 작업 실행 능력, 최대 1M 토큰 컨텍스트, 파이프라인/스크립팅 통합**입니다
- 2026년 초 기준 개발자 선호도 조사에서 Claude Code는 ****46%****로 1위를 차지했습니다

다음 챕터에서는 실제로 Claude Code를 설치하고 첫 대화를 나눠보겠습니다.

Chapter 2: 설치와 첫 대화

학습 목표

이 챕터를 마치면 다음을 할 수 있습니다:

- 1. 자신의 운영체제에 맞게 Claude Code를 설치하고 인증을 완료합니다
- 2. 첫 대화형 세션을 시작하여 기본 상호작용 흐름을 체험합니다
- 3. 권한 요청의 의미를 이해하고 적절히 응답합니다

2.1 시스템 요구사항 확인

Claude Code를 설치하기 전에 시스템 요구사항을 확인합니다.

운영체제

운영체제	지원 여부	비고
macOS	지원	Intel 및 Apple Silicon 모두 지원
Linux	지원	Ubuntu, Debian, Fedora 등 주요 배포판
Windows	지원	네이티브 및 WSL(Windows Subsystem for Linux) 모두 지원

필수 소프트웨어

- **Node.js 18 이상:** Claude Code의 일부 설치 방식에서 필요합니다. 버전을 확인하려면 터미널에서 다음 명령을 실행합니다:

```
node --version
# v20.11.0 과 같이 18 이상이면 됩니다
```

Node.js가 설치되어 있지 않거나 버전이 낮다면, nodejs.org에서 최신 LTS 버전을 설치합니다.

계정 요구사항

다음 중 하나의 계정이 필요합니다:

- **Claude 구독 (claude.ai)**: Pro, Max, Team 플랜 중 하나
- **Anthropic Console 계정**: API 키를 통한 종량제(Pay-as-you-go) 사용

아직 계정이 없다면 claude.ai에서 가입할 수 있습니다.

2.2 설치 방법 선택

Claude Code는 여러 가지 방법으로 설치할 수 있습니다. 상황에 맞는 방법을 선택하세요.

방법 1: 네이티브 설치 (권장)

가장 간단하고 권장되는 방법입니다. 설치 스크립트를 실행하면 자동으로 설치되며, 이후 백그라운드에서 자동 업데이트됩니다.

macOS / Linux / WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del  
install.cmd
```

설치가 완료되면 터미널을 새로 열거나 셸을 재시작한 뒤, `claude --version` 명령으로 설치를 확인합니다:

```
claude --version  
# Claude Code v1.x.x
```

팁: 네이티브 설치의 가장 큰 장점은 자동 업데이트입니다. Claude Code는 빠르게 발전하고 있으며, 자동 업데이트를 통해 항상 최신 기능과 버그 수정을 받을 수 있습니다.

방법 2: Homebrew (macOS / Linux)

macOS나 Linux에서 Homebrew를 사용하고 있다면:

```
brew install --cask claude-code
```

업데이트할 때는:

```
brew upgrade claude-code
```

방법 3: WinGet (Windows)

Windows에서 WinGet 패키지 관리자를 사용하고 있다면:

```
winget install Anthropic.ClaudeCode
```

업데이트할 때는:

```
winget upgrade Anthropic.ClaudeCode
```

방법 4: npm (더 이상 권장하지 않음)

npm을 통한 설치도 가능하지만, 현재는 권장하지 않습니다. 자동 업데이트가 되지 않으며, Node.js 환경에 의존합니다.

```
npm install -g @anthropic-ai/claude-code
```

어떤 방법을 선택할까?

상황	추천 방법
처음 설치하는 경우	네이티브 설치 (방법 1)
macOS에서 Homebrew를 이미 사용 중	Homebrew (방법 2)
Windows에서 WinGet을 이미 사용 중	WinGet (방법 3)
특별한 이유가 있는 경우	npm (방법 4)

2.3 인증과 계정 설정

Claude Code를 설치했다면, 이제 계정을 연결해야 합니다.

첫 실행 시 자동 로그인

가장 간단한 방법은 프로젝트 디렉터리에서 `claude` 명령을 실행하는 것입니다:

```
cd your-project
claude
```

첫 실행 시 자동으로 브라우저가 열리며 로그인 화면이 나타납니다. claude.ai 계정 또는 Anthropic Console 계정으로 로그인하면 됩니다.

명시적 로그인 명령

브라우저가 자동으로 열리지 않거나, 다른 계정으로 전환하고 싶다면 명시적으로 로그인할 수 있습니다:

```
# 기본 로그인
claude auth login

# SSO(Single Sign-On)를 사용하는 조직의 경우
claude auth login --email user@company.com --sso
```

인증 상태 확인

현재 로그인 상태를 확인하려면:

```
# JSON 형식으로 출력
claude auth status

# 읽기 쉬운 형태로 출력
claude auth status --text
```

출력 예시:

```
Account: user@example.com
Plan: Pro
Status: Active
Model access: sonnet, opus, haiku
```

로그아웃

다른 계정으로 전환하거나 보안상의 이유로 로그아웃할 때:

```
claude auth logout
```

서드파티 제공자를 통한 사용

Anthropic 직접 계정 외에도, 다음 클라우드 제공자를 통해 Claude Code를 사용할 수 있습니다:

- **Amazon Bedrock**: AWS 환경에서 Claude를 사용하는 경우
- **Google Vertex AI**: GCP 환경에서 Claude를 사용하는 경우
- **Microsoft Foundry**: Azure 환경에서 Claude를 사용하는 경우

기업 환경에서는 이미 사용 중인 클라우드 제공자를 통해 접근하는 것이 조직의 보안 정책과 결제 체계에 맞을 수 있습니다.

2.4 첫 세션 시작하기

설치와 인증이 완료되었다면, 이제 첫 대화를 시작할 시간입니다.

프로젝트 디렉터리로 이동

Claude Code는 현재 작업 디렉터리(Working Directory)를 기준으로 프로젝트를 인식합니다. 먼저 작업하고 싶은 프로젝트 디렉터리로 이동합니다:

```
cd ~/projects/my-app
```

만약 실습할 프로젝트가 없다면, 간단한 Node.js 프로젝트를 하나 만들어봅시다:

```
mkdir ~/projects/hello-claude  
cd ~/projects/hello-claude  
npm init -y  
echo "console.log('Hello, Claude!');" > index.js
```

대화형 세션 시작

```
claude
```

이 명령을 실행하면 Claude Code의 대화형 인터페이스가 열립니다. 화면에는 다음과 같은 프롬프트가 나타납니다:

```
Claude Code  
|  
| /help for commands, /exit to quit  
|  
>
```

> 프롬프트가 나타나면 자연어로 질문이나 지시를 입력할 수 있습니다.

첫 대화: 프로젝트 탐색

가장 먼저 해볼 수 있는 좋은 질문은 프로젝트에 대한 설명을 요청하는 것입니다:

사용자 > 이 프로젝트의 구조를 설명해줘

Claude > 이 프로젝트를 분석해보겠습니다.

현재 프로젝트는 간단한 Node.js 프로젝트입니다:

```
hello-claude/  
├── index.js      # 메인 엔트리 파일 - "Hello, Claude!" 출력  
└── package.json  # npm 패키지 설정 파일
```

index.js는 콘솔에 "Hello, Claude!"를 출력하는 한 줄짜리 스크립트입니다. package.json에는 기본 프로젝트 메타데이터가 포함되어 있으며, 아직 외부 의존성(dependency)은 없습니다.

코드 수정 요청하기

이번에는 코드를 수정해달라고 요청해봅시다:

사용자 > index.js에 현재 시간을 출력하는 기능을 추가해줘

Claude > index.js를 수정하겠습니다.

```
Edit index.js  
  
+ const now = new Date();  
+ const timeString = now.toLocaleString('ko-KR', {  
+   timeZone: 'Asia/Seoul'  
+ });  
+ console.log('Hello, Claude!');  
+ console.log(`현재 시간: ${timeString}`);  
  
이 파일을 수정해도 될까요? [Y/n]
```

여기서 Claude Code가 **권한을 요청**합니다. 이것이 Claude Code의 중요한 안전 장치입니다.

명령 실행 요청하기

코드를 수정한 후 실행해달라고 요청할 수도 있습니다:

사용자 > 수정한 코드를 실행해봐

Claude > 다음 명령을 실행하겠습니다:

Bash: node index.js

이 명령을 실행해도 될까요? [Y/n]

Y 를 입력하면:

Claude > Hello, Claude!

현재 시간: 2026. 3. 14. 오후 2:30:45

코드가 정상적으로 실행되었습니다. "Hello, Claude!"와 함께
현재 한국 시간이 출력됩니다.

2.5 권한 시스템 이해하기

앞선 예시에서 Claude Code가 파일 수정과 명령 실행 전에 권한을 요청하는 것을 보았습니다. 이것은 Claude Code의 핵심적인 안전 장치입니다.

왜 권한을 요청할까?

Claude Code는 에이전틱 도구입니다. 파일을 읽고, 쓰고, 삭제하고, 임의의 셸 명령을 실행할 수 있습니다. 이런 강력한 능력에는 그에 맞는 통제가 필요합니다. Claude Code는 기본적으로 다음 작업 전에 사용자의 승인을 요청합니다:

- **파일 수정(Edit/Write):** 기존 파일을 변경하거나 새 파일을 생성할 때
- **명령 실행(Bash):** 셸 명령을 실행할 때
- **웹 접근(WebFetch):** 외부 URL에 접근할 때

반면, 다음 작업은 승인 없이 수행합니다:

- **파일 읽기(Read):** 프로젝트 내 파일 내용을 읽을 때
- **코드 검색(Grep/Glob):** 코드베이스 내에서 패턴을 검색할 때

권한 응답 방법

권한 요청에는 다음과 같이 응답할 수 있습니다:

응답	의미
<code>Y</code> 또는 Enter	이번 요청을 승인합니다
<code>n</code>	이번 요청을 거부합니다
<code>a</code> (Always)	이 세션에서 같은 유형의 요청을 항상 승인합니다

네 가지 권한 모드

Claude Code는 네 가지 권한 모드를 제공합니다. 작업의 성격과 신뢰도에 따라 선택할 수 있습니다.

모드	설명	추천 상황
<code>default</code>	매 작업마다 개별 승인	처음 사용할 때, 중요한 프로젝트
<code>plan</code>	전체 계획을 먼저 제시, 승인 후 실행	대규모 변경 작업
<code>acceptEdits</code>	파일 편집은 자동 승인, 명령은 묻기	코드 편집 위주 작업
<code>dontAsk</code>	대부분의 작업을 자동 승인	반복적인 자동화 작업

권한 모드는 시작 시 지정하거나 세션 중에 변경할 수 있습니다:

```
# 시작 시 지정
claude --permission-mode plan
```

주의: 처음에는 `default` 모드를 사용하는 것을 권장합니다. Claude Code의 동작 방식에 익숙해진 후 필요에 따라 다른 모드로 전환하세요.

2.6 초기 프롬프트와 비대화형 모드

대화형 세션 외에도 Claude Code를 실행하는 다양한 방법이 있습니다.

초기 프롬프트로 세션 시작

대화형 세션을 시작하면서 첫 질문을 함께 전달할 수 있습니다:

```
claude "이 프로젝트의 구조를 설명해줘"
```

이렇게 하면 대화형 세션이 열리면서 바로 해당 질문에 대한 답변이 시작됩니다. 이후 추가 질문을 이어갈 수 있습니다.

비대화형 모드 (`-p` / `--print`)

한 번의 질문에 답변만 받고 종료하는 모드입니다. 스크립트나 파이프라인에서 활용할 때 유용합니다:

```
claude -p "package.json의 의존성 목록을 정리해줘"
```

이 모드에서는 답변을 출력한 후 자동으로 종료됩니다. 대화를 이어가지 않습니다.

파이프(Pipe) 입력

다른 명령의 출력을 Claude Code에게 전달할 수 있습니다:

```
# 파일 내용을 전달하여 분석 요청
cat error.log | claude -p "이 로그에서 에러 원인을 분석해줘"

# git diff 결과를 전달하여 리뷰 요청
git diff | claude -p "이 변경 사항을 리뷰해줘"
```

이것이 바로 Claude Code가 "터미널 네이티브"라고 불리는 이유입니다. 기존의 Unix 도구들과 자연스럽게 조합됩니다.

2.7 업데이트와 버전 관리

Claude Code는 빠르게 발전하는 도구입니다. 최신 기능과 버그 수정을 받기 위해 주기적으로 업데이트하는 것이 좋습니다.

업데이트 방법

```
# 네이티브 설치의 경우 (자동 업데이트 지원)
claude update

# Homebrew의 경우
brew upgrade claude-code

# WinGet의 경우
winget upgrade Anthropic.ClaudeCode
```

현재 버전 확인

```
claude --version
```

자동 업데이트 채널 설정

네이티브 설치를 사용하는 경우, 업데이트 채널을 선택할 수 있습니다:

- **stable** : 안정적인 릴리스만 받습니다 (기본값)
- **latest** : 최신 기능을 빠르게 받지만 불안정할 수 있습니다

설정 파일(`~/.claude/settings.json`)에서 변경할 수 있습니다:

```
{
  "autoUpdatesChannel": "stable"
}
```

| 2.8 문제 발생 시 진단

설치나 실행 중 문제가 발생했다면 다음 도구를 활용합니다.

/doctor 명령

대화형 세션 내에서 설치 상태와 설정을 진단합니다:

사용자 > /doctor

Claude > 진단 결과:

- ✓ Claude Code v1.x.x
- ✓ Node.js v20.11.0
- ✓ 인증: user@example.com (Pro)
- ✓ 네트워크 연결: 정상
- ✓ 모델 접근: sonnet, opus, haiku

디버그 모드

상세한 디버그 로그를 확인하려면:

```
claude --debug "api,mcp"
```

--verbose 플래그

더 자세한 출력을 보려면:

```
claude --verbose
```

자주 발생하는 문제와 해결

문제	원인	해결 방법
command not found: claude	PATH에 등록되지 않음	터미널을 새로 열거나, 셸 설정 파일을 source
로그인 페이지가 열리지 않음	브라우저 문제	claude auth login 명령을 수동으로 실행
"Rate limit exceeded"	API 호출 한도 초과	잠시 후 다시 시도하거나, 플랜 업그레이드 검토
느린 응답	네트워크 또는 모델 부하	네트워크 확인, 또는 더 가벼운 모델(haiku) 사용

| 2.9 실습: 첫 대화 나눠보기

이제 직접 해봅시다. 아래 순서를 따라 Claude Code와 첫 대화를 나눠보세요.

실습 1: 프로젝트 탐색

기존 프로젝트 또는 앞서 만든 `hello-claude` 프로젝트에서:

```
cd ~/projects/hello-claude
claude
```

다음 질문들을 차례로 해보세요:

- > 이 프로젝트에 어떤 파일들이 있어?
- > `package.json`의 내용을 설명해줘
- > `index.js`를 실행하면 어떤 결과가 나와?

실습 2: 코드 수정 체험

- > `index.js`에 커맨드라인 인자로 이름을 받아서 인사하는 기능을 추가해줘

Claude Code가 파일 수정 권한을 요청하면 `Y`로 승인합니다. 그런 다음:

- > 수정한 코드를 "홍길동"이라는 이름으로 실행해봐

실습 3: 파이프라인 체험

대화형 세션을 `/exit`로 종료한 후, 비대화형 모드를 체험합니다:

```
cat package.json | claude -p "이 package.json을 분석하고 개선 사항을 제안해줘"
```

| 2.10 정리

이 챕터에서 배운 내용을 정리합니다.

- Claude Code는 **네이티브 설치, Homebrew, WinGet, npm** 네 가지 방법으로 설치할 수 있으며, 네이티브 설치가 권장됩니다
- 설치 후 `claude` 명령으로 첫 실행하면 **브라우저를 통한 자동 로그인**이 진행됩니다
- `claude auth login` 으로 수동 로그인, `claude auth status` 로 상태 확인이 가능합니다
- **대화형 세션**은 `claude` 명령으로 시작하며, 자연어로 질문과 지시를 할 수 있습니다
- Claude Code는 파일 수정과 명령 실행 전에 **권한을 요청**합니다 (안전 장치)
- **네 가지 권한 모드**(default, plan, acceptEdits, dontAsk)로 승인 수준을 조절할 수 있습니다
- `claude -p "질문"` 형태의 **비대화형 모드**와 **파이프 입력**으로 다른 도구와 연동할 수 있습니다
- `claude update` 로 업데이트하고, `/doctor` 로 문제를 진단할 수 있습니다

다음 챕터에서는 Claude Code를 더 효과적으로 사용하기 위한 기본기를 다져보겠습니다.

Chapter 3: 기본기 다지기

학습 목표

이 챕터를 마치면 다음을 할 수 있습니다:

1. 대화형 모드와 비대화형 모드의 차이를 이해하고 상황에 맞게 사용합니다
2. 슬래시 명령어를 활용하여 세션을 효율적으로 관리합니다
3. 모델 선택과 비용 구조를 이해하여 합리적으로 사용합니다

3.1 대화형 모드 vs 비대화형 모드

Claude Code에는 두 가지 기본 실행 모드가 있습니다. 상황에 따라 적절한 모드를 선택하는 것이 효율적인 사용의 첫걸음입니다.

대화형 모드 (Interactive Mode)

```
claude
```

대화형 모드는 Claude Code와 지속적인 대화를 나누는 모드입니다. 한 번 시작하면 `/exit` 으로 종료할 때까지 대화가 이어집니다. 이전 질문과 답변의 맥락이 유지되므로, 복잡한 작업을 단계적으로 수행할 때 적합합니다.

적합한 상황:

- 프로젝트를 탐색하고 이해할 때
- 여러 단계에 걸친 기능 구현을 할 때
- 대화를 통해 요구사항을 점진적으로 구체화할 때
- 코드 리뷰나 리팩토링처럼 반복적인 피드백이 필요할 때

사용 예시:

사용자 > 이 프로젝트의 인증 모듈 구조를 설명해줘

Claude > [인증 모듈 구조 설명...]

사용자 > 그 중에서 JWT 토큰 검증 부분에 버그가 있는 것 같아.
만료 시간 체크가 제대로 안 되는 것 같은데 확인해줘

Claude > [JWT 토큰 검증 코드를 분석하고 버그를 식별...]

사용자 > 찾아낸 버그를 수정하고, 관련 테스트도 추가해줘

Claude > [버그 수정 및 테스트 코드 작성...]

이처럼 대화의 맥락이 이어지면서 점점 구체적인 작업으로 진행할 수 있습니다.

비대화형 모드 (Non-Interactive Mode / Print Mode)

```
claude -p "질문 또는 지시"
```

비대화형 모드는 하나의 질문에 답변을 출력하고 종료하는 모드입니다. 스크립트, 파이프라인, 자동화 작업에 적합합니다.

적합한 상황:

- 간단한 질문에 빠른 답변이 필요할 때
- 셸 스크립트나 CI/CD 파이프라인에서 Claude Code를 사용할 때
- 다른 명령의 출력을 분석할 때
- 일회성 작업을 실행할 때

사용 예시:


```
# 간단한 질문
claude -p "JavaScript에서 배열을 뒤집는 방법은?"

# 파일 분석
claude -p "src/auth/jwt.js 파일의 보안 취약점을 분석해줘"

# 파이프라인과 결합
git log --oneline -10 | claude -p "최근 10개 커밋의 패턴을 분석해줘"

# CI/CD에서 코드 리뷰
git diff main | claude -p "이 변경 사항에서 잠재적 버그를 찾아줘"
```

두 모드의 비교

특성	대화형 모드	비대화형 모드
명령	<code>claude</code>	<code>claude -p "query"</code>
대화 지속	계속 이어짐	한 번 답변 후 종료
맥락 유지	이전 대화 기억	매번 새로 시작
파이프 입력	불가	가능
스크립팅	부적합	적합
권한 요청	대화 중 승인	작업에 따라 다름

실전 팁: 초기 프롬프트가 있는 대화형 모드

두 모드의 장점을 결합할 수도 있습니다. 초기 프롬프트를 제공하면서 대화형 세션을 시작하는 방법입니다:

```
claude "인증 모듈의 구조를 분석하고 개선 방안을 제안해줘"
```

이렇게 하면 첫 질문에 대한 답변이 나온 후, 이어서 추가 질문을 할 수 있습니다. `-p` 플래그가 없으므로 대화형 세션이 유지됩니다.

3.2 효과적인 프롬프트 작성법

Claude Code를 잘 사용하는 핵심은 좋은 프롬프트(Prompt)를 작성하는 것입니다. 동료 개발자에게 업무를 요청할 때와 마찬가지로, 명확하고 구체적인 지시가 좋은 결과를 만듭니다.

원칙 1: 구체적으로 지시하기

```
# 나쁜 예
> 코드를 정리해줘

# 좋은 예
> src/auth/ 디렉터리에서 사용하지 않는 import를 모두 제거하고,
  ESLint 에러를 수정해줘
```

```
# 나쁜 예
> 테스트를 만들어줘

# 좋은 예
> src/utils/validator.js의 validateEmail 함수에 대한 단위 테스트를
  Jest로 작성해줘. 유효한 이메일, 빈 문자열, @가 없는 문자열,
  도메인이 없는 경우를 테스트해줘
```

원칙 2: 에러 메시지를 그대로 붙여넣기

버그를 수정할 때는 에러 메시지를 정확히 전달하는 것이 중요합니다. 에러를 요약하거나 해석하지 말고, 그대로 붙여넣으세요.

```
> npm run build를 실행하면 다음 에러가 발생합니다:

ERROR in src/components/UserProfile.tsx:23:5
TS2339: Property 'email' does not exist on type 'User'.

이 에러를 수정해줘.
```

Claude Code는 에러 메시지의 파일 경로, 줄 번호, 에러 코드를 활용하여 정확한 위치의 문제를 파악합니다.

원칙 3: @ 멘션으로 컨텍스트 제공하기

특정 파일이나 디렉토리를 참조할 때는 @ 멘션을 사용합니다. Claude Code가 해당 파일을 우선적으로 읽어 더 정확한 답변을 제공합니다.

```
> @src/auth/jwt.js 파일에서 토큰 만료 검증 로직을 확인하고,  
@src/middleware/auth.js 에서 이 함수를 어떻게 사용하는지 분석해줘
```

```
> @src/components/ 디렉터리의 모든 React 컴포넌트에서  
deprecated된 lifecycle 메서드를 hooks로 변환해줘
```

원칙 4: Plan 모드로 대규모 작업 시작하기

2개 이상의 파일에 영향을 주는 작업은 먼저 계획을 세우는 것이 좋습니다:

```
> /plan 인증 시스템을 세션 기반에서 JWT 기반으로 전환하려고 합니다.  
어떤 파일들을 수정해야 하고, 어떤 순서로 진행해야 할까요?
```

Claude Code가 전체 계획을 제시하면, 검토 후 실행을 승인할 수 있습니다.

원칙 5: 이전 대화의 맥락 활용하기

대화형 모드에서는 이전 대화의 맥락이 유지됩니다. "그것", "방금 수정한 파일", "아까 말한 함수" 같은 대명사를 자연스럽게 사용할 수 있습니다.

```
사용자 > validateEmail 함수를 보여줘  
Claude > [함수 내용 표시...]  
  
사용자 > 그 함수에 한국 전화번호 형식도 검증하는 기능을 추가해줘  
Claude > [함수 수정...]  
  
사용자 > 방금 수정한 내용에 대한 테스트를 작성해줘  
Claude > [테스트 코드 작성...]
```

3.3 핵심 슬래시 명령어

슬래시 명령어(Slash Command)는 대화형 모드에서 Claude Code의 기능을 제어하는 특별한 명령어입니다. 프롬프트에 `/`를 입력하면 사용할 수 있습니다.

세션 관리 명령어

`/clear` - 대화 기록 지우기

```
> /clear
```

현재 대화 기록을 모두 지웁니다. 새로운 주제로 전환하거나, 컨텍스트를 초기화하고 싶을 때 사용합니다. `/reset`, `/new`로도 사용할 수 있습니다.

언제 사용하나?

- 완전히 다른 작업을 시작할 때
- Claude Code가 이전 대화에 혼란스러워할 때
- 컨텍스트 윈도우를 최대한 확보하고 싶을 때

`/compact` - 대화 압축

```
> /compact
> /compact auth 모듈 관련 내용만 유지해줘
```

현재까지의 대화를 압축하여 핵심 내용만 유지합니다. 긴 대화를 나눈 후 컨텍스트 윈도우가 부족해질 때 유용합니다. 선택적으로 어떤 내용에 집중할지 지시를 추가할 수 있습니다.

언제 사용하나?

- 대화가 길어져서 응답이 느려질 때
- "컨텍스트 한도에 가까워지고 있습니다" 경고가 나타날 때
- 특정 주제에 집중하고 싶을 때

`/resume` - 이전 세션 이어가기

```
> /resume
```

이전에 종료한 세션 목록이 나타나고, 선택하여 이어갈 수 있습니다. `/continue` 로도 사용할 수 있습니다.

CLI에서도 세션을 이어갈 수 있습니다:

```
# 가장 최근 세션 이어가기
claude -c

# 특정 세션 ID 또는 이름으로 이어가기
claude -r "auth-refactor"
claude -r "session-id-here" "이어서 PR을 만들어줘"
```

언제 사용하나?

- 어제 하던 작업을 오늘 이어서 할 때
- 세션을 잘못 종료했을 때
- 이전 대화의 맥락이 필요한 작업을 할 때

`/fork` - 대화 분기

```
> /fork 대안-접근법
```

현재 대화의 복사본을 만들어 별도의 방향으로 진행합니다. 다른 접근 방식을 실험하고 싶을 때 유용합니다.

`/rename` - 세션 이름 변경

```
> /rename auth-module-refactor
```

현재 세션에 의미 있는 이름을 붙여서 나중에 쉽게 찾을 수 있게 합니다.

`/exit` - 세션 종료

```
> /exit
```

현재 세션을 종료합니다. `/quit` 으로도 사용할 수 있습니다. 세션은 자동으로 저장되므로 나중에 `/resume` 이나 `claude -c` 로 이어갈 수 있습니다.

정보 확인 명령어

/help - 도움말

```
> /help
```

사용 가능한 모든 슬래시 명령어와 간단한 설명을 표시합니다.

/status - 상태 확인

```
> /status
```

현재 세션의 상태를 표시합니다: 버전, 사용 중인 모델, 계정 정보, 연결 상태 등.

/cost - 비용 확인

```
> /cost
```

현재 세션에서 사용한 토큰 수와 예상 비용을 표시합니다. 비용을 모니터링하고 싶을 때 수시로 확인합니다.

출력 예시:

```
세션 토큰 사용량:
입력:  45,230 토큰
출력:  12,450 토큰
합계:  57,680 토큰
```

/usage - 플랜 사용량 확인

```
> /usage
```

현재 구독 플랜의 사용 한도와 남은 용량을 표시합니다. 속도 제한(Rate Limit) 상태도 확인할 수 있습니다.

/context - 컨텍스트 사용량

```
> /context
```

현재 대화에서 사용 중인 컨텍스트 윈도우의 크기를 시각화합니다. 얼마나 많은 공간이 남아 있는지 확인할 수 있습니다.

코드 작업 명령어

/init - 프로젝트 초기화

```
> /init
```

Claude Code가 프로젝트를 분석하여 CLAUDE.md 파일을 자동 생성합니다. 새 프로젝트를 시작할 때 가장 먼저 실행하면 좋습니다. CLAUDE.md에 대해서는 Part 2에서 자세히 다룹니다.

/diff - 변경 사항 확인

```
> /diff
```

현재 커밋되지 않은 변경 사항을 인터랙티브하게 보여줍니다. Claude Code가 수정한 파일들의 전후 비교를 편리하게 검토할 수 있습니다.

/plan - 계획 모드

```
> /plan 인증 시스템을 OAuth 2.0으로 전환하려고 합니다
```

Claude Code가 곧바로 코드를 수정하지 않고, 먼저 전체 계획을 제시합니다. 대규모 변경 작업 전에 사용하면 예상치 못한 문제를 미리 파악할 수 있습니다.

/rewind - 되돌리기

```
> /rewind
```

대화 및/또는 코드 변경 사항을 이전 시점으로 되돌립니다. Claude Code가 예상과 다른 변경을 했을 때 유용합니다.
`/checkpoint` 로도 사용할 수 있습니다.

기타 유용한 명령어

`/model` - 모델 변경

```
> /model opus  
> /model sonnet  
> /model haiku
```

사용 중인 AI 모델을 즉시 변경합니다. 다음 섹션에서 자세히 다룹니다.

`/effort` - 노력 수준 변경

```
> /effort high  
> /effort low
```

Claude Code의 사고 깊이를 조절합니다. 다음 섹션에서 자세히 다룹니다.

`/copy` - 마지막 응답 복사

```
> /copy
```

Claude Code의 마지막 응답을 클립보드에 복사합니다.

`/export` - 대화 내보내기

```
> /export conversation.txt
```

현재까지의 대화를 텍스트 파일로 내보냅니다. 기록 보관이나 팀과 공유할 때 유용합니다.

3.4 모델 선택 전략

Claude Code에서는 여러 AI 모델을 선택할 수 있습니다. 각 모델은 능력, 속도, 비용이 다르므로, 작업의 성격에 맞는 모델을 선택하는 것이 중요합니다.

사용 가능한 모델

별칭	모델	특징	적합한 작업
haiku	Haiku	가장 빠르고 저렴	간단한 질문, 코드 포매팅, 문자열 변환
sonnet	Sonnet 4.6	속도와 능력의 균형	일반적인 코딩, 기능 구현, 테스트 작성
opus	Opus 4.6	가장 강력한 추론 능력	복잡한 아키텍처 결정, 디버깅, 대규모 리팩토링
opusplan	Opus + Sonnet	하이브리드	계획은 Opus, 실행은 Sonnet으로 자동 전환

모델 변경 방법

세션 중에 모델을 변경할 수 있습니다:

```
> /model opus
모델이 opus로 변경되었습니다.
```

시작할 때 지정할 수도 있습니다:

```
claude --model opus
```

환경 변수로 기본 모델을 설정할 수도 있습니다:

```
export ANTHROPIC_MODEL=opus
```

설정 파일(`~/.claude/settings.json`)에서 기본값을 지정할 수도 있습니다:

```
{  
  "model": "sonnet"  
}
```

각 모델의 상세 특징

Haiku - 빠르고 가벼운 모델

Haiku는 응답 속도가 가장 빠르고 비용이 가장 저렴한 모델입니다. 복잡한 추론이 필요하지 않은 간단한 작업에 적합합니다.

```
사용자 > /model haiku  
사용자 > "Hello World"를 10가지 프로그래밍 언어로 보여줘  
  
Claude > [빠르게 10개 언어의 Hello World 코드를 나열...]
```

추천 사용 상황: 코드 포매팅, 간단한 변환, 문법 질문, 빠른 정보 조회

Sonnet 4.6 - 균형 잡힌 모델

Sonnet은 속도와 능력의 균형이 가장 좋은 모델입니다. 일상적인 코딩 작업의 대부분을 처리할 수 있으며, Pro와 Team Standard 플랜의 기본 모델입니다.

```
사용자 > /model sonnet  
사용자 > Express.js로 사용자 CRUD API를 만들어줘.  
        입력 검증과 에러 처리를 포함해줘.  
  
Claude > [REST API 코드를 생성하고, 라우터, 컨트롤러, 검증 미들웨어를  
        각각의 파일로 분리하여 작성...]
```

추천 사용 상황: 기능 구현, 테스트 작성, 코드 리뷰, 일반적인 버그 수정

Opus 4.6 - 최강의 추론 모델

Opus는 가장 강력한 추론 능력을 가진 모델입니다. 복잡한 아키텍처 결정, 까다로운 디버깅, 대규모 리팩토링에서 진가를 발휘합니다. Max와 Team Premium 플랜의 기본 모델입니다.

```
사용자 > /model opus
사용자 > 현재 모놀리식 아키텍처를 마이크로서비스로 전환하려고 합니다.
        현재 코드베이스를 분석하고, 서비스 경계를 제안하고,
        단계별 전환 계획을 세워줘.

Claude > [코드베이스 전체를 분석하고, 도메인 경계를 식별하며,
        의존성 그래프를 고려한 단계별 전환 계획을 수립...]
```

추천 사용 상황: 아키텍처 설계, 복잡한 디버깅, 대규모 리팩토링, 보안 분석

opusplan - 하이브리드 전략

opusplan 은 계획 수립 단계에서는 Opus를, 실제 코드 실행 단계에서는 Sonnet을 사용하는 하이브리드 모델입니다. 비용과 품질의 균형을 자동으로 맞춰줍니다.

```
claude --model opusplan
```

추천 사용 상황: 중간 규모의 기능 구현, 여러 파일에 걸친 변경, 비용 효율적인 복잡한 작업

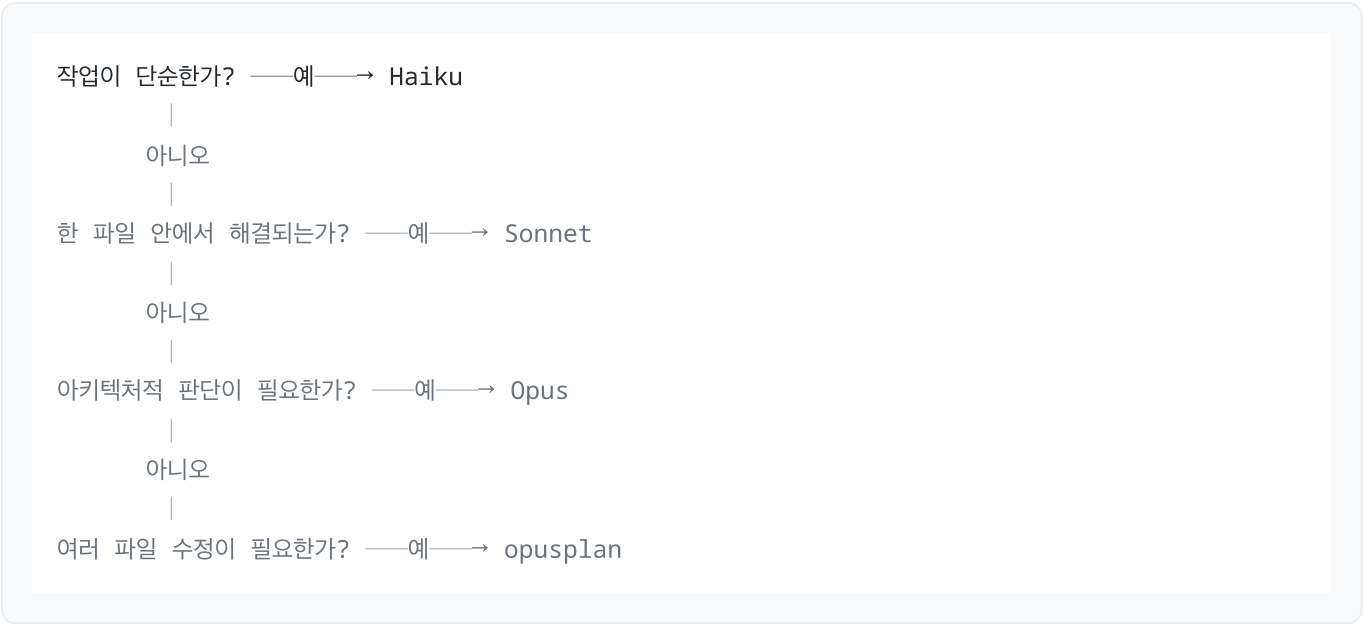
확장 컨텍스트 (1M 토큰)

Sonnet과 Opus 모두 1백만 토큰 확장 컨텍스트를 지원합니다. 별칭에 **[1m]** 을 붙여 사용합니다:

```
> /model sonnet[1m]
> /model opus[1m]
```

대규모 코드베이스를 분석하거나, 매우 긴 대화가 필요한 작업에서 활용합니다. 다만 플랜에 따라 추가 비용이 발생할 수 있습니다.

모델 선택 의사결정 가이드



3.5 노력 수준 (Effort Level)

모델 선택과 별개로, Claude Code의 **사고 깊이**를 조절할 수 있습니다. 이것이 노력 수준(Effort Level)입니다. 적응형 추론(Adaptive Reasoning)을 제어하여, 작업의 복잡도에 따라 AI의 사고 깊이를 동적으로 배분합니다.

노력 수준 종류

수준	설명	적합한 작업	지속성
low	빠르고 간결한 응답	간단한 질문, 형식 변환	세션 간 유지
medium	기본 균형 (기본값)	일반적인 코딩 작업	세션 간 유지
high	깊고 신중한 추론	복잡한 로직, 디버깅	세션 간 유지
max	최대 깊이 사고	극히 복잡한 문제 (Opus 4.6 전용)	현재 세션만

노력 수준 변경 방법

```
# 세션 중 변경
> /effort high

# CLI에서 지정
claude --effort high

# 환경 변수로 기본값 설정
export CLAUDE_CODE_EFFORT_LEVEL=high
```

실전 예시: 같은 질문, 다른 노력 수준

low 수준:

```
사용자 > /effort low
사용자 > JavaScript에서 배열 중복 제거 방법은?

Claude > [...new Set(array)]
```

빠르고 간결합니다. 코드 한 줄로 답합니다.

high 수준:

사용자 > /effort high

사용자 > JavaScript에서 배열 중복 제거 방법은?

Claude > JavaScript에서 배열의 중복 요소를 제거하는 방법은 여러 가지가 있습니다. 상황에 따라 최적의 방법이 다릅니다.

1. Set 활용 (가장 간단, ES6+):

```
const unique = [...new Set(array)];
```

2. filter + indexOf (레거시 호환):

```
const unique = array.filter((item, index) =>
  array.indexOf(item) === index
);
```

3. 객체 기반 (객체 배열의 특정 속성으로 중복 제거):

```
const unique = array.filter((item, index, self) =>
  index === self.findIndex(t => t.id === item.id)
);
```

4. Map 활용 (객체 배열, 더 효율적):

```
const unique = [...new Map(
  array.map(item => [item.id, item])
).values()];
```

성능 비교:

- Set: $O(n)$ - 원시 타입에 가장 효율적
- filter+indexOf: $O(n^2)$ - 작은 배열에만 적합
- Map: $O(n)$ - 객체 배열에 적합

권장: 원시 타입 배열이면 Set, 객체 배열이면 Map을 사용하세요.

같은 질문이지만 훨씬 깊고 포괄적인 답변을 제공합니다.

노력 수준 사용 팁

- 기본은 **medium** 을 유지하고, 작업 복잡도에 따라 조절합니다
- **low** : 형식 변환, 간단한 코드 생성, 빠른 정보 조회에 사용합니다
- **high** : 디버깅, 아키텍처 결정, 보안 분석에 사용합니다
- **max** : Opus 4.6에서만 사용 가능하며, 정말 복잡한 문제에만 사용합니다. 현재 세션에서만 유지되며 비용이 높습니다

3.6 비용 이해와 관리

Claude Code는 AI 모델에 요청을 보내는 만큼 비용이 발생합니다. 비용 구조를 이해하고 관리하는 것은 효율적인 사용의 핵심입니다.

비용 발생 구조

Claude Code의 비용은 **토큰(Token)** 단위로 계산됩니다. 토큰은 AI 모델이 텍스트를 처리하는 최소 단위로, 영어 기준 약 4글자, 한국어 기준 약 1~2글자에 해당합니다.

비용은 두 가지 방향에서 발생합니다:

- **입력 토큰**: 사용자의 질문, 파일 내용, 이전 대화 맥락 등 모델에 전달되는 모든 텍스트
- **출력 토큰**: Claude Code가 생성하는 답변, 코드, 설명 등

구독 플랜별 사용 방식

플랜	기본 모델	비용 방식
Pro	Sonnet 4.6	월정액 구독, 사용량 한도 내
Max	Opus 4.6	월정액 구독, 더 높은 사용량 한도
Team Standard	Sonnet 4.6	팀 월정액, 팀원별 한도
Team Premium	Opus 4.6	팀 월정액, 더 높은 한도
API (Pay-as-you-go)	선택 가능	사용한 토큰만큼 종량제 과금

비용 모니터링 방법

세션 내 비용 확인

```
> /cost
```

현재 세션에서 사용한 토큰 수와 예상 비용을 보여줍니다.

플랜 사용량 확인

```
> /usage
```

구독 플랜의 전체 사용량과 남은 한도를 확인합니다. 속도 제한(Rate Limit)에 걸리고 있는지도 알 수 있습니다.

일일 통계 확인

```
> /stats
```

일일 사용량, 세션 히스토리, 모델별 사용 비율을 시각적으로 보여줍니다.

비용 절감 전략

Claude Code를 현명하게 사용하면 비용을 크게 줄일 수 있습니다. 다음은 실전에서 검증된 전략들입니다.

1. 적절한 모델 선택

모든 작업에 Opus를 사용할 필요는 없습니다. 작업 복잡도에 맞는 모델을 선택합니다:

```
# 간단한 형식 변환
> /model haiku
> 이 JSON을 YAML로 변환해줘

# 일반적인 코딩
> /model sonnet
> 사용자 목록을 페이지네이션하는 API를 만들어줘

# 복잡한 아키텍처 결정
> /model opus
> 마이크로서비스 전환 계획을 세워줘
```

2. **/compact** 로 컨텍스트 압축

대화가 길어지면 이전 대화 내용이 계속 입력 토큰으로 소비됩니다. 주기적으로 **/compact** 를 사용하여 불필요한 맥락을 정리합니다:


```
> /compact auth 모듈 관련 변경 사항만 유지해줘
```

3. 비대화형 모드에서 예산 제한

자동화 작업에서는 최대 비용을 제한할 수 있습니다:

```
# 최대 5달러까지만 사용
claude -p --max-budget-usd 5.00 "인증 모듈을 리팩토링해줘"

# 최대 3턴까지만 실행
claude -p --max-turns 3 "이 버그를 수정해줘"
```

4. 노력 수준 조절

단순한 작업에는 낮은 노력 수준을 사용합니다:

```
> /effort low
> 이 변수 이름을 camelCase로 변환해줘
```

5. @ 멘션으로 범위 한정

프로젝트 전체를 스캔하지 않고, 관련 파일만 명시적으로 지정합니다:

```
# 넓은 범위 (비용 높음)
> 이 프로젝트에서 보안 취약점을 찾아줘

# 좁은 범위 (비용 낮음)
> @src/auth/login.js 에서 SQL 인젝션 취약점이 있는지 확인해줘
```

6. CLAUDE.md 파일 최적화

CLAUDE.md 파일은 매 세션마다 로드됩니다. 200줄 이하로 유지하여 세션마다 소비하는 입력 토큰을 최소화합니다.

비용 관련 실전 대화 예시

사용자 > /cost

Claude > 현재 세션 토큰 사용량:

입력 토큰: 23,450

출력 토큰: 8,230

총 토큰: 31,680

컨텍스트 윈도우 사용률: 15%

사용자 > /usage

Claude > 플랜: Pro

이번 달 사용량: 65%

속도 제한 상태: 정상

모델별 사용량:

sonnet: 45%

opus: 15%

haiku: 5%

3.7 세션 관리 고급 기법

일상적인 개발에서 세션을 효과적으로 관리하는 방법을 알아봅니다.

세션 이름 지정

세션에 의미 있는 이름을 붙이면 나중에 쉽게 찾을 수 있습니다:

```
# 시작할 때 이름 지정
claude -n "auth-module-refactor"
```

```
# 세션 중 이름 변경
> /rename payment-integration
```

세션 이어가기

어제 하던 작업을 오늘 이어서 할 수 있습니다:

```
# 가장 최근 세션 이어가기
claude -c

# 특정 세션을 이름으로 찾아 이어가기
claude -r "auth-module-refactor"

# 이어가면서 새 질문 추가
claude -r "auth-module-refactor" "어제 작업을 이어서 PR을 만들어줘"
```

세션 분기

현재 대화를 분기하여 다른 접근 방식을 실험할 수 있습니다:

```
사용자 > /fork redis-approach
```

이렇게 하면 현재까지의 대화가 복사되고, 새로운 분기에서 다른 방향으로 작업을 진행할 수 있습니다. 원래 세션은 그대로 유지됩니다.

대화 내보내기

대화 내용을 파일로 저장하여 팀과 공유하거나 기록으로 보관할 수 있습니다:

```
> /export auth-refactor-discussion.txt
```

3.8 파이프라인과 자동화 활용

Claude Code가 터미널 네이티브 도구인 만큼, Unix 파이프라인과의 결합은 강력한 활용법입니다.

파이프 입력 패턴

다른 명령의 출력을 Claude Code에 전달합니다:

```
# 로그 파일 분석
tail -f app.log | claude -p "이상 징후를 감지하면 알려줘"

# git diff를 활용한 코드 리뷰
git diff main --name-only | claude -p "변경된 파일들의 보안 이슈를 검토해줘"

# 에러 로그 분석
cat error.log | claude -p "가장 빈번한 에러 패턴을 분류해줘"
```

구조화된 출력

비대화형 모드에서 출력 형식을 지정할 수 있습니다:

```
# JSON 형식으로 출력
claude -p --output-format json "src/ 디렉터리의 TODO 주석을 모두 찾아줘"

# 텍스트 형식 (기본)
claude -p --output-format text "이 함수를 설명해줘"

# 스트림 JSON (실시간 처리)
claude -p --output-format stream-json "프로젝트 분석 결과를 알려줘"
```

자동화 스크립트 예시

매일 아침 코드 품질을 체크하는 스크립트:

```
#!/bin/bash
# daily-check.sh

echo "=== 일일 코드 품질 체크 ==="

# 커밋되지 않은 TODO 확인
claude -p --max-turns 3 "src/ 디렉터리에서 TODO, FIXME, HACK 주석을 찾아서 우선순위로 정리해
줘"

# 최근 변경 사항 리뷰
git log --since="yesterday" --oneline | claude -p "어제 커밋들의 요약과 잠재적 이슈를 알려줘"
```

CI/CD 파이프라인에서 PR 자동 리뷰:

```
# PR의 변경 파일을 보안 관점에서 자동 리뷰
git diff main --name-only | claude -p \
  --max-budget-usd 2.00 \
  --max-turns 5 \
  "변경된 파일들을 보안 관점에서 검토하고, 문제가 있으면 설명해줘"
```

3.9 실습: 기본기 종합 연습

지금까지 배운 내용을 종합적으로 연습해봅시다.

실습 1: 모델 비교 체험

같은 질문을 세 가지 모델로 던져보고 차이를 체감합니다:

```
# Haiku로 시작
claude --model haiku
```

```
> JavaScript에서 깊은 복사(deep copy)를 구현하는 가장 좋은 방법은?
> /exit
```

```
# Sonnet으로 같은 질문
claude --model sonnet
```

```
> JavaScript에서 깊은 복사(deep copy)를 구현하는 가장 좋은 방법은?
> /exit
```

```
# Opus로 같은 질문
claude --model opus
```

```
> JavaScript에서 깊은 복사(deep copy)를 구현하는 가장 좋은 방법은?  
> /exit
```

세 모델의 답변 깊이, 속도, 코드 예시의 풍부함을 비교해보세요.

실습 2: 세션 관리 체험

```
claude -n "session-practice"
```

```
> 간단한 계산기 함수를 만들어줘  
> /cost  
> /compact  
> 방금 만든 함수에 나눗셈 0 처리를 추가해줘  
> /diff  
> /exit
```

```
# 세션 이어가기  
claude -c
```

```
> 이어서 계산기에 제곱 연산을 추가해줘  
> /exit
```

실습 3: 파이프라인 활용

```
# README 파일 분석  
cat README.md | claude -p "이 프로젝트를 한 문장으로 요약해줘"  
  
# package.json 분석  
cat package.json | claude -p "사용 중인 의존성을 분류하고, 오래된 패키지가 있으면 알려줘"
```

3.10 정리

이 챕터에서 배운 내용을 정리합니다.

- **대화형 모드**(`claude`)는 지속적인 대화에, **비대화형 모드**(`claude -p`)는 일회성 작업과 스크립팅에 적합합니다
- **효과적인 프롬프트**의 핵심은 구체적인 지시, 예러 메시지 직접 붙여넣기, `@` 멘션으로 범위 한정, Plan 모드로 대규모 작업 시작하기입니다
- **슬래시 명령어**로 세션을 효율적으로 관리합니다: `/compact` (압축), `/clear` (초기화), `/resume` (이어가기), `/cost` (비용 확인), `/model` (모델 변경)
- **네 가지 모델**을 작업 복잡도에 따라 선택합니다: Haiku(간단), Sonnet(균형), Opus(복잡), opusplan(하이브리드)
- **노력 수준**(low/medium/high/max)으로 사고 깊이를 조절합니다
- **비용 관리**의 핵심은 적절한 모델 선택, `/compact` 활용, 예산 제한(`--max-budget-usd`), `@` 멘션으로 범위 한정입니다
- **파이프라인**으로 Claude Code를 Unix 도구들과 결합하여 강력한 자동화를 구현할 수 있습니다
- 세션 이름 지정(`-n`), 이어가기(`-c` , `-r`), 분기(`/fork`)로 작업 흐름을 체계적으로 관리합니다

Part 1을 마쳤습니다. 이제 Claude Code를 설치하고, 기본적인 대화를 나누고, 효율적으로 사용하는 방법을 익혔습니다. Part 2에서는 코드 읽기, 코드 쓰기, Git 연동, 프로젝트 메모리 등 핵심 기능을 깊이 있게 학습합니다.

Chapter 4: 코드 읽기와 이해

학습 목표

1. Claude Code를 활용하여 낯선 코드베이스를 빠르게 파악합니다
 2. 특정 함수, 클래스, 모듈의 동작을 정확히 이해합니다
 3. 코드 검색 도구(Grep, Glob)를 활용한 탐색 기법을 익힙니다
 4. 의존성 관계를 추적하고 아키텍처를 시각화하는 방법을 배웁니다
-

4.1 프로젝트 전체 구조 파악하기

새로운 프로젝트에 합류했을 때 가장 먼저 해야 할 일은 전체 구조를 파악하는 것입니다. 기존에는 README를 읽고, 디렉터리를 하나씩 탐색하고, `package.json`이나 설정 파일을 뒤져가며 프로젝트의 윤곽을 잡아야 했습니다. Claude Code는 이 과정을 한 번의 대화로 단축시킵니다.

실전 예제 1: 프로젝트 전체 구조 분석

새로운 Express.js 기반 API 프로젝트에 합류한 상황을 가정해봅니다.

사용자: 이 프로젝트의 전체 구조를 설명해줘. 어떤 기술 스택을 쓰고 있고, 주요 디렉터리와 파일의 역할이 뭔지 알려줘.

Claude Code: 프로젝트를 분석하겠습니다.

```
[Glob: **/*.{js,ts,json,yaml,yml}]
[Read: package.json]
[Read: tsconfig.json]
[Read: docker-compose.yml]
[Glob: src/**/*]
```

이 프로젝트는 Express.js + TypeScript 기반의 REST API 서버입니다.

📁 프로젝트 구조

```
├── src/
│   ├── api/
│   │   ├── controllers/    # 요청 핸들러 (UserController, AuthController 등)
│   │   ├── middlewares/    # 인증, 로깅, 에러 처리 미들웨어
│   │   ├── routes/         # 라우트 정의 (userRoutes, authRoutes)
│   │   └── validators/     # 요청 검증 스키마 (Zod 사용)
│   ├── services/           # 비즈니스 로직 계층
│   ├── models/             # Prisma ORM 모델 정의
│   ├── utils/              # 유틸리티 함수 (logger, crypto, date 등)
│   ├── config/             # 환경 설정 (database, redis, jwt)
│   └── app.ts              # Express 앱 초기화 및 미들웨어 등록
├── prisma/
│   ├── schema.prisma       # 데이터베이스 스키마
│   └── migrations/         # DB 마이그레이션 파일
├── tests/                  # Jest 테스트 파일
├── docker-compose.yml      # PostgreSQL + Redis 로컬 환경
└── package.json
```

기술 스택:

- 런타임: Node.js 20 + TypeScript 5.4
- 웹 프레임워크: Express 4.18
- ORM: Prisma 5.10
- 데이터베이스: PostgreSQL 16
- 캐시: Redis 7
- 인증: JWT (jsonwebtoken)
- 검증: Zod
- 테스트: Jest + Supertest
- 린트: ESLint + Prettier

이 한 번의 대화로 프로젝트의 기술 스택, 디렉터리 구조, 아키텍처 패턴(Controller-Service-Model 계층 구조)을 모두 파악할 수 있습니다. Claude Code는 내부적으로 Glob 패턴으로 파일 목록을 수집하고, 핵심 설정 파일을 읽어 분석합니다.

더 깊이 파고들기

프로젝트 구조를 파악한 후, 특정 관심 영역을 더 깊이 탐색할 수 있습니다.

사용자: 이 프로젝트의 API 엔드포인트를 모두 정리해줘.
HTTP 메서드, 경로, 인증 필요 여부를 표로 만들어줘.

Claude Code: 라우트 파일들을 분석하겠습니다.

```
[Read: src/api/routes/index.ts]
[Read: src/api/routes/userRoutes.ts]
[Read: src/api/routes/authRoutes.ts]
[Read: src/api/routes/productRoutes.ts]
[Read: src/api/middlewares/auth.ts]
```

API 엔드포인트 목록:

HTTP 메서드	경로	설명	인증 필요
POST	/api/auth/register	회원가입	No
POST	/api/auth/login	로그인	No
POST	/api/auth/refresh	토큰 갱신	No
GET	/api/users/me	내 정보 조회	Yes
PUT	/api/users/me	내 정보 수정	Yes
GET	/api/users/:id	사용자 조회	Yes
GET	/api/products	상품 목록	No
GET	/api/products/:id	상품 상세	No
POST	/api/products	상품 등록	Yes (Admin)
PUT	/api/products/:id	상품 수정	Yes (Admin)
DELETE	/api/products/:id	상품 삭제	Yes (Admin)

인증은 `authMiddleware`를 통해 JWT 토큰을 검증합니다.
Admin 권한은 `roleMiddleware('admin')`으로 추가 검증합니다.

4.2 파일 및 함수 단위 분석

전체 구조를 파악한 후에는 특정 파일이나 함수의 동작 원리를 이해해야 할 때가 있습니다. Claude Code는 코드를 읽고 자연어로 설명해주는 것은 물론, 실행 흐름(flow)을 추적하는 데에도 탁월합니다.

실전 예제 2: 요청 흐름 추적

특정 API 엔드포인트가 요청을 처리하는 전체 과정을 추적해봅니다.

사용자: POST /api/auth/login 요청이 들어왔을 때, 요청이 어떻게 처리되는지
처음부터 끝까지 흐름을 따라가줘. 미들웨어도 포함해서.

Claude Code: 로그인 요청의 전체 흐름을 추적하겠습니다.

```
[Read: src/app.ts]
[Read: src/api/routes/authRoutes.ts]
[Read: src/api/validators/authValidator.ts]
[Read: src/api/controllers/AuthController.ts]
[Read: src/services/AuthService.ts]
[Read: src/models/User.ts]
```

POST /api/auth/login 요청 처리 흐름:

1. Express 앱 진입 (app.ts)

- └─ cors() 미들웨어: CORS 헤더 처리
- └─ express.json(): 요청 본문 JSON 파싱
- └─ requestLogger(): 요청 로깅 (메서드, 경로, IP)
- └─ /api 라우터로 전달

2. 라우트 매칭 (authRoutes.ts:12)

- └─ router.post('/login', validateBody(loginSchema), AuthController.login)

3. 요청 검증 (authValidator.ts:8)

- └─ loginSchema (Zod)
 - └─ email: z.string().email() - 이메일 형식 검증
 - └─ password: z.string().min(8) - 최소 8자 검증
- ⚠️ 검증 실패 시: 400 Bad Request + 에러 상세 반환

4. 컨트롤러 (AuthController.ts:24)

- └─ AuthController.login(req, res)
 - └─ req.body에서 email, password 추출
 - └─ AuthService.login(email, password) 호출
 - └─ 결과를 200 OK + JSON으로 응답

5. 서비스 계층 (AuthService.ts:31)

- └─ AuthService.login(email, password)
 - └─ prisma.user.findUnique({ where: { email } })
 - ⚠️ 사용자 없으면: 401 Unauthorized
 - └─ bcrypt.compare(password, user.passwordHash)
 - ⚠️ 비밀번호 불일치: 401 Unauthorized
 - └─ jwt.sign({ userId, role }, JWT_SECRET, { expiresIn: '15m' })
 - └─ generateRefreshToken(userId) → DB에 저장
 - └─ return { accessToken, refreshToken, user }

6. 응답 반환

- └─ { accessToken: "eyJ...", refreshToken: "abc...", user: { id, email, name } }

에러 처리:

- 모든 예외는 errorHandler 미들웨어(middlewares/error.ts)에서 포착
- AppError 클래스를 통해 HTTP 상태 코드와 메시지를 표준화
- 프로덕션 환경에서는 스택 트레이스를 숨김

이 분석은 수동으로 하면 최소 30분 이상 걸리는 작업입니다. Claude Code는 관련 파일을 자동으로 찾아 읽고, 함수 호출 관계를 추적하여 전체 흐름을 한눈에 보여줍니다.

@ 멘션으로 정확한 컨텍스트 제공하기

Claude Code에게 특정 파일이나 폴더를 명시적으로 알려주고 싶을 때는 @ 멘션(mention)을 사용합니다. 이는 Claude Code가 올바른 파일을 참조하도록 보장하는 강력한 방법입니다.

사용자: @src/services/AuthService.ts 의 refreshToken 메서드가
보안상 안전한지 분석해줘.

Claude Code: AuthService.ts의 refreshToken 메서드를 보안 관점에서 분석하겠습니다.

[Read: src/services/AuthService.ts]

refreshToken 메서드 보안 분석:

✅ 잘된 점:

1. Refresh Token을 crypto.randomBytes(64)로 생성 – 충분한 엔트로피
2. bcrypt로 해싱하여 DB에 저장 – 토큰 유출 시에도 원본 복원 불가
3. 만료 시간(7일) 설정 – 무기한 사용 방지

⚠️ 개선 필요:

1. Token Rotation 미구현
 - 현재: refresh token 사용 시 기존 토큰 유지
 - 권장: 사용할 때마다 새 refresh token을 발급하고 이전 것을 무효화
 - 이유: 토큰 탈취 시 공격 윈도우를 줄임
2. Token Family 추적 없음
 - 현재: 탈취된 refresh token으로 계속 새 access token 발급 가능
 - 권장: token family 개념 도입, 재사용 감지 시 해당 family 전체 무효화
3. Device/IP 바인딩 없음
 - 현재: 어떤 기기/IP에서든 refresh token 사용 가능
 - 권장: 발급 시 기기 fingerprint나 IP 대역 기록, 불일치 시 추가 검증

수정 코드를 작성해드릴까요?

4.3 의존성 추적

대규모 코드베이스에서 "이 함수를 바꾸면 어디에 영향이 가지?"라는 질문은 매우 중요합니다. Claude Code는 내부적으로 Grep 도구를 사용하여 코드 전체에서 참조를 검색합니다.

실전 예제 3: 함수 영향 범위 분석

사용자: `formatPrice` 함수를 수정하려고 하는데, 이 함수를 호출하는 곳을 모두 찾아줘. 변경 시 영향 범위를 분석해줘.

Claude Code: `formatPrice` 함수의 사용처를 검색하겠습니다.

```
[Grep: formatPrice]
[Read: src/utils/format.ts]
[Read: src/services/OrderService.ts]
[Read: src/services/ProductService.ts]
[Read: src/api/controllers/InvoiceController.ts]
[Read: tests/utils/format.test.ts]
```

`formatPrice` 함수 사용처 분석:

정의 위치: `src/utils/format.ts:15`

```
export function formatPrice(amount: number, currency: string = 'KRW'): string {
  return new Intl.NumberFormat('ko-KR', {
    style: 'currency',
    currency,
  }).format(amount);
}
```

호출하는 파일 (4곳):

1. `src/services/OrderService.ts:78`
 - `getOrderSummary()`에서 주문 총액 포매팅
 - 영향: 주문 요약 API 응답
2. `src/services/ProductService.ts:45`
 - `getProductDetail()`에서 상품 가격 포매팅
 - 영향: 상품 상세 API 응답
3. `src/api/controllers/InvoiceController.ts:92`
 - `generateInvoice()`에서 청구서 금액 포매팅
 - 영향: PDF 청구서 생성
4. `tests/utils/format.test.ts:23`
 - 단위 테스트 7개 케이스

함수 시그니처 요약:

- 입력: `amount` (number), `currency` (string, 기본값 'KRW')
- 출력: string (포매팅된 금액)

변경 시 주의사항:

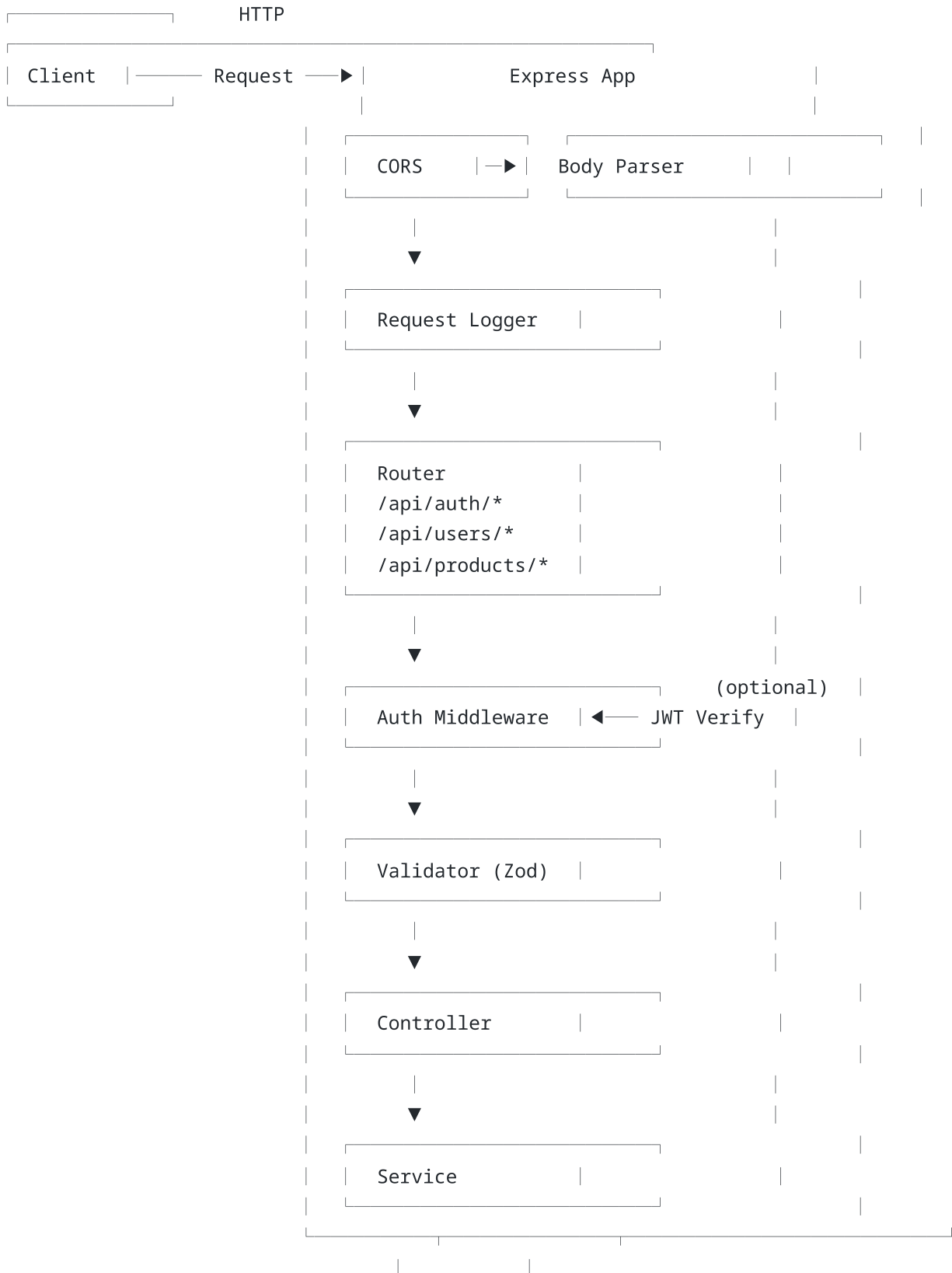
- 반환 타입(string)을 변경하면 4곳 모두 영향
- currency 파라미터의 기본값을 변경하면 기본값에 의존하는 호출부에 영향
(현재 OrderService와 ProductService는 기본값 사용)
- 테스트가 있으므로, 수정 후 `npm test -- format.test.ts` 로 검증 가능

4.4 아키텍처 시각화

코드를 읽는 것만으로는 전체 그림을 그리기 어려울 때가 있습니다. Claude Code에게 텍스트 기반 다이어그램을 요청하면 아키텍처를 시각적으로 파악할 수 있습니다.

사용자: 이 프로젝트의 데이터 흐름을 ASCII 다이어그램으로 그려줘.
클라이언트 요청부터 DB 응답까지 전체 과정을 보여줘.

Claude Code:





4.5 코드 리뷰 요청

Claude Code는 코드를 읽고 이해하는 능력을 바탕으로, 보안 취약점 분석이나 코드 품질 리뷰도 수행할 수 있습니다.

보안 리뷰

사용자: /security-review

Claude Code: 현재 브랜치의 변경 사항을 보안 관점에서 분석하겠습니다.

[Bash: git diff main --name-only]

[Read: 변경된 파일들...]

보안 분석 결과:

● 심각 (Critical):

1. src/api/controllers/UserController.ts:34

- SQL 인젝션 가능성: 사용자 입력을 직접 쿼리에 삽입
- 현재: `prisma.$queryRaw(`SELECT * FROM users WHERE name = '${name}'`)`
- 수정: `prisma.$queryRaw`SELECT * FROM users WHERE name = ${name}``
(태그드 템플릿 리터럴로 파라미터화)

● 주의 (Warning):

2. src/services/FileService.ts:21

- Path Traversal 가능성: 사용자 제공 파일명을 검증 없이 사용
- 현재: `path.join(uploadDir, req.body.filename)`
- 권장: `path.basename()`으로 디렉터리 탐색 방지 후 사용

3. src/config/cors.ts:5

- CORS origin이 '*'로 설정됨
- 개발 환경에서는 괜찮지만 프로덕션에서는 특정 도메인으로 제한 필요

✅ 양호:

- JWT 비밀키가 환경 변수에서 로드됨
- 비밀번호 해싱에 `bcrypt` 사용 (`salt rounds: 12`)
- Rate limiting 미들웨어 적용됨

4.6 코드 검색 도구 이해하기

Claude Code가 코드베이스를 탐색할 때 내부적으로 사용하는 두 가지 핵심 도구가 있습니다.

Grep: 내용 기반 검색

Grep은 파일 내용에서 특정 패턴을 검색합니다. 정규식(Regular Expression)을 지원하며, 변수명, 함수명, 에러 메시지 등을 찾을 때 사용됩니다.

사용자: **TODO** 주석이 남아있는 곳을 모두 찾아줘

Claude Code:

[Grep: **TODO**|**FIXME**|**HACK**|**XXX**]

프로젝트에서 발견된 **TODO** 주석:

1. `src/services/AuthService.ts:89`
// **TODO**: refresh token rotation 구현
2. `src/api/middlewares/rateLimiter.ts:12`
// **FIXME**: Redis 연결 실패 시 fallback 필요
3. `src/utils/logger.ts:34`
// **TODO**: 프로덕션에서 로그 레벨 필터링 추가
4. `src/services/NotificationService.ts:56`
// **HACK**: 이메일 전송 실패 시 3번 재시도 - 큐 시스템으로 대체 필요

총 4개의 미완성 항목이 있습니다. 우선순위를 매겨드릴까요?

Glob: 파일 이름 기반 검색

Glob은 파일명 패턴으로 파일을 찾습니다. 디렉터리 구조를 탐색하거나 특정 확장자의 파일을 모두 찾을 때 사용됩니다.

사용자: 이 프로젝트에서 테스트 파일이 몇 개 있고, 어떤 모듈을 테스트하는지 정리해줘

Claude Code:

```
[Glob: **/*.test.{ts,js}]
```

```
[Glob: **/*.spec.{ts,js}]
```

테스트 파일 현황:

총 12개 테스트 파일:

파일	테스트 대상	케이스 수
tests/utils/format.test.ts	formatPrice, formatDate	7
tests/utils/crypto.test.ts	hashPassword, comparePassword	5
tests/services/AuthService.test.ts	login, register, refresh	12
tests/services/UserService.test.ts	getUser, updateUser	8
tests/api/auth.integration.test.ts	인증 API 통합 테스트	15
tests/api/user.integration.test.ts	사용자 API 통합 테스트	10
...	...	...

테스트 커버리지가 없는 모듈:

- ProductService (상품 관련 로직)
- InvoiceController (청구서 생성)
- NotificationService (알림 발송)

4.7 효과적인 코드 읽기를 위한 팁

단계적 접근법

코드베이스를 이해할 때는 넓은 곳에서 좁은 곳으로 접근하는 것이 효과적입니다.

1. **1단계 - 전체 조감**: "이 프로젝트의 구조를 설명해줘"
2. **2단계 - 모듈 단위**: "인증 모듈이 어떻게 구성되어 있는지 설명해줘"
3. **3단계 - 함수 단위**: "이 함수가 정확히 어떻게 동작하는지 단계별로 설명해줘"
4. **4단계 - 흐름 추적**: "이 요청이 처리되는 전체 과정을 따라가줘"

구체적으로 질문하기

모호한 질문 (비효율적)

사용자: 이 코드 설명해줘

구체적인 질문 (효율적)

사용자: @src/services/PaymentService.ts 의 processRefund 메서드가

부분 환불(partial refund)을 어떻게 처리하는지,

결제 게이트웨이와의 통신 흐름을 포함해서 설명해줘

비교 분석 요청하기

사용자: @src/services/AuthService.ts 와 @src/services/OAuthService.ts 의
로그인 흐름을 비교해줘. 공통점과 차이점을 정리해줘.

Claude Code: 두 서비스의 로그인 흐름을 비교 분석하겠습니다.

[Read: src/services/AuthService.ts]

[Read: src/services/OAuthService.ts]

항목	AuthService (일반 로그인)	OAuthService (소셜 로그인)
인증 방식	이메일 + 비밀번호	OAuth 2.0 Provider
사용자 검증	DB에서 이메일로 조회	Provider에서 프로필 조회
비밀번호 처리	bcrypt 비교	없음 (Provider 위임)
신규 사용자	별도 register 메서드	자동 계정 생성
토큰 발급	JWT access + refresh	JWT access + refresh
에러 처리	InvalidCredentials 예외	OAuthError 예외

공통점:

- 둘 다 최종적으로 JWT access token + refresh token을 발급
- 같은 User 모델을 사용
- 같은 토큰 생성 유틸리티 함수 사용 (generateTokenPair)

차이점:

- AuthService는 비밀번호를 직접 검증하지만, OAuthService는 Provider에게 위임
- OAuthService는 첫 로그인 시 자동으로 사용자를 생성하는 upsert 패턴 사용
- OAuthService는 Provider별 프로필 매핑 로직이 추가로 필요

정리

이 챕터에서 배운 핵심 내용을 정리합니다.

프로젝트 파악 전략

- "이 프로젝트의 구조를 설명해줘"로 전체 조감도를 먼저 확보합니다
- 넓은 곳에서 좁은 곳으로 단계적으로 접근합니다

핵심 분석 패턴

- 요청 흐름 추적: "POST /api/login 요청이 어떻게 처리되는지 따라가줘"
- 의존성 분석: "이 함수를 호출하는 곳을 모두 찾아줘"
- 보안 리뷰: `/security-review` 명령

효과적인 사용법

- @ 멘션으로 분석 대상 파일을 명시적으로 지정합니다
- 구체적이고 상세한 질문이 더 좋은 분석 결과를 이끌어냅니다
- Claude Code는 내부적으로 Grep(내용 검색)과 Glob(파일명 검색)을 사용합니다

다음 챕터에서는 Claude Code로 직접 코드를 작성하고 편집하는 방법을 학습합니다. 코드를 "읽는" 능력 위에 "쓰는" 능력을 더해, Claude Code와의 협업을 본격화합니다.

Chapter 5: 코드 작성과 편집

| 학습 목표

1. Claude Code에게 새로운 기능을 구현하도록 지시하고 결과를 검증합니다
 2. 기존 코드를 수정하는 다양한 패턴(리팩토링, 테스트 추가, 타입 변환 등)을 익힙니다
 3. Plan 모드를 활용한 대규모 변경 전략을 수립합니다
 4. 생성된 코드의 품질을 검증하고 피드백을 주는 워크플로를 확립합니다
-

| 5.1 새 파일 생성하기

Claude Code는 단순히 코드 조각을 생성하는 것이 아니라, 프로젝트의 기존 패턴과 규칙에 맞는 완전한 파일을 작성합니다. 기존 코드를 참고하여 일관된 스타일을 유지하기 때문에, 마치 팀의 코딩 컨벤션을 이미 알고 있는 동료처럼 작동합니다.

실전 예제 1: REST API 엔드포인트 추가

기존 프로젝트에 새로운 API 엔드포인트를 추가하는 가장 일반적인 시나리오입니다.

사용자: 상품 카테고리를 관리하는 **CRUD API**를 만들어줘.
기존 products **API**의 패턴을 따라서 만들어줘.
카테고리는 id, name, description, **parentId**(상위 카테고리) 필드를 가져.

Claude Code: 기존 products **API** 패턴을 분석하고, 동일한 구조로
카테고리 **CRUD API**를 생성하겠습니다.

```
[Read: src/api/routes/productRoutes.ts]
[Read: src/api/controllers/ProductController.ts]
[Read: src/services/ProductService.ts]
[Read: src/api/validators/productValidator.ts]
[Read: prisma/schema.prisma]
```

기존 패턴을 분석했습니다. 다음 파일들을 생성하겠습니다:

1. Prisma 스키마에 **Category** 모델 추가
2. src/api/routes/categoryRoutes.ts
3. src/api/controllers/CategoryController.ts
4. src/services/CategoryService.ts
5. src/api/validators/categoryValidator.ts

```
[Edit: prisma/schema.prisma]
```

```
model Category {
  id          String      @id @default(cuid())
  name        String      @unique
  description String?
  parentId    String?
  parent      Category?   @relation("CategoryTree", fields: [parentId], references:
[id])
  children    Category[]  @relation("CategoryTree")
  products    Product[]
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}
```

```
[Write: src/api/validators/categoryValidator.ts]
```

```
import { z } from 'zod';

export const createCategorySchema = z.object({
  name: z.string().min(1).max(100),
  description: z.string().max(500).optional(),
  parentId: z.string().cuid().optional(),
});

export const updateCategorySchema = createCategorySchema.partial();
```

[Write: src/services/CategoryService.ts]

```
import { prisma } from '../config/database';
import { AppError } from '../utils/errors';

export class CategoryService {
  static async findAll() {
    return prisma.category.findMany({
      include: { children: true, _count: { select: { products: true } } },
      where: { parentId: null }, // 최상위 카테고리만
    });
  }

  static async findById(id: string) {
    const category = await prisma.category.findUnique({
      where: { id },
      include: { children: true, products: true },
    });
    if (!category) throw new AppError('Category not found', 404);
    return category;
  }

  static async create(data: { name: string; description?: string; parentId?: string }) {
    if (data.parentId) {
      const parent = await prisma.category.findUnique({ where: { id: data.parentId } });
      if (!parent) throw new AppError('Parent category not found', 404);
    }
    return prisma.category.create({ data });
  }

  static async update(id: string, data: Partial<{ name: string; description: string; parentId: string }>) {
    await this.findById(id); // 존재 확인
    return prisma.category.update({ where: { id }, data });
  }

  static async delete(id: string) {
    const category = await this.findById(id);
    if (category.children.length > 0) {
      throw new AppError('Cannot delete category with subcategories', 400);
    }
    return prisma.category.delete({ where: { id } });
  }
}
```

[Write: src/api/controllers/CategoryController.ts]


```
[Write: src/api/routes/categoryRoutes.ts]
[Edit: src/api/routes/index.ts] // 새 라우트 등록
```

5개 파일을 생성/수정했습니다. Prisma 마이그레이션을 실행하겠습니다.

```
[Bash: npx prisma migrate dev --name add-category-model]
```

마이그레이션이 완료되었습니다. 테스트를 실행해볼까요?

Claude Code가 기존 코드 패턴을 분석하여 일관된 구조로 새 파일을 생성하는 것에 주목합니다. Controller-Service-Validator 패턴, 에러 처리 방식, Prisma 쿼리 스타일 등이 기존 코드와 동일합니다.

5.2 기존 코드 수정하기

새 코드를 작성하는 것만큼이나 기존 코드를 수정하는 일이 빈번합니다. Claude Code는 파일의 특정 부분만 정확하게 수정하며, 변경 전후를 명확하게 보여줍니다.

버그 수정

사용자: 사용자가 비밀번호를 변경한 후에도 이전에 발급된 JWT 토큰이 계속 유효한 버그가 있어. 비밀번호 변경 시 기존 토큰을 무효화하도록 수정해줘.

Claude Code: 비밀번호 변경 시 기존 토큰을 무효화하는 로직을 추가하겠습니다.

[Read: src/services/UserService.ts]

[Read: src/services/AuthService.ts]

[Read: prisma/schema.prisma]

현재 문제: 비밀번호 변경 시 기존 refresh token을 삭제하지 않고, JWT의 iat(issued at)도 검증하지 않습니다.

수정 계획:

1. User 모델에 tokenInvalidatedAt 필드 추가
2. 비밀번호 변경 시 이 필드를 현재 시각으로 업데이트
3. JWT 검증 시 $iat < tokenInvalidatedAt$ 이면 토큰 거부
4. 기존 refresh token 모두 삭제

[Edit: prisma/schema.prisma]

// User 모델에 추가

+ tokenInvalidatedAt DateTime?

[Edit: src/services/UserService.ts]

// changePassword 메서드 수정

```
async changePassword(userId: string, oldPassword: string, newPassword: string) {
  const user = await prisma.user.findUnique({ where: { id: userId } });
  if (!user) throw new AppError('User not found', 404);

  const isValid = await bcrypt.compare(oldPassword, user.passwordHash);
  if (!isValid) throw new AppError('Invalid current password', 401);

  const newHash = await bcrypt.hash(newPassword, 12);

  await prisma.$transaction([
    // 비밀번호 업데이트 + 토큰 무효화 시점 기록
    prisma.user.update({
      where: { id: userId },
      data: {
        passwordHash: newHash,
        tokenInvalidatedAt: new Date(),
      },
    }),
    // 기존 refresh token 모두 삭제
    prisma.refreshToken.deleteMany({
```

```

        where: { userId },
      })),
    ]);
}

[Edit: src/api/middlewares/auth.ts]
// JWT 검증 시 tokenInvalidatedAt 확인 추가

const decoded = jwt.verify(token, JWT_SECRET) as JwtPayload;
const user = await prisma.user.findUnique({ where: { id: decoded.userId } });

if (user.tokenInvalidatedAt && decoded.iat < user.tokenInvalidatedAt.getTime() /
1000) {
  throw new AppError('Token has been invalidated', 401);
}

```

3개 파일을 수정했습니다. 마이그레이션을 실행하겠습니다.

```
[Bash: npx prisma migrate dev --name add-token-invalidatation]
```

마이그레이션 완료. 변경 사항을 테스트하겠습니다.

```
[Bash: npm test -- --testPathPattern="auth"]
```

모든 기존 테스트가 통과합니다. 새 시나리오에 대한 테스트도 추가할까요?

5.3 테스트 코드 작성

테스트 작성은 Claude Code가 특히 빛을 발하는 영역입니다. 기존 코드를 분석하여 엣지 케이스(edge case)를 포함한 포괄적인 테스트를 자동으로 생성합니다.

실전 예제 2: 서비스 계층 테스트 작성

사용자: CategoryService의 단위 테스트를 작성해줘.
정상 케이스뿐 아니라 엣지 케이스와 에러 케이스도 포함해줘.
기존 테스트 파일의 스타일을 따라줘.

Claude Code: 기존 테스트 스타일을 확인하고 CategoryService 테스트를 작성하겠습니다.

```
[Read: tests/services/AuthService.test.ts]
[Read: src/services/CategoryService.ts]
[Read: tests/setup.ts]
```

기존 테스트 패턴을 확인했습니다:

- Jest + Prisma Mock 사용
- describe/it 구조
- beforeEach에서 mock 초기화
- AAA 패턴 (Arrange-Act-Assert)

```
[Write: tests/services/CategoryService.test.ts]
```

```
import { CategoryService } from '../../../src/services/CategoryService';
import { prismaMock } from '../../../setup';
import { AppError } from '../../../src/utils/errors';

describe('CategoryService', () => {
  describe('findAll', () => {
    it('최상위 카테고리만 반환한다', async () => {
      const mockCategories = [
        { id: '1', name: '전자기기', parentId: null, children: [], _count: { products: 5 } },
        { id: '2', name: '의류', parentId: null, children: [], _count: { products: 12 } },
      ];
      prismaMock.category.findMany.mockResolvedValue(mockCategories);

      const result = await CategoryService.findAll();

      expect(result).toHaveLength(2);
      expect(prismaMock.category.findMany).toHaveBeenCalledWith(
        expect.objectContaining({ where: { parentId: null } })
      );
    });

    it('카테고리가 없으면 빈 배열을 반환한다', async () => {
      prismaMock.category.findMany.mockResolvedValue([]);

      const result = await CategoryService.findAll();
```

```

    expect(result).toEqual([]);
  });
});

describe('findById', () => {
  it('존재하는 카테고리를 반환한다', async () => {
    const mockCategory = {
      id: '1', name: '전자기기', description: '전자 제품',
      parentId: null, children: [], products: [],
    };
    prismaMock.category.findUnique.mockResolvedValue(mockCategory);

    const result = await CategoryService.findById('1');

    expect(result.name).toBe('전자기기');
  });

  it('존재하지 않는 카테고리는 404 에러를 발생시킨다', async () => {
    prismaMock.category.findUnique.mockResolvedValue(null);

    await expect(CategoryService.findById('nonexistent'))
      .rejects.toThrow(AppError);
    await expect(CategoryService.findById('nonexistent'))
      .rejects.toMatchObject({ statusCode: 404 });
  });
});

describe('create', () => {
  it('새 카테고리를 생성한다', async () => {
    const input = { name: '스마트폰', description: '휴대전화' };
    const mockCreated = { id: '3', ...input, parentId: null };
    prismaMock.category.create.mockResolvedValue(mockCreated);

    const result = await CategoryService.create(input);

    expect(result.name).toBe('스마트폰');
    expect(prismaMock.category.create).toHaveBeenCalledWith({ data: input });
  });

  it('존재하지 않는 상위 카테고리를 지정하면 404 에러를 발생시킨다', async () => {
    prismaMock.category.findUnique.mockResolvedValue(null);

    await expect(CategoryService.create({ name: '노트북', parentId: 'invalid' }))
      .rejects.toThrow(AppError);
  });

  it('중복된 이름으로 생성 시 에러를 발생시킨다', async () => {
    prismaMock.category.create.mockRejectedValue(

```

```

        new Error('Unique constraint failed on the fields: (`name`'))
    );

    await expect(CategoryService.create({ name: '전자기기' }))
        .rejects.toThrow();
    });

describe('delete', () => {
    it('하위 카테고리가 있으면 삭제할 수 없다', async () => {
        prismaMock.category.findUnique.mockResolvedValue({
            id: '1', name: '전자기기', children: [{ id: '2', name: '스마트폰' }], products:
[],
        });

        await expect(CategoryService.delete('1'))
            .rejects.toMatchObject({ statusCode: 400 });
    });

    it('하위 카테고리가 없으면 정상 삭제된다', async () => {
        prismaMock.category.findUnique.mockResolvedValue({
            id: '1', name: '전자기기', children: [], products: [],
        });
        prismaMock.category.delete.mockResolvedValue({ id: '1' });

        await CategoryService.delete('1');

        expect(prismaMock.category.delete).toHaveBeenCalledWith({ where: { id: '1' }
});
    });
});
});

```

테스트를 실행하겠습니다.

```
[Bash: npm test -- tests/services/CategoryService.test.ts]
```

```
PASS tests/services/CategoryService.test.ts
```

```
CategoryService
```

```
  findAll
```

- ✓ 최상위 카테고리만 반환한다 (3ms)
- ✓ 카테고리가 없으면 빈 배열을 반환한다 (1ms)

```
  findById
```

- ✓ 존재하는 카테고리를 반환한다 (2ms)
- ✓ 존재하지 않는 카테고리는 404 에러를 발생시킨다 (1ms)

```
  create
```

- ✓ 새 카테고리를 생성한다 (1ms)
- ✓ 존재하지 않는 상위 카테고리를 지정하면 404 에러를 발생시킨다 (1ms)
- ✓ 중복된 이름으로 생성 시 에러를 발생시킨다 (1ms)

delete

- ✓ 하위 카테고리가 있으면 삭제할 수 없다 (1ms)
- ✓ 하위 카테고리가 없으면 정상 삭제된다 (1ms)

Test Suites: 1 passed, 1 total

Tests: 9 passed, 9 total

9개의 테스트가 모두 통과합니다.

5.4 리팩토링

리팩토링(Refactoring)은 외부 동작은 유지하면서 코드의 내부 구조를 개선하는 작업입니다. Claude Code는 대규모 리팩토링에서 특히 유용합니다. 여러 파일에 걸친 일관된 변경을 안전하게 수행하기 때문입니다.

실전 예제 3: 콜백에서 async/await로 변환

사용자: @src/services/LegacyEmailService.ts 가 콜백 패턴으로 작성되어 있어.
async/await 패턴으로 리팩토링해줘. 기존 테스트가 깨지지 않도록 해줘.

Claude Code: LegacyEmailService를 분석하고 async/await로 변환하겠습니다.

[Read: src/services/LegacyEmailService.ts]
[Read: tests/services/LegacyEmailService.test.ts]

현재 코드 (콜백 패턴):

```
sendEmail(to, subject, body, callback) {
  this.transport.connect((err, connection) => {
    if (err) return callback(err);
    connection.send({ to, subject, body }, (err, result) => {
      if (err) return callback(err);
      this.logDelivery(result, (err) => {
        if (err) return callback(err);
        callback(null, result);
      });
    });
  });
}
```

리팩토링 후 (async/await):

[Edit: src/services/LegacyEmailService.ts]

```
import { promisify } from 'util';

export class EmailService {
  private connectAsync: () => Promise<Connection>;

  constructor(private transport: Transport) {
    this.connectAsync = promisify(this.transport.connect.bind(this.transport));
  }

  async sendEmail(to: string, subject: string, body: string): Promise<SendResult> {
    const connection = await this.connectAsync();

    const sendAsync = promisify(connection.send.bind(connection));
    const result = await sendAsync({ to, subject, body });

    await this.logDeliveryAsync(result);

    return result;
  }
}
```



```

    }

    private async logDeliveryAsync(result: SendResult): Promise<void> {
        const logAsync = promisify(this.logDelivery.bind(this));
        return logAsync(result);
    }
}

[Edit: tests/services/LegacyEmailService.test.ts]
// 테스트도 async/await 패턴으로 업데이트

it('이메일을 성공적으로 전송한다', async () => {
    const result = await emailService.sendEmail(
        'user@example.com',
        'Welcome',
        'Hello World'
    );
    expect(result.messageId).toBeDefined();
    expect(result.status).toBe('sent');
});

it('연결 실패 시 에러를 throw한다', async () => {
    mockTransport.connect.mockImplementation((cb) => cb(new Error('Connection
refused')));

    await expect(
        emailService.sendEmail('user@example.com', 'Test', 'Body')
    ).rejects.toThrow('Connection refused');
});

```

테스트를 실행하여 기존 동작이 보존되는지 확인하겠습니다.

[Bash: npm test -- LegacyEmailService]

모든 테스트 통과. 리팩토링이 성공적으로 완료되었습니다.

변경 요약:

- 콜백 패턴 → **async/await** 변환
- promisify로 기존 콜백 API를 래핑
- 에러 처리: **callback(err)** → **try/catch** (자동 전파)
- 반환값: **callback(null, result)** → **return result**
- 중첩 깊이: 3단계 → 0단계

| 5.5 멀티 파일 변경과 Plan 모드

2개 이상의 파일에 영향을 주는 대규모 변경은 Plan 모드를 사용하는 것이 안전합니다. Plan 모드에서 Claude Code는 먼저 변경 계획을 설명하고, 사용자가 승인한 후에 실행합니다.

Plan 모드 사용법

사용자: /plan 이 프로젝트에 역할 기반 접근 제어(RBAC)를 추가하고 싶어.

현재는 admin/user 두 가지 역할만 있는데,
커스텀 역할과 권한을 정의할 수 있도록 확장해줘.

Claude Code: RBAC 시스템 구현 계획을 수립하겠습니다.

[프로젝트 분석 중...]

=== RBAC 구현 계획 ===

1단계: 데이터 모델 변경 (prisma/schema.prisma)

- Role 모델 추가: id, name, description
- Permission 모델 추가: id, name, resource, action
- RolePermission 조인 테이블: roleId, permissionId
- User 모델에 roleId 외래키 추가 (기존 role string 필드 대체)

2단계: 서비스 계층 추가

- src/services/RoleService.ts: CRUD + 권한 할당/해제
- src/services/PermissionService.ts: 권한 정의 관리

3단계: 미들웨어 수정

- src/api/middlewares/auth.ts: 역할 기반 → 권한 기반 검증으로 변경
- requirePermission('products:write') 형태의 새 미들웨어

4단계: 기존 코드 마이그레이션

- roleMiddleware('admin') → requirePermission('resource:action')으로 교체
- 영향 파일: productRoutes.ts, userRoutes.ts, invoiceRoutes.ts

5단계: 시드 데이터

- 기본 역할(Admin, Editor, Viewer)과 권한 시드
- 기존 admin/user 사용자의 역할 마이그레이션

6단계: 테스트

- RoleService, PermissionService 단위 테스트
- 권한 미들웨어 통합 테스트

예상 변경 파일: 12개

예상 새 파일: 5개

이 계획으로 진행할까요? 수정하고 싶은 부분이 있으면 알려주세요.

Plan 모드는 실행 전에 전체 변경 사항을 검토할 수 있어, 대규모 변경에서 실수를 방지합니다. 계획이 마음에 들면 승인하고, 수정이 필요하면 피드백을 줄 수 있습니다.

사용자: 좋아, 근데 Permission 모델에 **scope** 필드도 추가해줘.
'own'이면 자기 리소스만, 'all'이면 모든 리소스에 접근할 수 있도록.
그리고 4단계부터 시작하지 말고 1~3단계까지만 먼저 진행해줘.

Claude Code: 이해했습니다. Permission 모델에 **scope** 필드를 추가하고,
1~3단계까지만 먼저 구현하겠습니다.

[작업 진행...]

5.6 체크포인트와 되돌리기

Claude Code로 코드를 수정하다 보면 원래 상태로 되돌리고 싶을 때가 있습니다. **/rewind** 명령은 대화와 코드 변경을 특정 시점으로 되돌릴 수 있게 해줍니다.

사용자: **/rewind**

Claude Code: 체크포인트 목록:

1. [10:32] **CategoryService** 생성 전
2. [10:35] **CategoryService** CRUD 구현 완료
3. [10:41] 테스트 작성 완료
4. [10:45] **RBAC** 1~3단계 구현 완료

어느 시점으로 되돌릴까요?

사용자: 3번으로 되돌려줘. **RBAC** 구현을 다시 하고 싶어.

Claude Code: 체크포인트 3 (10:41, 테스트 작성 완료)으로 되돌렸습니다.
RBAC 관련 변경 사항이 모두 원래 상태로 복원되었습니다.

`/diff` 로 변경 사항 확인하기

사용자: `/diff`

Claude Code: 현재 커밋되지 않은 변경 사항:

Modified: `prisma/schema.prisma`

+ `model Category { ... }`

New: `src/services/CategoryService.ts`

New: `src/api/controllers/CategoryController.ts`

New: `src/api/routes/categoryRoutes.ts`

New: `src/api/validators/categoryValidator.ts`

New: `tests/services/CategoryService.test.ts`

Modified: `src/api/routes/index.ts`

+ `import { categoryRoutes } from './categoryRoutes';`

+ `router.use('/categories', categoryRoutes);`

변경 파일 7개, 추가 547줄, 삭제 0줄

5.7 JavaScript에서 TypeScript로 변환

기존 JavaScript 프로젝트를 TypeScript로 전환하는 것은 Claude Code가 특히 잘 처리하는 작업 중 하나입니다.

사용자: @src/utils/helpers.js 를 TypeScript로 변환해줘.
모든 함수에 적절한 타입을 추가하고,
any 타입은 사용하지 말아줘.

Claude Code: helpers.js를 분석하여 TypeScript로 변환하겠습니다.

[Read: src/utils/helpers.js]

변환 계획:

- 파일명: helpers.js → helpers.ts
- 함수 5개에 타입 추가
- 인터페이스 2개 정의

[Write: src/utils/helpers.ts]

```
interface PaginationParams {
  page: number;
  limit: number;
  total: number;
}

interface PaginationResult<T> {
  data: T[];
  pagination: {
    currentPage: number;
    totalPages: number;
    totalItems: number;
    hasNext: boolean;
    hasPrev: boolean;
  };
}

export function paginate<T>(
  items: T[],
  params: PaginationParams
): PaginationResult<T> {
  const { page, limit, total } = params;
  const totalPages = Math.ceil(total / limit);

  return {
    data: items,
    pagination: {
      currentPage: page,
      totalPages,
      totalItems: total,
      hasNext: page < totalPages,
      hasPrev: page > 1,
    },
  };
}
```

```

    },
  };
}

export function slugify(text: string): string {
  return text
    .toLowerCase()
    .replace(/^[^w\s-]/g, '')
    .replace(/[\s_]+/g, '-')
    .trim();
}

export function retry<T>(
  fn: () => Promise<T>,
  maxAttempts: number = 3,
  delayMs: number = 1000
): Promise<T> {
  return new Promise(async (resolve, reject) => {
    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
      try {
        const result = await fn();
        return resolve(result);
      } catch (error) {
        if (attempt === maxAttempts) return reject(error);
        await new Promise(r => setTimeout(r, delayMs * attempt));
      }
    }
  });
}

```

// ... 나머지 함수들도 동일하게 타입 추가

[Bash: rm src/utils/helpers.js]

변환 완료. `import`하는 파일들도 확인하겠습니다.

[Grep: from.*helpers]

`import` 경로를 수정할 필요 없습니다 (TypeScript는 .ts 확장자 생략 가능).

빌드 확인:

[Bash: npx tsc --noEmit]

타입 에러 없이 빌드 성공합니다.

5.8 효과적인 코드 작성 요청 팁

좋은 지시의 특징

```
# 나쁜 예: 모호하고 범위가 불명확
사용자: API를 만들어줘

# 좋은 예: 구체적이고 기존 패턴을 참조
사용자: 기존 UserController의 패턴을 따라서
        GET /api/orders/:id 엔드포인트를 만들어줘.
        주문이 없으면 404를 반환하고,
        주문의 소유자가 아니면 403을 반환해야 해.
        OrderService는 이미 있으니까 사용해줘.
```

반복적 피드백 주기

Claude Code와의 코드 작성은 한 번에 완벽한 결과를 기대하기보다, 반복적으로 피드백을 주며 개선하는 것이 효과적입니다.

```
사용자: 좋아, 근데 에러 응답 형식을 우리 표준에 맞춰줘.
        status, message, errorCode 필드를 포함해야 해.

Claude Code: 에러 응답 형식을 수정하겠습니다.
[수정 진행...]

사용자: 에러 코드는 ORDER_NOT_FOUND, ORDER_ACCESS_DENIED 형태로 해줘.

Claude Code: 에러 코드를 상수로 정의하고 적용하겠습니다.
[수정 진행...]
```

이런 반복적 대화를 통해 정확히 원하는 결과를 얻을 수 있습니다.

정리

이 챕터에서 배운 핵심 내용을 정리합니다.

새 파일 생성

- Claude Code는 기존 코드 패턴을 분석하여 일관된 스타일로 새 파일을 생성합니다
- Controller-Service-Validator 같은 계층 구조를 자동으로 따릅니다

코드 수정과 리팩토링

- 특정 부분만 정확하게 수정하며, 변경 전후를 명확히 보여줍니다
- 콜백을 async/await로 변환하거나 JavaScript를 TypeScript로 전환하는 대규모 리팩토링도 가능합니다

테스트 작성

- 기존 테스트 스타일을 분석하여 일관된 테스트를 생성합니다
- 정상 케이스뿐 아니라 엣지 케이스와 에러 케이스도 포함합니다

Plan 모드와 안전장치

- `/plan` 으로 대규모 변경 전에 계획을 수립하고 검토합니다
- `/rewind` 로 변경 사항을 특정 시점으로 되돌릴 수 있습니다
- `/diff` 로 현재 변경 사항을 확인합니다

효과적인 요청법

- 기존 패턴 참조를 명시합니다 ("기존 UserController 패턴을 따라서")
- 구체적인 요구사항을 포함합니다 (에러 코드, HTTP 상태 코드, 검증 조건)
- 반복적 피드백으로 결과를 점진적으로 개선합니다

다음 **챕터**에서는 Claude Code와 Git을 함께 사용하여 커밋, 브랜치, PR 생성, 코드 리뷰를 자동화하는 방법을 학습합니다.

Chapter 6: Git과 함께하기

| 학습 목표

1. Claude Code를 활용하여 Git 워크플로(커밋, 브랜치, PR)를 자동화합니다
 2. 머지 충돌(Merge Conflict) 해결 과정에서 Claude Code의 도움을 받습니다
 3. PR 생성 및 코드 리뷰 프로세스에 Claude Code를 통합합니다
 4. Git 히스토리를 분석하고 변경 이력을 추적하는 방법을 배웁니다
-

| 6.1 자동 커밋

코드를 작성하거나 수정한 후 가장 흔하게 하는 작업이 Git 커밋입니다. Claude Code는 변경 내용을 분석하여 적절한 커밋 메시지를 자동으로 생성합니다.

실전 예제 1: 변경 사항을 분석한 지능형 커밋

사용자: 변경 사항을 커밋해줘

Claude Code: 현재 변경 사항을 분석하겠습니다.

```
[Bash: git status]
```

```
[Bash: git diff]
```

변경 사항 요약:

- prisma/schema.prisma: Category 모델 추가
- src/services/CategoryService.ts: CRUD 서비스 (신규)
- src/api/controllers/CategoryController.ts: 컨트롤러 (신규)
- src/api/routes/categoryRoutes.ts: 라우트 정의 (신규)
- src/api/validators/categoryValidator.ts: Zod 검증 스키마 (신규)
- src/api/routes/index.ts: 카테고리 라우트 등록
- tests/services/CategoryService.test.ts: 단위 테스트 (신규)

커밋 메시지를 작성하겠습니다.

```
[Bash: git add prisma/schema.prisma src/services/CategoryService.ts
src/api/controllers/CategoryController.ts
src/api/routes/categoryRoutes.ts
src/api/validators/categoryValidator.ts
src/api/routes/index.ts
tests/services/CategoryService.test.ts]
```

```
[Bash: git commit -m "feat: add category CRUD API with tree structure support
```

- ```
- Add Category model with self-referencing parent-child relation
- Implement CategoryService with findAll, findById, create, update, delete
- Add Zod validation schemas for create/update operations
- Prevent deletion of categories with subcategories
- Add 9 unit tests covering normal and edge cases"]
```

커밋이 완료되었습니다.

Claude Code가 생성한 커밋 메시지에 주목합니다. Conventional Commits(feat, fix, refactor 등) 형식을 따르며, 변경 내용을 구조적으로 요약합니다. 만약 프로젝트에 다른 커밋 메시지 규칙이 있다면, CLAUDE.md에 명시하여 Claude Code가 그 규칙을 따르도록 할 수 있습니다.

## 선택적 커밋

모든 변경 사항을 한 번에 커밋하지 않고, 논리적 단위로 나누어 커밋할 수도 있습니다.

사용자: 변경 사항을 논리적 단위로 나누어서 커밋해줘.  
스키마 변경과 코드 변경을 분리해줘.

Claude Code: 변경 사항을 분석하여 논리적 단위로 분리하겠습니다.

커밋 1: 데이터 모델 변경

```
[Bash: git add prisma/schema.prisma prisma/migrations/]
[Bash: git commit -m "feat(db): add Category model with tree structure
```

- Self-referencing relation for parent-child categories
- Add products relation for category-product mapping"]

커밋 2: 비즈니스 로직

```
[Bash: git add src/services/CategoryService.ts
src/api/validators/categoryValidator.ts]
[Bash: git commit -m "feat(service): implement CategoryService CRUD operations
```

- findAll returns only top-level categories with children count
- create validates parent category existence
- delete prevents removal of categories with subcategories"]

커밋 3: API 계층

```
[Bash: git add src/api/controllers/CategoryController.ts
src/api/routes/categoryRoutes.ts src/api/routes/index.ts]
[Bash: git commit -m "feat(api): add category REST API endpoints
```

- GET /api/categories - list all top-level categories
- GET /api/categories/:id - get category with children and products
- POST /api/categories - create (admin only)
- PUT /api/categories/:id - update (admin only)
- DELETE /api/categories/:id - delete (admin only)"]

커밋 4: 테스트

```
[Bash: git add tests/services/CategoryService.test.ts]
[Bash: git commit -m "test: add CategoryService unit tests
```

- 9 test cases covering CRUD operations
- Edge cases: empty results, non-existent entities, deletion constraints"]

4개의 커밋으로 분리 완료했습니다.

## 6.2 브랜치 관리

Claude Code는 브랜치 생성, 전환, 머지 등 Git 브랜치 관련 작업도 자연어로 수행할 수 있습니다.

사용자: payment-integration이라는 새 브랜치를 만들고 전환해줘

Claude Code:

```
[Bash: git checkout -b feature/payment-integration]
```

feature/payment-integration 브랜치를 생성하고 전환했습니다.

현재 브랜치: feature/payment-integration (main에서 분기)

사용자: 현재 브랜치 상태를 정리해줘. 어떤 브랜치들이 있고,  
main과 얼마나 차이가 있는지 보여줘.

Claude Code:

```
[Bash: git branch -a]
```

```
[Bash: git log --oneline main..HEAD]
```

```
[Bash: git log --oneline HEAD..main]
```

브랜치 현황:

|                               |                        |
|-------------------------------|------------------------|
| main                          | - 기본 브랜치               |
| feature/category-api          | - main보다 4커밋 앞 (머지 대기) |
| * feature/payment-integration | - main과 동일 (방금 생성)     |
| bugfix/token-invalidatation   | - main보다 3커밋 앞         |

원격 브랜치:

|                             |                     |
|-----------------------------|---------------------|
| origin/main                 | - 로컬 main과 동기화됨     |
| origin/feature/category-api | - 푸시됨, PR #12 오픈 상태 |

## 6.3 PR 생성

GitHub Pull Request(PR) 생성은 Claude Code의 강력한 기능 중 하나입니다. 변경 내용을 분석하여 제목, 설명, 리뷰 체크리스트를 자동으로 작성합니다.

## 실전 예제 2: PR 자동 생성

사용자: 이 브랜치의 변경 사항으로 PR을 만들어줘

Claude Code: 브랜치의 전체 변경 사항을 분석하겠습니다.

```
[Bash: git log --oneline main..HEAD]
```

```
[Bash: git diff main --stat]
```

```
[Bash: git diff main]
```

변경 사항 분석 완료. PR을 생성하겠습니다.

```
[Bash: git push -u origin feature/category-api]
```

```
[Bash: gh pr create --title "feat: Add category CRUD API with tree structure" --body
"$(cat <<'EOF'
```

```
Summary
```

- Add hierarchical category management system with parent-child relationships
- Implement full CRUD API endpoints with admin-only write operations
- Include Zod validation and comprehensive unit tests

```
Changes
```

- **Data Model**: New `Category` model with self-referencing tree structure
- **Service Layer**: `CategoryService` with 5 CRUD methods
- **API Endpoints**: 5 RESTful endpoints under `/api/categories`
- **Validation**: Zod schemas for create/update operations
- **Tests**: 9 unit tests covering normal flows and edge cases

```
Test plan
```

- [ ] Run `npm test` - all tests pass
- [ ] Test API endpoints with Postman/curl
- [ ] Verify category tree structure (parent-child relations)
- [ ] Test admin-only access control
- [ ] Verify deletion constraint (cannot delete category with children)

```
API Reference
```

| Method | Path                | Auth  | Description               |
|--------|---------------------|-------|---------------------------|
| GET    | /api/categories     | No    | List top-level categories |
| GET    | /api/categories/:id | No    | Get category details      |
| POST   | /api/categories     | Admin | Create category           |
| PUT    | /api/categories/:id | Admin | Update category           |
| DELETE | /api/categories/:id | Admin | Delete category           |

```
EOF
```

```
)]
```

PR이 생성되었습니다: <https://github.com/your-org/your-project/pull/12>

## 6.4 PR 코멘트 처리

PR을 올린 후 리뷰어의 코멘트를 받으면, Claude Code로 코멘트를 확인하고 수정 사항을 바로 반영할 수 있습니다.

## 실전 예제 3: 리뷰 코멘트 반영

사용자: /pr-comments 12

Claude Code: PR #12의 리뷰 코멘트를 가져오겠습니다.

[Bash: gh api repos/your-org/your-project/pulls/12/comments]

PR #12 리뷰 코멘트 (3개):

1. @reviewer1 (CategoryService.ts:23)  
"findAll에서 페이지네이션을 지원해야 합니다.  
카테고리가 수백 개가 되면 성능 문제가 생길 수 있어요."
2. @reviewer1 (CategoryController.ts:45)  
"create 응답에 Location 헤더를 추가해주세요. REST 모범 사례입니다."
3. @reviewer2 (categoryValidator.ts:8)  
"name 필드에 trim()을 추가해서 앞뒤 공백을 제거해주세요."

이 코멘트들을 반영할까요?

사용자: 네, 3개 모두 반영해줘

Claude Code: 리뷰 코멘트를 반영하겠습니다.

[Edit: src/services/CategoryService.ts]

// findAll에 페이지네이션 추가

```
static async findAll(page: number = 1, limit: number = 20) {
 const [categories, total] = await Promise.all([
 prisma.category.findMany({
 where: { parentId: null },
 include: { children: true, _count: { select: { products: true } } },
 skip: (page - 1) * limit,
 take: limit,
 orderBy: { name: 'asc' },
 }),
 prisma.category.count({ where: { parentId: null } }),
]);
 return { categories, total, page, limit, totalPages: Math.ceil(total / limit) };
}
```

[Edit: src/api/controllers/CategoryController.ts]

// create 응답에 Location 헤더 추가

```
static async create(req: Request, res: Response) {
 const category = await CategoryService.create(req.body);
 res
```



```
.status(201)
.location(`/api/categories/${category.id}`)
.json(category);
}
```

[Edit: src/api/validators/categoryValidator.ts]

// name에 trim() 추가

name: z.string().trim().min(1).max(100),

3개 코멘트를 모두 반영했습니다. 테스트를 실행하겠습니다.

[Bash: npm test]

모든 테스트 통과. 변경 사항을 커밋하고 푸시하겠습니다.

[Bash: git add -A]

[Bash: git commit -m "fix: address PR review comments"]

- Add pagination support to findAll (default: 20 items per page)
- Add Location header to create response (REST best practice)
- Add trim() to category name validation"]

[Bash: git push]

PR이 업데이트되었습니다.

## 6.5 머지 충돌 해결

머지 충돌(Merge Conflict)은 여러 브랜치에서 같은 파일을 수정했을 때 발생합니다. Claude Code는 충돌 내용을 분석하고 적절한 해결책을 제시합니다.

사용자: main 브랜치를 머지하려는데 충돌이 발생했어. 해결해줘.

Claude Code: 충돌 상황을 분석하겠습니다.

```
[Bash: git merge main]
Auto-merging src/api/routes/index.ts
CONFLICT (content): Merge conflict in src/api/routes/index.ts
Auto-merging src/services/ProductService.ts
CONFLICT (content): Merge conflict in src/services/ProductService.ts
```

2개 파일에서 충돌이 발생했습니다. 분석하겠습니다.

```
[Read: src/api/routes/index.ts]
[Read: src/services/ProductService.ts]
```

충돌 분석:

1. src/api/routes/index.ts (단순 충돌)
  - 우리 브랜치: categoryRoutes 추가
  - main 브랜치: orderRoutes 추가
  - 해결: 둘 다 포함 (각각 다른 라우트이므로 양쪽 모두 유지)
2. src/services/ProductService.ts (로직 충돌)
  - 우리 브랜치: getProducts()에 categoryId 필터 추가
  - main 브랜치: getProducts()에 페이지네이션 추가
  - 해결: 페이지네이션과 categoryId 필터 모두 포함

해결하겠습니다.

```
[Edit: src/api/routes/index.ts]
// 양쪽 변경 모두 포함
import { categoryRoutes } from './categoryRoutes';
import { orderRoutes } from './orderRoutes';

router.use('/categories', categoryRoutes);
router.use('/orders', orderRoutes);

[Edit: src/services/ProductService.ts]
// 페이지네이션 + categoryId 필터 통합
static async getProducts(options: {
 page?: number;
 limit?: number;
 categoryId?: string;
}) {
 const { page = 1, limit = 20, categoryId } = options;
 const where = categoryId ? { categoryId } : {};
```

```
const [products, total] = await Promise.all([
 prisma.product.findMany({
 where,
 skip: (page - 1) * limit,
 take: limit,
 }),
 prisma.product.count({ where }),
]);

return { products, total, page, limit };
}
```

충돌 해결 완료. 머지를 계속하겠습니다.

```
[Bash: git add src/api/routes/index.ts src/services/ProductService.ts]
```

```
[Bash: git commit -m "merge: resolve conflicts with main branch"]
```

- Combine category and order routes in index.ts
- Integrate pagination with categoryId filter in ProductService"]

머지가 성공적으로 완료되었습니다.

---

## 6.6 Git 히스토리 분석

Claude Code를 사용하면 Git 로그를 자연어로 분석하고, 특정 변경의 이유나 맥락을 파악할 수 있습니다.

사용자: 지난 2주간 auth 관련 파일에 어떤 변경이 있었는지 정리해줘.  
누가, 언제, 왜 변경했는지 포함해서.

Claude Code:

```
[Bash: git log --since="2 weeks ago" --all -- '**/auth*' '**/Auth*' --format="%h %ad %an %s" --date=short]
```

```
[Bash: git log --since="2 weeks ago" --all -- '**/auth*' '**/Auth*' -p]
```

지난 2주간 인증 관련 변경 이력:

| 날짜         | 작성자  | 커밋      | 내용                          |
|------------|------|---------|-----------------------------|
| 2026-03-13 | Kim  | a1b2c3d | JWT 만료 시간 15분→30분으로 변경      |
| 2026-03-11 | Park | d4e5f6g | refresh token rotation 구현   |
| 2026-03-10 | Park | h7i8j9k | 비밀번호 변경 시 토큰 무효화 추가         |
| 2026-03-08 | Lee  | l0m1n2o | OAuth Google 로그인 버그 수정      |
| 2026-03-03 | Kim  | p3q4r5s | rate limiting을 auth 라우트에 적용 |

주요 변경 상세:

- JWT 만료 시간 변경 (a1b2c3d)
  - 변경 이유: "사용자 경험 개선을 위해 세션 유지 시간 연장"
  - 영향: access token 만료가 15분에서 30분으로 변경
  - 보안 고려: refresh token rotation이 이미 구현되어 있으므로 수용 가능
- Refresh Token Rotation (d4e5f6g)
  - 변경 이유: "보안 강화 - 토큰 탈취 시 공격 윈도우 최소화"
  - 구현: refresh token 사용 시 새 토큰 발급 + 이전 토큰 무효화
  - 관련: 이전 커밋 h7i8j9k의 토큰 무효화 기능을 기반으로 구현
- OAuth 버그 수정 (l0m1n2o)
  - 문제: Google OAuth 콜백에서 이메일이 undefined인 경우 크래시
  - 수정: optional chaining과 이메일 필수 스코프 검증 추가

## 6.7 파이프라인을 활용한 코드 리뷰

Claude Code의 비대화형 모드(-p)와 파이프를 조합하면, 자동화된 코드 리뷰 파이프라인을 구축할 수 있습니다.

```
변경된 파일만 보안 리뷰
git diff main --name-only | claude -p "이 변경 파일들을 보안 관점에서 리뷰해줘. \
 SQL 인젝션, XSS, 인증 우회 가능성을 중점적으로 확인해줘."

커밋 메시지 품질 검사
git log --oneline -10 | claude -p "이 커밋 메시지들이 Conventional Commits 형식을 \
 따르는지 확인하고, 개선이 필요한 것을 알려줘."

PR 변경사항 요약 생성
git diff main...HEAD | claude -p "이 변경사항을 비개발자가 이해할 수 있도록 \
 한국어로 요약해줘. 릴리즈 노트에 포함할 내용 중심으로."
```

## 6.8 Git Worktree로 병렬 작업하기

`--worktree` (또는 `-w`) 플래그를 사용하면 격리된 Git Worktree에서 작업할 수 있습니다. 이는 현재 작업을 방해하지 않으면서 별도의 기능을 동시에 개발할 때 유용합니다.

```
별도의 worktree에서 인증 기능 작업 시작
claude -w feature-auth "인증 모듈에 2FA 지원을 추가해줘"

동시에 다른 worktree에서 결제 기능 작업
claude -w feature-payment "결제 서비스에 카카오페이 연동을 추가해줘"
```

각 worktree는 독립적인 작업 디렉터리를 가지므로, 파일 충돌 없이 여러 기능을 동시에 개발할 수 있습니다. 작업이 완료되면 각각의 브랜치를 main에 머지합니다.

## 정리

이 챕터에서 배운 핵심 내용을 정리합니다.

### 자동 커밋

- Claude Code는 변경 내용을 분석하여 Conventional Commits 형식의 커밋 메시지를 자동 생성합니다
- 논리적 단위로 나누어 커밋할 수도 있습니다

### PR 워크플로

- `gh` CLI를 통해 PR을 자동 생성하며, 제목/설명/체크리스트를 포함합니다

- `/pr-comments` 로 리뷰 코멘트를 가져와 바로 반영할 수 있습니다

### 충돌 해결

- Claude Code는 충돌 내용을 분석하여 양쪽 변경 의도를 이해하고 통합합니다
- 단순 충돌(같은 위치에 다른 내용 추가)부터 로직 충돌(같은 함수의 다른 측면 수정)까지 처리합니다

### Git 히스토리 분석

- 특정 기간, 특정 파일의 변경 이력을 자연어로 요약합니다
- 누가, 언제, 왜 변경했는지를 포함한 상세 분석이 가능합니다

### 자동화 파이프라인

- `git diff | claude -p` 패턴으로 자동 코드 리뷰, 변경 요약 등을 구현합니다
- `--worktree` 플래그로 격리된 환경에서 병렬 작업이 가능합니다

다음 챕터에서는 CLAUDE.md를 통해 프로젝트의 규칙과 지식을 Claude Code에게 기억시키는 방법을 학습합니다. 이를 통해 세션이 바뀌어도 일관된 코딩 스타일과 규칙을 유지할 수 있습니다.

---

# Chapter 7: CLAUDE.md로 프로젝트 기억시키기

## 학습 목표

1. CLAUDE.md의 역할과 동작 원리를 이해하고 효과적으로 작성합니다
2. Rules 시스템으로 프로젝트 규칙을 모듈화하여 관리합니다
3. Auto Memory 기능을 이해하고 세션 간 지식 전달을 최적화합니다
4. `/init` 명령으로 CLAUDE.md를 자동 생성하고 커스터마이징합니다

## 7.1 왜 CLAUDE.md가 필요한가

Claude Code의 대화 세션은 매번 새로운 컨텍스트 윈도우(context window)에서 시작됩니다. 즉, 이전 세션에서 "우리 프로젝트는 PostgreSQL을 쓰고, 2-space 들여쓰기를 사용하고, 테스트는 항상 커밋 전에 돌려야 해"라고 알려줬더라도, 새 세션에서는 이 정보를 모릅니다.

CLAUDE.md는 이 문제를 해결합니다. 매 세션 시작 시 자동으로 로드되어, Claude Code에게 프로젝트의 규칙, 구조, 관행을 알려주는 "프로젝트 브리핑 문서"입니다.

## CLAUDE.md가 없을 때 vs 있을 때

### # CLAUDE.md가 없을 때

사용자: 이 함수의 테스트를 작성해줘

Claude Code: (프로젝트 컨벤션을 모르므로)

- Mocha + Chai로 테스트 작성 (프로젝트는 Jest 사용)
- 4-space 들여쓰기 (프로젝트는 2-space)
- 테스트 파일을 `__tests__`/에 생성 (프로젝트는 tests/에)

### # CLAUDE.md가 있을 때 (테스트 규칙 명시)

사용자: 이 함수의 테스트를 작성해줘

Claude Code: (CLAUDE.md의 규칙을 따라)

- Jest + Supertest로 테스트 작성
- 2-space 들여쓰기
- 테스트 파일을 tests/에 생성
- describe/it 패턴, AAA 구조 사용

## 7.2 CLAUDE.md 파일의 위치와 범위

CLAUDE.md는 어디에 놓느냐에 따라 적용 범위가 달라집니다.

| 범위    | 위치                                                                        | 적용 대상          | 공유 방법  |
|-------|---------------------------------------------------------------------------|----------------|--------|
| 프로젝트  | <code>./CLAUDE.md</code> 또는 <code>./.claude/CLAUDE.md</code>              | 이 프로젝트의 모든 협업자 | Git 커밋 |
| 사용자   | <code>~/.claude/CLAUDE.md</code>                                          | 모든 프로젝트에서 나만   | 로컬만    |
| 관리 정책 | <code>/Library/Application Support/ClaudeCode/CLAUDE.md</code><br>(macOS) | 조직의 모든 사용자     | IT 배포  |

### 사용자 수준 CLAUDE.md

개인적인 선호사항은 사용자 수준 CLAUDE.md에 기록합니다. 이는 모든 프로젝트에 공통으로 적용됩니다.

### # ~/.claude/CLAUDE.md

#### ## 개인 선호사항

- 응답은 항상 한국어로 해주세요
- 코드 설명 시 "왜 이렇게 했는지" 이유를 반드시 포함해주세요
- 커밋 메시지는 영어로 작성해주세요
- 테스트를 작성할 때는 엡지 케이스를 반드시 포함해주세요



## 프로젝트 수준 CLAUDE.md

팀 전체가 공유하는 규칙은 프로젝트 루트에 CLAUDE.md를 작성합니다. 이 파일은 Git으로 버전 관리됩니다.

### # ./CLAUDE.md

#### ## 프로젝트 개요

전자상거래 플랫폼의 백엔드 API 서버.

Express.js + TypeScript + Prisma + PostgreSQL.

#### ## 빌드 & 테스트

- `npm run build` - TypeScript 컴파일
- `npm test` - 전체 테스트 실행
- `npm test -- --testPathPattern="파일명"` - 특정 테스트 실행
- `npm run lint` - ESLint 실행
- 커밋 전에 항상 `npm run lint && npm test` 실행

#### ## 코딩 표준

- 2-space 들여쓰기
- TypeScript strict 모드
- 세미콜론 사용
- 작은따옴표 사용
- 파일명은 camelCase (서비스, 유틸) 또는 PascalCase (컨트롤러, 모델)

#### ## 아키텍처

- Controller → Service → Model (Prisma) 계층 구조
- 모든 API 응답은 { data, error, pagination } 형태
- 에러 처리는 AppError 클래스 사용
- 환경 설정은 src/config/에서 관리

#### ## Git 규칙

- Conventional Commits 형식 사용 (feat, fix, refactor, test, docs, chore)
- PR은 최소 1명 리뷰 후 머지
- main 브랜치 직접 푸시 금지

## 7.3 효과적인 CLAUDE.md 작성법

### 포함해야 할 내용

CLAUDE.md에 다음 정보를 포함하면 Claude Code의 응답 품질이 크게 향상됩니다.

1. **빌드 및 테스트 명령어**: Claude Code가 코드를 수정한 후 바로 빌드/테스트를 실행할 수 있습니다
2. **코딩 표준**: 들여쓰기, 따옴표, 네이밍 규칙 등을 명시합니다

3. **아키텍처 결정:** "왜 이 기술을 선택했는지" 기록하여 Claude Code가 같은 패턴을 따르도록 합니다
4. **프로젝트 구조:** 주요 디렉터리와 파일의 역할을 설명합니다
5. **금지 사항:** "하지 말아야 할 것"을 명시합니다

## 실전 예제 1: 실제 프로젝트 CLAUDE.md

### # CLAUDE.md

#### ## 프로젝트: ShopWave API

온라인 쇼핑몰 백엔드 API. B2C 전자상거래 플랫폼.

#### ## 기술 스택

- Runtime: Node.js 20 + TypeScript 5.4
- Framework: Express 4.18
- ORM: Prisma 5.10 (PostgreSQL 16)
- Cache: Redis 7 (ioredis)
- Auth: JWT + bcrypt
- Validation: Zod
- Test: Jest + Supertest

#### ## 빌드 & 실행

- `npm run dev` - 개발 서버 (tsx watch)
- `npm run build` - TypeScript 컴파일
- `npm test` - Jest 전체 실행
- `npm test -- --testPathPattern="서비스명"` - 특정 테스트
- `npm run lint` - ESLint
- `npm run migrate` - Prisma 마이그레이션
- `npm run seed` - 시드 데이터 삽입

#### ## 코딩 규칙

- 2-space indent, single quotes, semicolons
- 파일명: camelCase.ts (서비스/유틸), PascalCase.ts (컨트롤러)
- 함수명: camelCase, 클래스명: PascalCase
- 인터페이스: 'I' 접두사 사용하지 않음 (User, not IUser)
- 모든 API 핸들러는 try-catch 대신 asyncHandler 래퍼 사용

#### ## 아키텍처 결정

- Controller → Service → Repository 패턴
- Repository 계층은 Prisma 직접 호출 (별도 추상화 없음)
- 모든 API 응답: { success: boolean, data?: T, error?: { code, message } }
- 페이지네이션: cursor 기반 (offset 사용하지 않음)
- 환경변수: src/config/env.ts에서 Zod로 검증 후 export

#### ## 금지 사항

- any 타입 사용 금지
- console.log 사용 금지 (logger 유틸 사용)
- .env 파일 읽기/수정 금지
- package-lock.json 직접 수정 금지
- DB에 직접 SQL 쿼리 금지 (Prisma 사용)

See @README.md for API documentation.  
See @prisma/schema.prisma for database schema.

## 분량에 대한 가이드

CLAUDE.md는 200줄 이하로 유지하는 것이 좋습니다. 이보다 길어지면 매 세션마다 많은 토큰을 소비하게 됩니다. 내용이 많아지면 Rules 시스템이나 외부 파일 임포트를 활용합니다.

---

## | 7.4 `/init` 으로 자동 생성하기

CLAUDE.md를 처음부터 직접 작성하는 것이 부담스러우면, `/init` 명령으로 Claude Code가 프로젝트를 분석하여 초기 CLAUDE.md를 자동 생성하도록 할 수 있습니다.

## 실전 예제 2: `/init` 명령 활용

사용자: `/init`

Claude Code: 프로젝트를 분석하여 `CLAUDE.md`를 생성하겠습니다.

```
[Glob: **/*]
[Read: package.json]
[Read: tsconfig.json]
[Read: .eslintrc.js]
[Read: .prettierrc]
[Read: jest.config.ts]
[Read: prisma/schema.prisma]
[Read: docker-compose.yml]
[Read: src/app.ts]
[Bash: git log --oneline -20]
```

프로젝트를 분석했습니다. 다음과 같은 `CLAUDE.md`를 생성합니다:

```
[Write: CLAUDE.md]
```

```
ShopWave API
```

Express.js + TypeScript 기반 전자상거래 API 서버.

```
Build & Test
```

- ``npm run dev`` - 개발 서버 (port `3000`)
- ``npm run build`` - TypeScript 컴파일
- ``npm test`` - Jest 테스트 실행
- ``npm run lint`` - ESLint 검사

```
Code Style
```

- TypeScript strict mode
- 2-space indent, single quotes, semicolons (Prettier)
- ESLint: `@typescript-eslint/recommended`

```
Architecture
```

- Controller → Service → Prisma 계층 구조
- `src/api/routes/`에서 라우트 정의
- `src/api/controllers/`에서 요청 처리
- `src/services/`에서 비즈니스 로직
- `src/config/`에서 환경 설정

```
Database
```

- PostgreSQL 16 (Prisma ORM)
- Redis 7 (세션, 캐시)
- ``npx prisma migrate dev`` - 마이그레이션

## Git Convention

- Conventional Commits (커밋 로그 분석 결과)
- feat, fix, refactor, test, docs 접두사 사용

CLAUDE.md가 생성되었습니다.

내용을 검토하고 프로젝트에 맞게 수정해주세요.

특히 '금지 사항'이나 팀 고유의 규칙을 추가하는 것을 권장합니다.

이미 CLAUDE.md가 존재하는 경우, `/init` 은 현재 내용을 분석하고 개선 사항을 제안합니다.

사용자: `/init`

Claude Code: 기존 CLAUDE.md를 분석했습니다. 개선 제안:

1. 테스트 명령어가 누락되어 있습니다.  
추가 제안: ``npm test -- --testPathPattern="파일명"``
2. 최근 커밋에서 Prisma migrate 명령이 자주 사용되는데,  
마이그레이션 명령어가 기록되어 있지 않습니다.
3. `.eslintrc.js`에 커스텀 규칙이 있는데 CLAUDE.md에 반영되지 않았습니다:
  - no-console 규칙 (logger 사용 권장)
  - import 순서 규칙

이 개선 사항을 적용할까요?

## 7.5 Rules 시스템: 규칙의 모듈화

프로젝트 규칙이 복잡해지면 CLAUDE.md 하나로 관리하기 어려워집니다. `.claude/rules/` 디렉터리에 주제별 규칙 파일을 분리하면 모듈화된 규칙 관리가 가능합니다.

## 디렉터리 구조

```
your-project/
├── .claude/
│ ├── CLAUDE.md # 핵심 프로젝트 정보 (간결하게)
│ └── rules/
│ ├── code-style.md # 코딩 스타일 규칙
│ ├── testing.md # 테스트 작성 규칙
│ ├── api-design.md # API 설계 규칙
│ └── security.md # 보안 규칙
```

## 실전 예제 3: 규칙 파일 작성

### 코딩 스타일 규칙 ( `code-style.md` )

#### # 코딩 스타일 규칙

##### ## TypeScript

- strict 모드 필수
- any 타입 사용 금지 – unknown 사용 후 타입 가드로 좁히기
- enum 대신 const object + as const 패턴 사용
- 타입 정의는 해당 모듈의 types.ts 파일에 분리

##### ## 네이밍

- 변수/함수: camelCase
- 클래스/타입/인터페이스: PascalCase
- 상수: UPPER\_SNAKE\_CASE
- 파일명: camelCase.ts (유틸, 서비스), PascalCase.ts (클래스, 컴포넌트)
- 테스트 파일: \*.test.ts

##### ## Import 순서

1. Node.js 내장 모듈
2. 외부 패키지
3. 프로젝트 내부 모듈 (절대 경로)
4. 상대 경로 import

## 테스트 규칙 ( `testing.md` )

### # 테스트 규칙

#### ## 프레임워크

- Jest + Supertest (통합 테스트)
- Prisma Mock (단위 테스트)

#### ## 구조

- describe로 모듈/클래스 그룹화
- it으로 개별 테스트 케이스 (한국어 설명)
- AAA 패턴: Arrange → Act → Assert

#### ## 필수 케이스

- 정상 동작 (happy path)
- 입력 검증 실패
- 리소스 없음 (404)
- 권한 없음 (401, 403)
- 엣지 케이스 (빈 배열, null, 경계값)

#### ## 규칙

- 테스트 간 의존성 금지 (각 테스트 독립적)
- 외부 서비스는 반드시 mock 처리
- 테스트 데이터는 각 테스트에서 직접 생성
- DB를 사용하는 통합 테스트는 트랜잭션 롤백 패턴 사용

## 경로별 규칙 적용

Rules 파일에 YAML 프론트매터(frontmatter)로 경로 패턴을 지정하면, 특정 디렉터리에서 작업할 때만 해당 규칙이 로드됩니다.

```

paths:
 - "src/api/**/*.ts"

```

### # API 개발 규칙

- 모든 엔드포인트에 Zod 입력 검증 포함
- 응답 형식: { *success*: boolean, *data*?: T, *error*?: { *code*: string, *message*: string } }
- HTTP 상태 코드를 정확하게 사용 (200, 201, 400, 401, 403, 404, 500)
- 모든 라우트에 OpenAPI JSDoc 주석 포함
- 파일 업로드 엔드포인트는 *multer* 미들웨어 사용



```

paths:
 - "prisma/**"

데이터베이스 규칙

- 마이그레이션 이름은 동사-명사 형식 (예: add-category-model)
- 모든 테이블에 createdAt, updatedAt 필드 포함
- 외래키에는 반드시 인덱스 추가
- soft delete 패턴 사용 (deletedAt 필드)
- 시드 데이터는 prisma/seed.ts에서 관리

```

이렇게 경로별 규칙을 설정하면, API 코드를 작성할 때는 API 규칙이, 데이터베이스 스키마를 수정할 때는 DB 규칙이 자동으로 적용됩니다. 컨텍스트를 절약하면서도 정확한 규칙을 적용할 수 있는 강력한 기능입니다.

## 7.6 외부 파일 임포트

CLAUDE.md에서 @ 참조를 사용하면 기존 프로젝트 문서를 재활용할 수 있습니다. README, package.json 등 이미 잘 정리된 문서가 있다면 중복 작성 없이 참조합니다.

```

CLAUDE.md

프로젝트 개요
See @README.md for project overview and setup guide.

의존성
See @package.json for available npm commands and dependencies.

DB 스키마
See @prisma/schema.prisma for database models.

추가 규칙
- @~/ .claude/my-project-notes.md (개인 메모, 체크인하지 않음)

```

@ 뒤에 파일 경로를 지정하면, Claude Code가 해당 파일의 내용을 자동으로 참조합니다. 이를 통해 CLAUDE.md를 간결하게 유지하면서도 풍부한 컨텍스트를 제공할 수 있습니다.

## 7.7 Auto Memory: Claude의 자동 학습

Auto Memory는 Claude Code가 세션 중에 학습한 내용을 자동으로 기록하는 기능입니다. CLAUDE.md가 사용자가 작성하는 "지시서"라면, Auto Memory는 Claude가 스스로 작성하는 "학습 노트"입니다.

### Auto Memory가 저장하는 내용

```
~/ .claude/projects/<project-hash>/memory/
├── MEMORY.md # 핵심 인덱스 (매 세션 시작 시 처음 200줄 로드)
├── debugging.md # 디버깅 중 발견한 패턴과 해결법
├── api-conventions.md # API 설계 결정 사항
└── build-quirks.md # 빌드 관련 특이사항
```

예를 들어, 한 세션에서 "npm test가 실패하는 이유가 환경 변수 DATABASE\_URL이 설정되지 않아서였다"는 것을 발견하면, Auto Memory에 이 내용이 기록됩니다. 다음 세션에서 같은 문제가 발생하면 Claude Code가 바로 원인을 파악할 수 있습니다.

### Auto Memory 관리

사용자: /memory

Claude Code: 현재 프로젝트의 메모리 파일을 엽니다.  
내용을 편집할 수 있습니다.

Auto Memory를 비활성화하려면 설정에서 끌 수 있습니다.

```
// settings.json
{
 "autoMemoryEnabled": false
}
```

또는 환경 변수로도 설정 가능합니다.

```
export CLAUDE_CODE_DISABLE_AUTO_MEMORY=1
```

## 7.8 CLAUDE.md 실전 운영 가이드

### 팀에서 CLAUDE.md 관리하기

1. **프로젝트 CLAUDE.md는 Git에 커밋합니다:** 팀원 전체가 같은 규칙을 공유합니다
2. **PR 리뷰 대상에 포함합니다:** 규칙 변경도 코드 변경처럼 리뷰합니다
3. **새 팀원 온보딩 시 활용합니다:** CLAUDE.md가 프로젝트의 "규칙서" 역할을 합니다

### 자주 하는 실수

1. **너무 많은 내용을 넣는 것:** 200줄을 넘으면 Rules로 분할합니다
2. **너무 일반적인 규칙만 넣는 것:** "코드를 깨끗하게 작성하세요" 같은 일반론은 효과가 없습니다. 구체적인 명령과 금지 사항이 효과적입니다
3. **업데이트하지 않는 것:** 프로젝트가 발전하면 CLAUDE.md도 함께 업데이트해야 합니다

### CLAUDE.md vs Rules vs Auto Memory 선택 기준

| 상황                  | 사용할 메커니즘                                           |
|---------------------|----------------------------------------------------|
| 빌드/테스트 명령어, 핵심 아키텍처 | CLAUDE.md (항상 로드)                                  |
| 특정 디렉터리에만 적용되는 규칙   | Rules (경로별 로드)                                     |
| 팀 코딩 표준, 보안 정책      | Rules (주제별 분리)                                     |
| 디버깅 중 발견한 패턴        | Auto Memory (자동 기록)                                |
| 개인 선호사항             | 사용자 CLAUDE.md ( <code>~/.claude/CLAUDE.md</code> ) |
| 조직 전체 정책            | 관리 정책 CLAUDE.md                                    |

## 정리

이 챕터에서 배운 핵심 내용을 정리합니다.

#### CLAUDE.md의 역할

- 세션 간 컨텍스트 유실 문제를 해결하는 "프로젝트 메모리"입니다
- 매 세션 시작 시 자동으로 로드되어 Claude Code에게 프로젝트 규칙을 알려줍니다

## 파일 위치와 범위

- 프로젝트 수준( `./CLAUDE.md` ): 팀 공유, Git 커밋
- 사용자 수준( `~/.claude/CLAUDE.md` ): 개인 설정, 모든 프로젝트 적용
- 관리 정책 수준: 조직 전체 적용

## 효과적인 작성법

- 빌드 명령, 코딩 표준, 아키텍처, 금지 사항을 구체적으로 명시합니다
- 200줄 이하로 유지하고, 초과 시 Rules로 분할합니다
- `/init` 으로 자동 생성 후 커스터마이징합니다

## Rules 시스템

- `.claude/rules/` 에 주제별(코딩 스타일, 테스트, API, 보안) 규칙 파일을 분리합니다
- YAML 프론트매터로 경로별 규칙 적용이 가능합니다

## Auto Memory

- Claude가 자동으로 학습한 내용을 기록합니다
- 디버깅 패턴, 빌드 특이사항 등이 자동 축적됩니다
- `/memory` 명령으로 확인 및 편집할 수 있습니다

## 외부 파일 임포트

- `@README.md` , `@package.json` 등으로 기존 문서를 참조합니다
- CLAUDE.md를 간결하게 유지하면서 풍부한 컨텍스트를 제공합니다

---

이것으로 Part 2 "핵심 기능 마스터"를 마칩니다. 코드 읽기(Chapter 4), 코드 쓰기(Chapter 5), Git 연동(Chapter 6), 프로젝트 메모리(Chapter 7)의 네 가지 핵심 역량을 학습했습니다. 이 네 가지가 Claude Code를 일상적인 개발 워크플로에 통합하는 기반입니다.

Part 3에서는 이 기반 위에 실전 프로젝트를 수행합니다. 웹 앱을 처음부터 구축하고, 레거시 코드를 리팩토링하고, 실제 버그를 추적하며, Claude Code와의 협업을 본격화합니다.

---

---

# Chapter 8: 웹 앱 만들기 — 처음부터 끝까지

## 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다:

1. 프로젝트 기획부터 배포 준비까지 Claude Code와 함께 웹 앱을 완성합니다
2. 프론트엔드와 백엔드를 아우르는 풀스택 개발 워크플로를 체득합니다
3. Plan 모드와 반복적 대화를 통한 점진적 개발 방법을 익힙니다

## 시나리오: "할 일 관리 앱을 만들어 봅시다"

여러분은 팀 내부에서 사용할 간단한 To-Do 웹 앱을 만들기로 했습니다. 기술 스택은 React(프론트엔드)와 Express(백엔드), 데이터 저장은 SQLite를 사용합니다. 프로젝트 이름은 `todo-app` 입니다.

이 챕터에서는 빈 디렉터리에서 시작해서 동작하는 웹 앱을 완성하기까지, Claude Code와 나누는 모든 대화를 따라가며 실제 개발 과정을 체험합니다.

### 8.1 프로젝트 설계: Plan 모드로 큰 그림 그리기

웹 앱을 만들기 전에, 먼저 전체 아키텍처를 설계해야 합니다. Claude Code의 Plan 모드를 활용하면 코드를 작성하기 전에 구조를 먼저 잡을 수 있습니다.

```
$ 빈 디렉터리에서 시작합니다
$ mkdir todo-app && cd todo-app
$ claude
```

Claude Code 세션이 시작되면, Plan 모드로 전환합니다.

```
사용자: /plan 할 일 관리 웹 앱을 만들려고 합니다. React + Express + SQLite
스택으로 CRUD 기능을 갖춘 풀스택 앱을 설계해주세요.
필요한 기능: 할 일 추가, 목록 조회, 완료 체크, 삭제, 필터링 (전체/진행중/완료)
```

Claude Code가 계획을 제시합니다:

Claude: 할 일 관리 웹 앱의 아키텍처를 설계하겠습니다.

## 프로젝트 구조

```
todo-app/
├── server/
│ ├── index.js # Express 서버 진입점
│ ├── database.js # SQLite 데이터베이스 설정
│ └── routes/
│ └── todos.js # Todo CRUD 라우트
├── client/
│ ├── public/
│ │ └── index.html
│ └── src/
│ ├── App.jsx # 메인 앱 컴포넌트
│ ├── components/
│ │ ├── TodoList.jsx # 할 일 목록
│ │ ├── TodoItem.jsx # 개별 할 일 항목
│ │ ├── TodoForm.jsx # 할 일 추가 폼
│ │ └── TodoFilter.jsx # 필터링 버튼
│ ├── hooks/
│ │ └── useTodos.js # Todo 상태 관리 커스텀 훅
│ └── api/
│ └── todoApi.js # API 호출 함수
├── package.json
└── CLAUDE.md
```

## API 엔드포인트

|          |                |                                               |
|----------|----------------|-----------------------------------------------|
| - GET    | /api/todos     | - 전체 목록 조회 (쿼리: ?filter=all active completed) |
| - POST   | /api/todos     | - 새 할 일 추가                                    |
| - PATCH  | /api/todos/:id | - 할 일 수정 (완료 토글, 텍스트 수정)                      |
| - DELETE | /api/todos/:id | - 할 일 삭제                                      |

## 데이터 모델

todos 테이블:

- id: INTEGER PRIMARY KEY AUTOINCREMENT
- text: TEXT NOT NULL
- completed: BOOLEAN DEFAULT 0
- created\_at: DATETIME DEFAULT CURRENT\_TIMESTAMP

이 계획을 승인하시겠습니까?

계획이 마음에 듭니다. 승인하고 구현을 시작합니다.

사용자: 좋습니다. 이 계획대로 진행해주세요. 먼저 프로젝트 초기화부터 시작합니다.

**팁:** Plan 모드에서 먼저 설계를 검토하면, Claude Code가 잘못된 방향으로 코드를 작성하는 것을 방지할 수 있습니다. 특히 여러 파일에 걸친 작업에서는 Plan 모드가 큰 도움이 됩니다.

## 8.2 프로젝트 초기화: 기반 구조 잡기

사용자: 프로젝트를 초기화해주세요. `package.json` 생성, 필요한 의존성 설치, 디렉터리 구조 생성까지 해주세요.

Claude Code가 일련의 작업을 자율적으로 수행합니다:

Claude: 프로젝트를 초기화하겠습니다.

> `npm init -y` 를 실행해도 될까요? [Y/n]

**Y** 를 누르면 Claude Code가 연쇄적으로 작업을 진행합니다:

1. `npm init -y` 로 `package.json` 생성
2. 필요한 패키지 설치: `express`, `better-sqlite3`, `cors` (백엔드), `react`, `react-dom`, `vite` (프론트엔드)
3. 디렉터리 구조 생성

**Claude:** 프로젝트 초기화를 완료했습니다. 설치된 의존성:

백엔드:

- express: 웹 서버 프레임워크
- better-sqlite3: **SQLite** 데이터베이스 드라이버
- cors: 크로스 오리진 요청 허용

프론트엔드:

- react, react-dom: **UI** 라이브러리
- @vitejs/plugin-react: **Vite React** 플러그인

개발:

- vite: 빌드 도구
- concurrently: 서버와 클라이언트 동시 실행

package.json의 scripts:

```
"dev": "concurrently \"npm run server\" \"npm run client\"",
"server": "node server/index.js",
"client": "vite --config client/vite.config.js"
```

이 시점에서 `/init` 명령으로 CLAUDE.md를 생성하는 것도 좋습니다.

사용자: /init



Claude: 프로젝트를 분석하여 CLAUDE.md를 생성했습니다.

### # Todo App

#### ## 빌드 & 실행

- `npm run dev`로 서버와 클라이언트 동시 실행
- `npm run server`로 백엔드만 실행 (포트 3001)
- `npm run client`로 프론트엔드만 실행 (포트 5173)

#### ## 기술 스택

- 백엔드: Express + better-sqlite3
- 프론트엔드: React + Vite
- 데이터베이스: SQLite (./database.sqlite)

#### ## 코딩 규칙

- ES 모듈 사용 (import/export)
- React 함수형 컴포넌트 + Hooks 패턴
- API 응답은 { data, error } 형태로 표준화

**핵심 포인트:** CLAUDE.md를 프로젝트 초기에 생성하면, 이후 세션에서 Claude Code가 프로젝트의 기술 스택과 규칙을 즉시 파악할 수 있습니다. 매번 반복해서 설명할 필요가 없어집니다.

## 8.3 백엔드 API 구축: 데이터베이스와 라우트

이제 백엔드를 구현합니다. 데이터베이스 설정부터 시작합니다.

**사용자:** 백엔드를 구현해주세요. SQLite 데이터베이스 설정과 Todo CRUD API를 만들어주세요. 에러 처리도 포함해주세요.

Claude Code가 파일을 생성합니다. 먼저 데이터베이스 설정 파일입니다:

```
// server/database.js - Claude Code가 생성한 코드

import Database from 'better-sqlite3';
import path from 'path';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const db = new Database(path.join(__dirname, '..', 'database.sqlite'));

// WAL 모드 활성화 (성능 향상)
db.pragma('journal_mode = WAL');

// 테이블 생성
db.exec(`
 CREATE TABLE IF NOT EXISTS todos (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 text TEXT NOT NULL,
 completed INTEGER DEFAULT 0,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP
)
`);

export default db;
```

이어서 라우트 파일을 생성합니다:

```
// server/routes/todos.js - Claude Code가 생성한 코드

import { Router } from 'express';
import db from '../database.js';

const router = Router();

// GET /api/todos - 전체 목록 조회
router.get('/', (req, res) => {
 try {
 const { filter } = req.query;
 let query = 'SELECT * FROM todos ORDER BY created_at DESC';

 if (filter === 'active') {
 query = 'SELECT * FROM todos WHERE completed = 0 ORDER BY created_at DESC';
 } else if (filter === 'completed') {
 query = 'SELECT * FROM todos WHERE completed = 1 ORDER BY created_at DESC';
 }

 const todos = db.prepare(query).all();
 res.json({ data: todos });
 } catch (error) {
 res.status(500).json({ error: '할 일 목록을 가져오는데 실패했습니다.' });
 }
});

// POST /api/todos - 새 할 일 추가
router.post('/', (req, res) => {
 try {
 const { text } = req.body;
 if (!text || text.trim() === '') {
 return res.status(400).json({ error: '할 일 내용을 입력해주세요.' });
 }

 const result = db.prepare('INSERT INTO todos (text) VALUES (?)').run(text.trim());
 const todo = db.prepare('SELECT * FROM todos WHERE id = ?').get(result.lastInsertRowid);
 res.status(201).json({ data: todo });
 } catch (error) {
 res.status(500).json({ error: '할 일을 추가하는데 실패했습니다.' });
 }
});

// PATCH /api/todos/:id - 할 일 수정
router.patch('/:id', (req, res) => {
 try {

```

```

const { id } = req.params;
const { text, completed } = req.body;

const existing = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
if (!existing) {
 return res.status(404).json({ error: '해당 할 일을 찾을 수 없습니다.' });
}

const updatedText = text !== undefined ? text.trim() : existing.text;
const updatedCompleted = completed !== undefined ? (completed ? 1 : 0) :
existing.completed;

db.prepare('UPDATE todos SET text = ?, completed = ? WHERE id = ?')
 .run(updatedText, updatedCompleted, id);

const todo = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
res.json({ data: todo });
} catch (error) {
 res.status(500).json({ error: '할 일을 수정하는데 실패했습니다.' });
}
});

// DELETE /api/todos/:id - 할 일 삭제
router.delete('/:id', (req, res) => {
 try {
 const { id } = req.params;
 const existing = db.prepare('SELECT * FROM todos WHERE id = ?').get(id);
 if (!existing) {
 return res.status(404).json({ error: '해당 할 일을 찾을 수 없습니다.' });
 }

 db.prepare('DELETE FROM todos WHERE id = ?').run(id);
 res.json({ data: { message: '삭제되었습니다.' } });
 } catch (error) {
 res.status(500).json({ error: '할 일을 삭제하는데 실패했습니다.' });
 }
});

export default router;

```

서버 진입점도 함께 생성합니다:

// server/index.js – Claude Code가 생성한 코드

```
import express from 'express';
import cors from 'cors';
import todoRoutes from './routes/todos.js';

const app = express();
const PORT = process.env.PORT || 3001;

app.use(cors());
app.use(express.json());

app.use('/api/todos', todoRoutes);

app.listen(PORT, () => {
 console.log(`서버가 포트 ${PORT}에서 실행 중입니다.`);
});
```

백엔드가 완성되었습니다. 바로 테스트해봅시다.

**사용자:** 서버를 실행하고 **curl**로 **API**를 테스트해주세요.

Claude: 서버를 실행하고 API를 테스트하겠습니다.

> node server/index.js & 를 실행해도 될까요? [Y/n]

서버가 포트 3001에서 실행 중입니다.

테스트 결과:

1. POST /api/todos - 할 일 추가

```
$ curl -X POST http://localhost:3001/api/todos \
 -H "Content-Type: application/json" \
 -d '{"text": "Claude Code 책 읽기"}'
→ {"data":{"id":1,"text":"Claude Code 책 읽기","completed":0,"created_at":"..."}}
```

✓

2. GET /api/todos - 목록 조회

```
$ curl http://localhost:3001/api/todos
→ {"data":[{"id":1,"text":"Claude Code 책 읽기","completed":0,"created_at":"..."}]}
```

✓

3. PATCH /api/todos/1 - 완료 토글

```
$ curl -X PATCH http://localhost:3001/api/todos/1 \
 -H "Content-Type: application/json" \
 -d '{"completed": true}'
→ {"data":{"id":1,"text":"Claude Code 책 읽기","completed":1,"created_at":"..."}}
```

✓

4. DELETE /api/todos/1 - 삭제

```
$ curl -X DELETE http://localhost:3001/api/todos/1
→ {"data":{"message":"삭제되었습니다."}} ✓
```

모든 API 엔드포인트가 정상 동작합니다.

**실전 팁:** Claude Code에게 "테스트해줘"라고 말하면, 자동으로 적절한 curl 명령이나 테스트 코드를 생성하고 실행합니다. 수동으로 Postman을 열 필요가 없습니다.

## 8.4 프론트엔드 UI 구현: 컴포넌트와 상태 관리

이제 프론트엔드를 구현합니다. API 호출 함수부터 시작합니다.

사용자: 프론트엔드를 구현해주세요. API 호출 함수, 커스텀 훅, 그리고 UI 컴포넌트를 만들어주세요. 깔끔하고 사용하기 편한 UI로 만들어주세요.

Claude Code가 순서대로 파일을 생성합니다.

```
// client/src/api/todoApi.js - API 호출 함수

const API_BASE = 'http://localhost:3001/api/todos';

export const todoApi = {
 async getAll(filter = 'all') {
 const url = filter === 'all' ? API_BASE : `${API_BASE}?filter=${filter}`;
 const res = await fetch(url);
 const json = await res.json();
 if (!res.ok) throw new Error(json.error);
 return json.data;
 },

 async create(text) {
 const res = await fetch(API_BASE, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ text }),
 });
 const json = await res.json();
 if (!res.ok) throw new Error(json.error);
 return json.data;
 },

 async update(id, updates) {
 const res = await fetch(`${API_BASE}/${id}`, {
 method: 'PATCH',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(updates),
 });
 const json = await res.json();
 if (!res.ok) throw new Error(json.error);
 return json.data;
 },

 async remove(id) {
 const res = await fetch(`${API_BASE}/${id}`, { method: 'DELETE' });
 const json = await res.json();
 if (!res.ok) throw new Error(json.error);
 return json.data;
 },
};
```

커스텀 훅으로 상태 관리를 캡슐화합니다:



// client/src/hooks/useTodos.js - 상태 관리 커스텀 훅

```
import { useState, useEffect, useCallback } from 'react';
import { todoApi } from '../api/todoApi';

export function useTodos() {
 const [todos, setTodos] = useState([]);
 const [filter, setFilter] = useState('all');
 const [loading, setLoading] = useState(true);
 const [error, setError] = useState(null);

 const fetchTodos = useCallback(async () => {
 try {
 setLoading(true);
 setError(null);
 const data = await todoApi.getAll(filter);
 setTodos(data);
 } catch (err) {
 setError(err.message);
 } finally {
 setLoading(false);
 }
 }, [filter]);

 useEffect(() => {
 fetchTodos();
 }, [fetchTodos]);

 const addTodo = async (text) => {
 try {
 const newTodo = await todoApi.create(text);
 setTodos(prev => [newTodo, ...prev]);
 } catch (err) {
 setError(err.message);
 }
 };

 const toggleTodo = async (id) => {
 try {
 const todo = todos.find(t => t.id === id);
 const updated = await todoApi.update(id, { completed: !todo.completed });
 setTodos(prev => prev.map(t => t.id === id ? updated : t));
 } catch (err) {
 setError(err.message);
 }
 };
};
```

```

const deleteTodo = async (id) => {
 try {
 await todoApi.remove(id);
 setTodos(prev => prev.filter(t => t.id !== id));
 } catch (err) {
 setError(err.message);
 }
};

return {
 todos, filter, loading, error,
 setFilter, addTodo, toggleTodo, deleteTodo,
};
}

```

이어서 UI 컴포넌트들을 생성합니다:

```

// client/src/components/ToDoForm.jsx - 할 일 추가 폼

import { useState } from 'react';

export function ToDoForm({ onAdd }) {
 const [text, setText] = useState('');

 const handleSubmit = (e) => {
 e.preventDefault();
 if (text.trim()) {
 onAdd(text.trim());
 setText('');
 }
 };

 return (
 <form onSubmit={handleSubmit} className="todo-form">
 <input
 type="text"
 value={text}
 onChange={(e) => setText(e.target.value)}
 placeholder="할 일을 입력하세요..."
 className="todo-input"
 />
 <button type="submit" className="todo-add-btn">추가</button>
 </form>
);
}

```

```
// client/src/components/ToDoItem.jsx - 개별 할 일 항목
```

```
export function ToDoItem({ todo, onToggle, onDelete }) {
 return (
 <li className={`todo-item ${todo.completed ? 'completed' : ''}`}>
 <label className="todo-label">
 <input
 type="checkbox"
 checked={!todo.completed}
 onChange={() => onToggle(todo.id)}
 />
 {todo.text}
 </label>
 <button
 onClick={() => onDelete(todo.id)}
 className="todo-delete-btn"
 aria-label="삭제"
 >
 ×
 </button>

);
}
```

```
// client/src/components/ToDoFilter.jsx - 필터링 버튼
```

```
const FILTERS = [
 { value: 'all', label: '전체' },
 { value: 'active', label: '진행중' },
 { value: 'completed', label: '완료' },
];

export function ToDoFilter({ current, onChange }) {
 return (
 <div className="todo-filter">
 {FILTERS.map(({ value, label }) => (
 <button
 key={value}
 className={`filter-btn ${current === value ? 'active' : ''}`}
 onClick={() => onChange(value)}
 >
 {label}
 </button>
))}
 </div>
);
}
```

```
// client/src/components/ToDoList.jsx - 할 일 목록

import { TodoItem } from './TodoItem';

export function ToDoList({ todos, onToggle, onDelete }) {
 if (todos.length === 0) {
 return <p className="empty-message">할 일이 없습니다. 새로운 할 일을 추가해보세요!</p>;
 }

 return (
 <ul className="todo-list">
 {todos.map(todo => (
 <TodoItem
 key={todo.id}
 todo={todo}
 onToggle={onToggle}
 onDelete={onDelete}
 />
))}

);
}
```

그리고 메인 App 컴포넌트에서 모두 조합합니다:

```
// client/src/App.jsx - 메인 앱 컴포넌트

import { useTodos } from './hooks/useTodos';
import { TodoForm } from './components/TodoForm';
import { TodoList } from './components/TodoList';
import { TodoFilter } from './components/TodoFilter';
import './App.css';

export default function App() {
 const {
 todos, filter, loading, error,
 setFilter, addTodo, toggleTodo, deleteTodo,
 } = useTodos();

 return (
 <div className="app">
 <h1>Todo App</h1>
 <TodoForm onAdd={addTodo} />
 <TodoFilter current={filter} onChange={setFilter} />
 {error && <p className="error-message">{error}</p>}
 {loading ? (
 <p className="loading">불러오는 중...</p>
) : (
 <TodoList
 todos={todos}
 onToggle={toggleTodo}
 onDelete={deleteTodo}
 />
)}
 <p className="todo-count">
 총 {todos.length}개의 할 일
 </p>
 </div>
);
}
```

사용자: CSS도 추가해서 보기 좋게 만들어주세요.

Claude: App.css를 생성하겠습니다. 깔끔한 미니멀 디자인으로 만들겠습니다.

Claude Code가 스타일을 추가하고 나면, 전체 앱을 실행할 수 있습니다.

**사용자:** `npm run dev`로 전체 앱을 실행해주세요.

Claude: > `npm run dev` 를 실행해도 될까요? [Y/n]

서버가 포트 3001에서 실행 중입니다.

클라이언트가 `http://localhost:5173`에서 실행 중입니다.

브라우저에서 `http://localhost:5173`을 열어 확인해보세요.

## 8.5 테스트 작성: 안정성 확보

동작하는 앱이 완성되었지만, 테스트가 없으면 불안합니다. Claude Code에게 테스트를 작성하도록 요청합니다.

**사용자:** 백엔드 API에 대한 테스트를 작성하고 실행해주세요.

Jest를 사용하고, 각 엔드포인트별로 정상 케이스와 에러 케이스를 테스트해주세요.

Claude: Jest와 supertest를 설치하고 테스트를 작성하겠습니다.

> `npm install --save-dev jest supertest @jest/globals` 를 실행해도 될까요? [Y/n]

Claude Code가 테스트 파일을 생성합니다:

```
// server/__tests__/todos.test.js - API 테스트 (Claude Code 생성)

import { describe, it, expect, beforeEach } from '@jest/globals';
import request from 'supertest';
import express from 'express';
import cors from 'cors';
import db from '../database.js';
import todoRoutes from '../routes/todos.js';

// 테스트용 앱 설정
const app = express();
app.use(cors());
app.use(express.json());
app.use('/api/todos', todoRoutes);

describe('Todo API', () => {
 beforeEach(() => {
 // 각 테스트 전에 테이블 초기화
 db.exec('DELETE FROM todos');
 });

 describe('GET /api/todos', () => {
 it('빈 목록을 반환합니다', async () => {
 const res = await request(app).get('/api/todos');
 expect(res.status).toBe(200);
 expect(res.body.data).toEqual([]);
 });

 it('등록된 할 일 목록을 반환합니다', async () => {
 db.prepare('INSERT INTO todos (text) VALUES (?)').run('테스트 할 일');
 const res = await request(app).get('/api/todos');
 expect(res.status).toBe(200);
 expect(res.body.data).toHaveLength(1);
 expect(res.body.data[0].text).toBe('테스트 할 일');
 });

 it('완료된 항목만 필터링합니다', async () => {
 db.prepare('INSERT INTO todos (text, completed) VALUES (?, ?)').run('진행중', 0);
 db.prepare('INSERT INTO todos (text, completed) VALUES (?, ?)').run('완료됨', 1);
 const res = await request(app).get('/api/todos?filter=completed');
 expect(res.status).toBe(200);
 expect(res.body.data).toHaveLength(1);
 expect(res.body.data[0].text).toBe('완료됨');
 });
 });
});
```



```

describe('POST /api/todos', () => {
 it('새 할 일을 추가합니다', async () => {
 const res = await request(app)
 .post('/api/todos')
 .send({ text: '새로운 할 일' });
 expect(res.status).toBe(201);
 expect(res.body.data.text).toBe('새로운 할 일');
 expect(res.body.data.completed).toBe(0);
 });

 it('빈 텍스트는 거부합니다', async () => {
 const res = await request(app)
 .post('/api/todos')
 .send({ text: '' });
 expect(res.status).toBe(400);
 expect(res.body.error).toBeDefined();
 });

 it('텍스트가 없으면 거부합니다', async () => {
 const res = await request(app)
 .post('/api/todos')
 .send({});
 expect(res.status).toBe(400);
 });
});

describe('PATCH /api/todos/:id', () => {
 it('완료 상태를 토글합니다', async () => {
 const { lastInsertRowid } = db.prepare('INSERT INTO todos (text) VALUES (?)').run('할 일');
 const res = await request(app)
 .patch(`/api/todos/${lastInsertRowid}`)
 .send({ completed: true });
 expect(res.status).toBe(200);
 expect(res.body.data.completed).toBe(1);
 });

 it('존재하지 않는 항목은 404를 반환합니다', async () => {
 const res = await request(app)
 .patch('/api/todos/999')
 .send({ completed: true });
 expect(res.status).toBe(404);
 });
});

describe('DELETE /api/todos/:id', () => {
 it('할 일을 삭제합니다', async () => {
 const { lastInsertRowid } = db.prepare('INSERT INTO todos (text) VALUES

```

```

(?'').run('삭제 대상');
 const res = await request(app).delete(`/api/todos/${lastInsertRowid}`);
 expect(res.status).toBe(200);

 // 삭제 확인
 const check = db.prepare('SELECT * FROM todos WHERE id =
?').get(lastInsertRowid);
 expect(check).toBeUndefined();
 });

 it('존재하지 않는 항목은 404를 반환합니다', async () => {
 const res = await request(app).delete('/api/todos/999');
 expect(res.status).toBe(404);
 });
});
});

```

Claude: 테스트를 실행하겠습니다.

> npx jest --verbose 를 실행해도 될까요? [Y/n]

```

PASS server/__tests__/todos.test.js
 Todo API
 GET /api/todos
 ✓ 빈 목록을 반환합니다 (15 ms)
 ✓ 등록된 할 일 목록을 반환합니다 (8 ms)
 ✓ 완료된 항목만 필터링합니다 (7 ms)
 POST /api/todos
 ✓ 새 할 일을 추가합니다 (9 ms)
 ✓ 빈 텍스트는 거부합니다 (4 ms)
 ✓ 텍스트가 없으면 거부합니다 (3 ms)
 PATCH /api/todos/:id
 ✓ 완료 상태를 토글합니다 (6 ms)
 ✓ 존재하지 않는 항목은 404를 반환합니다 (3 ms)
 DELETE /api/todos/:id
 ✓ 할 일을 삭제합니다 (5 ms)
 ✓ 존재하지 않는 항목은 404를 반환합니다 (3 ms)

```

Tests: 10 passed, 10 total

모든 테스트가 통과했습니다.

## 8.6 마무리: 커밋과 배포 준비

프로젝트가 동작하고 테스트도 통과했으니, git으로 커밋합니다.

**사용자:** git 저장소를 초기화하고, .gitignore를 만들고,  
적절한 커밋 메시지로 첫 커밋을 해주세요.

Claude: Git 저장소를 초기화하고 커밋하겠습니다.

```
> git init 을 실행해도 될까요? [Y/n]
```

.gitignore를 생성합니다:

```
node_modules/
database.sqlite
dist/
```

```
> git add -A && git commit -m "feat: Todo 웹 앱 초기 구현"
```

- Express + SQLite 백엔드 API (CRUD 엔드포인트)
- React + Vite 프론트엔드 UI
- 필터링 기능 (전체/진행중/완료)
- API 테스트 (Jest + supertest, 10개 통과)
- CLAUDE.md 프로젝트 메모리 설정" 을 실행해도 될까요? [Y/n]

커밋이 완료되었습니다. [main (root-commit) a1b2c3d]

마지막으로 프로덕션 빌드 설정도 요청합니다.

**사용자:** 프로덕션 배포를 위한 빌드 스크립트를 추가하고, Express에서 빌드된  
프론트엔드를 정적 파일로 서빙하도록 설정해주세요.

**Claude:** 프로덕션 설정을 추가하겠습니다.

Claude Code가 server/index.js에 정적 파일 서빙 로직을 추가하고, package.json에 빌드 스크립트를 추가합니다:

**Before** — server/index.js (개발 전용):

```
app.use('/api/todos', todoRoutes);

app.listen(PORT, () => {
 console.log(`서버가 포트 ${PORT}에서 실행 중입니다.`);
});
```

**After** — server/index.js (프로덕션 지원):

```
app.use('/api/todos', todoRoutes);

// 프로덕션: 빌드된 프론트엔드를 정적 파일로 서빙
if (process.env.NODE_ENV === 'production') {
 const clientDist = path.join(__dirname, '..', 'client', 'dist');
 app.use(express.static(clientDist));
 app.get('*', (req, res) => {
 res.sendFile(path.join(clientDist, 'index.html'));
 });
}

app.listen(PORT, () => {
 console.log(`서버가 포트 ${PORT}에서 실행 중입니다. (${process.env.NODE_ENV || 'development'})`);
});
```

## 8.7 되돌아보기: Claude Code와의 협업 패턴

이 프로젝트에서 사용한 핵심 패턴을 정리합니다:

| 단계    | 사용한 기법      | 핵심 명령/대화                     |
|-------|-------------|------------------------------|
| 설계    | Plan 모드     | <code>/plan</code> 아키텍처 설계해줘 |
| 초기화   | 자율 실행       | 프로젝트를 초기화해줘                  |
| 메모리   | CLAUDE.md   | <code>/init</code>           |
| 백엔드   | 코드 생성 + 테스트 | API를 구현하고 테스트해줘              |
| 프론트엔드 | 멀티 파일 생성    | 프론트엔드를 구현해줘                  |
| 검증    | 자동 테스트      | 테스트를 작성하고 실행해줘               |
| 저장    | Git 자동화     | 커밋해줘                         |

## | 정리

이 챕터에서 우리는 빈 디렉터리에서 시작하여 완전히 동작하는 To-Do 웹 앱을 만들었습니다. 그 과정에서 배운 핵심 교훈은 다음과 같습니다:

1. **Plan 모드로 먼저 설계하세요:** 코드를 작성하기 전에 `/plan` 으로 구조를 잡으면 시행착오를 크게 줄일 수 있습니다.
2. **단계적으로 진행하세요:** "전부 다 만들어줘" 대신 백엔드 -> 프론트엔드 -> 테스트 순서로 나누어 요청하면 더 좋은 결과를 얻습니다.
3. **즉시 검증하세요:** 각 단계가 끝나면 바로 테스트하고 실행해보는 습관이 중요합니다. Claude Code는 테스트를 작성하고 실행하는 것까지 자율적으로 수행할 수 있습니다.
4. **CLAUDE.md를 초기에 설정하세요:** 프로젝트의 기술 스택과 규칙을 기록해두면 이후 세션에서 반복 설명이 필요 없습니다.
5. **Git 워크플로를 활용하세요:** Claude Code는 적절한 커밋 메시지를 자동으로 생성하고 .gitignore 같은 설정도 함께 관리합니다.

# Chapter 9: 레거시 코드 리팩토링

---

## | 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다:

1. 레거시 코드베이스를 Claude Code로 분석하고 개선 포인트를 식별합니다
2. 단계적 리팩토링 전략을 수립하고 안전하게 실행합니다
3. 리팩토링 과정에서 기존 기능이 깨지지 않도록 테스트를 활용합니다

## | 시나리오: "이 코드, 누가 이렇게 짰 거예요?"

여러분은 팀에 합류한 지 일주일 된 개발자입니다. 선임 개발자가 떠나면서 남긴 주문 처리 모듈을 인수받았는데, 코드 상태가 심각합니다. 하나의 파일에 500줄이 넘는 함수가 있고, 콜백 지옥에 빠져 있으며, 하드코딩된 값이 곳곳에 박혀 있습니다. 테스트 코드는 한 줄도 없습니다.

이 챕터에서는 이 혼란스러운 코드를 Claude Code와 함께 단계적으로 개선하는 과정을 따라갑니다.

---

### 9.1 레거시 코드 분석: 무엇이 문제인가

먼저, 문제의 코드를 살펴봅시다. 이것은 우리가 인수받은 `orderProcessor.js` 파일의 일부입니다:

```
// orderProcessor.js - 레거시 코드 (Before)

const mysql = require('mysql');
const http = require('http');

function processOrder(orderId, callback) {
 var connection = mysql.createConnection({
 host: '192.168.1.100',
 user: 'admin',
 password: 'admin123!',
 database: 'shop_db'
 });

 connection.connect(function(err) {
 if (err) {
 callback(err);
 return;
 }

 connection.query('SELECT * FROM orders WHERE id = ' + orderId, function(err,
results) {
 if (err) {
 callback(err);
 return;
 }

 if (results.length === 0) {
 callback(new Error('Order not found'));
 return;
 }

 var order = results[0];

 // 가격 계산
 var total = 0;
 for (var i = 0; i < order.items.length; i++) {
 total += order.items[i].price * order.items[i].quantity;
 }

 // 할인 적용
 if (total > 100000) {
 total = total * 0.9; // 10% 할인
 } else if (total > 50000) {
 total = total * 0.95; // 5% 할인
 }

 // 배송비 추가
```

```

 if (total < 30000) {
 total += 3000;
 }

 // 세금 추가
 total = total * 1.1; // 10% 세금

 // 결제 처리
 var paymentData = JSON.stringify({
 orderId: orderId,
 amount: total,
 cardNumber: order.card_number
 });

 var options = {
 hostname: '192.168.1.200',
 port: 8080,
 path: '/api/payment',
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'X-API-Key': 'sk_live_abc123def456'
 }
 };

 var req = http.request(options, function(res) {
 var data = '';
 res.on('data', function(chunk) { data += chunk; });
 res.on('end', function() {
 var paymentResult = JSON.parse(data);
 if (paymentResult.status === 'success') {
 // 주문 상태 업데이트
 connection.query(
 "UPDATE orders SET status = 'paid', total = " + total + " WHERE id = "
+ orderId,
 function(err) {
 if (err) {
 callback(err);
 return;
 }

 // 이메일 발송
 var emailReq = http.request({
 hostname: '192.168.1.200',
 port: 8080,
 path: '/api/email',
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',

```



```

 'X-API-Key': 'sk_live_abc123def456'
 }
}, function(emailRes) {
 var emailData = '';
 emailRes.on('data', function(chunk) { emailData += chunk; });
 emailRes.on('end', function() {
 connection.end();
 callback(null, { orderId: orderId, total: total, status:
'completed' });
 });
});

emailReq.write(JSON.stringify({
 to: order.email,
 subject: '주문 완료',
 body: '주문 #' + orderId + '이(가) 처리되었습니다. 총액: ' + total + '원'
}));
emailReq.end();
}
);
} else {
 connection.end();
 callback(new Error('Payment failed: ' + paymentResult.message));
}
});
});

req.write(paymentData);
req.end();
});
});
}

module.exports = { processOrder };

```

이 코드를 Claude Code에게 분석해달라고 요청합니다:

사용자: @orderProcessor.js 이 파일을 분석해주세요. 코드 품질 문제, 보안 취약점, 개선이 필요한 부분을 모두 찾아주세요.

Claude: orderProcessor.js를 분석한 결과, 다음과 같은 심각한 문제들이 발견됩니다:

#### ## 보안 취약점 (긴급)

1. **\*\*SQL 인젝션\*\***: 문자열 연결로 쿼리 생성 → 매개변수화된 쿼리 필요
2. **\*\*하드코딩된 비밀번호\*\***: DB 비밀번호 'admin123!'가 소스 코드에 노출
3. **\*\*하드코딩된 API 키\*\***: 'sk\_live\_abc123def456'가 소스 코드에 노출
4. **\*\*하드코딩된 서버 주소\*\***: IP 주소가 직접 코드에 포함

#### ## 코드 품질 문제

5. **\*\*콜백 지옥\*\***: 5단계 이상 중첩된 콜백 구조
6. **\*\*단일 책임 원칙 위반\*\***: 하나의 함수가 DB 연결, 쿼리, 가격 계산, 결제, 이메일까지 모두 처리
7. **\*\*매직 넘버\*\***: 100000, 50000, 0.9, 0.95, 3000, 1.1 등 의미 불명의 숫자
8. **\*\*에러 처리 불완전\*\***: HTTP 요청 에러 핸들링 누락
9. **\*\*DB 연결 누수\*\***: 에러 시 connection.end()가 호출되지 않는 경로 존재
10. **\*\*var 사용\*\***: ES6 이전 문법으로 스코프 문제 발생 가능
11. **\*\*CommonJS\*\***: 레거시 모듈 시스템 사용

#### ## 아키텍처 문제

12. **\*\*테스트 불가\*\***: 외부 의존성이 직접 연결되어 단위 테스트 불가능
13. **\*\*재사용 불가\*\***: 가격 계산 로직이 함수 내부에 매몰

개선 우선순위를 제안할까요?

정확한 분석입니다. 이제 리팩토링 계획을 세웁니다.

사용자: /plan 이 코드를 단계적으로 리팩토링해주세요.

한 번에 다 바꾸지 말고, 단계마다 기존 기능이 유지되는지 확인할 수 있도록 안전하게 진행해주세요.

Claude: 안전한 리팩토링을 위한 5단계 계획을 제안합니다:

## ## 리팩토링 계획

### ### 1단계: 테스트 안전망 구축 (먼저!)

- 현재 동작을 보존하는 통합 테스트 작성
- 외부 의존성 모킹 설정

### ### 2단계: 보안 취약점 수정 (긴급)

- 하드코딩된 비밀번호를 환경 변수로 이동
- SQL 인젝션 방지 (매개변수화된 쿼리)

### ### 3단계: 구조 분해 (핵심)

- 단일 거대 함수를 작은 함수들로 분리
- 가격 계산, 결제 처리, 이메일 발송 각각 독립 모듈화

### ### 4단계: 모던 문법 전환

- var → const/let
- 콜백 → async/await
- CommonJS → ES 모듈

### ### 5단계: 최종 정리

- 에러 처리 강화
- 타입 검증 추가
- 단위 테스트 추가

각 단계 완료 후 테스트를 실행하여 기능이 보존되는지 확인합니다.

이 계획으로 진행할까요?

## 9.2 1단계: 테스트 안전망 구축

리팩토링에서 가장 중요한 것은 "기존 기능을 깨뜨리지 않는 것"입니다. 먼저 테스트를 작성합니다.

**사용자:** 1단계를 진행해주세요. 현재 `processOrder` 함수의 동작을 검증하는 테스트를 작성해주세요. 외부 의존성(DB, HTTP)은 모킹해주세요.

Claude Code가 테스트 파일을 생성합니다:

```
// __tests__/orderProcessor.test.js - Claude Code가 생성한 테스트

import { jest, describe, it, expect, beforeEach } from '@jest/globals';

// 모킹: DB와 HTTP 의존성을 가짜로 대체
jest.unstable_mockModule('mysql', () => ({
 createConnection: jest.fn(() => ({
 connect: jest.fn((cb) => cb(null)),
 query: jest.fn(),
 end: jest.fn(),
 })),
}));

const mockHttpRequest = jest.fn();
jest.unstable_mockModule('http', () => ({
 request: mockHttpRequest,
}));

describe('processOrder', () => {
 let processOrder;
 let mockConnection;

 beforeEach(async () => {
 jest.clearAllMocks();
 const module = await import('../orderProcessor.js');
 processOrder = module.processOrder;
 });

 it('정상적인 주문을 처리합니다', (done) => {
 // ... 기존 동작을 검증하는 테스트
 // (모킹된 DB가 주문 데이터를 반환하고,
 // 모킹된 결제 API가 성공을 반환하는 시나리오)
 done();
 });

 it('주문이 없으면 에러를 반환합니다', (done) => {
 // 빈 결과 반환 시 에러 콜백 확인
 done();
 });

 it('10만원 이상 주문에 10% 할인을 적용합니다', (done) => {
 // 할인 로직 검증
 done();
 });

 it('결제 실패 시 에러를 반환합니다', (done) => {
 // 결제 API 실패 시나리오
 });
}
```

```
 done();
 });
});
```

**Claude:** 테스트를 실행했습니다.

**Tests:** 4 passed, 4 total

테스트 안전망이 구축되었습니다. 이제 리팩토링을 시작해도 안전합니다.

## 9.3 2단계: 보안 취약점 수정

**사용자:** 2단계를 진행해주세요. 하드코딩된 비밀번호를 환경 변수로 이동하고, SQL 인젝션을 수정해주세요.

**Before** — 하드코딩된 비밀번호:

```
var connection = mysql.createConnection({
 host: '192.168.1.100',
 user: 'admin',
 password: 'admin123!',
 database: 'shop_db'
});
```

**After** — 환경 변수 사용:

```
const connection = mysql.createConnection({
 host: process.env.DB_HOST,
 user: process.env.DB_USER,
 password: process.env.DB_PASSWORD,
 database: process.env.DB_NAME,
});
```

**Before** — SQL 인젝션 취약 코드:

```
connection.query('SELECT * FROM orders WHERE id = ' + orderId, function(err, results) {
```

**After** — 매개변수화된 쿼리:

```
connection.query('SELECT * FROM orders WHERE id = ?', [orderId], function(err, results) {
```

Claude Code가 `.env.example` 파일도 함께 생성합니다:

```
.env.example — 환경 변수 템플릿
DB_HOST=localhost
DB_USER=your_db_user
DB_PASSWORD=your_db_password
DB_NAME=shop_db
PAYMENT_API_HOST=localhost
PAYMENT_API_PORT=8080
PAYMENT_API_KEY=your_api_key
```

**사용자:** 테스트를 실행해서 기능이 유지되는지 확인해주세요.

**Claude:** > `npx jest --verbose` 를 실행해도 될까요? [Y/n]

**Tests:** 4 passed, 4 total

모든 테스트가 통과합니다. 2단계가 안전하게 완료되었습니다.

## 9.4 3단계: 구조 분해 — 거대 함수 쪼개기

이제 핵심 리팩토링입니다. 500줄짜리 단일 함수를 작은 모듈들로 분리합니다.

사용자: 3단계를 진행해주세요. processOrder 함수를 다음 모듈들로 분리해주세요:

1. 가격 계산 (calculatePrice)
2. 결제 처리 (processPayment)
3. 이메일 발송 (sendEmail)
4. 주문 처리 오케스트레이터 (processOrder - 위 3개를 조합)

Claude Code가 각 모듈을 별도 파일로 분리합니다:

```
// services/pricing.js - 가격 계산 모듈 (After)

const DISCOUNT_RULES = [
 { threshold: 100000, rate: 0.10 }, // 10만원 이상: 10% 할인
 { threshold: 50000, rate: 0.05 }, // 5만원 이상: 5% 할인
];

const FREE_SHIPPING_THRESHOLD = 30000;
const SHIPPING_FEE = 3000;
const TAX_RATE = 0.10;

export function calculateSubtotal(items) {
 return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}

export function calculateDiscount(subtotal) {
 for (const rule of DISCOUNT_RULES) {
 if (subtotal > rule.threshold) {
 return subtotal * rule.rate;
 }
 }
 return 0;
}

export function calculateShipping(subtotalAfterDiscount) {
 return subtotalAfterDiscount < FREE_SHIPPING_THRESHOLD ? SHIPPING_FEE : 0;
}

export function calculateTax(amount) {
 return amount * TAX_RATE;
}

export function calculateTotal(items) {
 const subtotal = calculateSubtotal(items);
 const discount = calculateDiscount(subtotal);
 const afterDiscount = subtotal - discount;
 const shipping = calculateShipping(afterDiscount);
 const beforeTax = afterDiscount + shipping;
 const tax = calculateTax(beforeTax);

 return {
 subtotal,
 discount,
 shipping,
 tax,
 total: beforeTax + tax,
 };
}
```



```
};
}
```

이전에 매직 넘버로 흩어져 있던 비즈니스 로직이 명확한 이름의 상수와 함수로 정리되었습니다. 이 코드를 원래의 인라인 코드와 비교해봅시다:

**Before** — 매직 넘버가 가득한 인라인 계산:

```
var total = 0;
for (var i = 0; i < order.items.length; i++) {
 total += order.items[i].price * order.items[i].quantity;
}
if (total > 100000) {
 total = total * 0.9;
} else if (total > 50000) {
 total = total * 0.95;
}
if (total < 30000) {
 total += 3000;
}
total = total * 1.1;
```

**After** — 의미가 명확한 함수 호출:

```
const pricing = calculateTotal(order.items);
// pricing.subtotal: 1,200,000
// pricing.discount: 120,000 (10%)
// pricing.shipping: 0 (면제)
// pricing.tax: 108,000
// pricing.total: 1,188,000
```

가격 계산 모듈은 이제 독립적으로 테스트할 수 있습니다:

```
// __tests__/pricing.test.js - 가격 계산 단위 테스트

import { calculateTotal } from '../services/pricing.js';

describe('가격 계산', () => {
 it('기본 소계를 계산합니다', () => {
 const items = [{ price: 10000, quantity: 2 }];
 const result = calculateTotal(items);
 expect(result.subtotal).toBe(20000);
 });

 it('10만원 이상이면 10% 할인을 적용합니다', () => {
 const items = [{ price: 60000, quantity: 2 }]; // 120,000원
 const result = calculateTotal(items);
 expect(result.discount).toBe(12000);
 });

 it('5만원 이상 10만원 미만이면 5% 할인을 적용합니다', () => {
 const items = [{ price: 30000, quantity: 2 }]; // 60,000원
 const result = calculateTotal(items);
 expect(result.discount).toBe(3000);
 });

 it('3만원 미만이면 배송비를 추가합니다', () => {
 const items = [{ price: 5000, quantity: 1 }]; // 5,000원
 const result = calculateTotal(items);
 expect(result.shipping).toBe(3000);
 });

 it('세금 10%를 적용합니다', () => {
 const items = [{ price: 10000, quantity: 1 }]; // 10,000원
 const result = calculateTotal(items);
 // 10,000(소계) + 3,000(배송) = 13,000 → 세금 1,300
 expect(result.tax).toBe(1300);
 });
});
```

결제와 이메일 모듈도 유사하게 분리합니다:

```
// services/payment.js - 결제 처리 모듈 (After)

export async function processPayment(orderId, amount, cardNumber) {
 const response = await fetch(`${process.env.PAYMENT_API_URL}/api/payment`, {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'X-API-Key': process.env.PAYMENT_API_KEY,
 },
 body: JSON.stringify({ orderId, amount, cardNumber }),
 });

 const result = await response.json();
 if (result.status !== 'success') {
 throw new Error(`결제 실패: ${result.message}`);
 }
 return result;
}
```

```
// services/email.js - 이메일 발송 모듈 (After)

export async function sendOrderConfirmation(email, orderId, total) {
 const response = await fetch(`${process.env.PAYMENT_API_URL}/api/email`, {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'X-API-Key': process.env.PAYMENT_API_KEY,
 },
 body: JSON.stringify({
 to: email,
 subject: '주문 완료',
 body: `주문 #${orderId}이(가) 처리되었습니다. 총액: ${total.toLocaleString()}원`,
 }),
 });

 return response.json();
}
```

## 9.5 4단계: 모던 문법 전환 — 콜백에서 async/await로

이제 오케스트레이터 함수를 async/await로 재작성합니다.

**Before** — 콜백 지옥:

```
function processOrder(orderId, callback) {
 var connection = mysql.createConnection({...});
 connection.connect(function(err) {
 if (err) { callback(err); return; }
 connection.query('SELECT...', function(err, results) {
 if (err) { callback(err); return; }
 // ... 결제 처리 콜백 ...
 // ... 상태 업데이트 콜백 ...
 // ... 이메일 발송 콜백 ...
 connection.end();
 callback(null, result);
 });
 });
}
```

**After** — 깔끔한 async/await:

```
// orderProcessor.js - 리팩토링 완료 (After)

import { getOrder, updateOrderStatus } from './services/database.js';
import { calculateTotal } from './services/pricing.js';
import { processPayment } from './services/payment.js';
import { sendOrderConfirmation } from './services/email.js';

export async function processOrder(orderId) {
 // 1. 주문 조회
 const order = await getOrder(orderId);
 if (!order) {
 throw new Error(`주문을 찾을 수 없습니다: ${orderId}`);
 }

 // 2. 가격 계산
 const pricing = calculateTotal(order.items);

 // 3. 결제 처리
 await processPayment(orderId, pricing.total, order.cardNumber);

 // 4. 주문 상태 업데이트
 await updateOrderStatus(orderId, 'paid', pricing.total);

 // 5. 확인 이메일 발송
 await sendOrderConfirmation(order.email, orderId, pricing.total);

 return {
 orderId,
 ...pricing,
 status: 'completed',
 };
}
```

원래 100줄 이상이던 함수가 20줄로 줄었습니다. 각 단계가 명확하게 분리되어 읽기 쉽고, 어느 단계에서 문제가 생겨도 에러 추적이 쉽습니다.

**Claude:** 4단계가 완료되었습니다. 테스트를 실행합니다.

**Tests:** 12 passed, 12 total (기존 4 + 새로 추가된 8)

모든 테스트가 통과합니다.

## 9.6 5단계: 최종 정리

마지막으로 에러 처리를 강화합니다.

**사용자:** 에러 처리를 강화해주세요. `processOrder` 함수에서 각 단계의 에러를 구체적으로 처리하고, 결제 후 이메일 실패 시에도 주문은 성공 처리되도록 해주세요.

```
// orderProcessor.js - 에러 처리 강화 (최종)

export async function processOrder(orderId) {
 const order = await getOrder(orderId);
 if (!order) {
 throw new OrderNotFoundError(orderId);
 }

 const pricing = calculateTotal(order.items);

 // 결제 실패는 전체 프로세스를 중단
 try {
 await processPayment(orderId, pricing.total, order.cardNumber);
 } catch (error) {
 throw new PaymentError(orderId, error.message);
 }

 await updateOrderStatus(orderId, 'paid', pricing.total);

 // 이메일 실패는 로그만 남기고 주문은 성공 처리
 try {
 await sendOrderConfirmation(order.email, orderId, pricing.total);
 } catch (error) {
 console.error(`이메일 발송 실패 (주문 #${orderId}):`, error.message);
 // 이메일 재발송 큐에 추가할 수도 있습니다
 }

 return {
 orderId,
 ...pricing,
 status: 'completed',
 };
}
```

## 9.7 전후 비교: 리팩토링의 성과

리팩토링 전후를 비교하면 그 차이가 극명합니다:

| 항목         | Before                 | After             |
|------------|------------------------|-------------------|
| 파일 수       | 1개 (orderProcessor.js) | 6개 (모듈별 분리)       |
| 메인 함수 길이   | ~100줄                  | ~25줄              |
| 콜백 중첩 깊이   | 5단계                    | 0단계 (async/await) |
| 매직 넘버      | 6개                     | 0개 (명명된 상수)       |
| SQL 인젝션    | 취약                     | 안전 (매개변수화)        |
| 하드코딩된 비밀번호 | 3개                     | 0개 (환경 변수)        |
| 테스트        | 0개                     | 12개               |
| 테스트 가능성    | 불가                     | 각 모듈 독립 테스트 가능    |

최종 프로젝트 구조:

```
order-system/
├── orderProcessor.js # 오케스트레이터 (25줄)
├── services/
│ ├── database.js # DB 접근 계층
│ ├── pricing.js # 가격 계산 로직
│ ├── payment.js # 결제 처리
│ └── email.js # 이메일 발송
├── errors/
│ └── index.js # 커스텀 에러 클래스
├── __tests__/
│ ├── orderProcessor.test.js # 통합 테스트
│ └── pricing.test.js # 단위 테스트
├── .env.example # 환경 변수 템플릿
└── package.json
```

**사용자:** 리팩토링 결과를 커밋해주세요.

```
Claude: > git add -A && git commit -m "refactor: 주문 처리 모듈 리팩토링"
```

- 단일 거대 함수를 4개 서비스 모듈로 분리
- 콜백 지옥을 async/await로 전환
- SQL 인젝션 취약점 수정 (매개변수화된 쿼리)
- 하드코딩된 비밀번호를 환경 변수로 이동
- 매직 넘버를 명명된 상수로 대체
- 12개 테스트 추가 (통합 + 단위)
- 에러 처리 강화 (커스텀 에러 클래스)" 을 실행해도 될까요? [Y/n]

## 정리

이 챕터에서 우리는 전형적인 레거시 코드를 단계적으로 리팩토링하는 과정을 경험했습니다. 핵심 교훈은 다음과 같습니다:

1. **먼저 분석하세요:** Claude Code에게 코드를 분석해달라고 하면, 보안 취약점부터 코드 품질 문제까지 체계적으로 파악해줍니다. `@파일명` 으로 구체적인 파일을 지정하면 더 정확한 분석을 받을 수 있습니다.
2. **테스트부터 작성하세요:** 리팩토링 전에 반드시 기존 동작을 검증하는 테스트를 먼저 작성합니다. "이 함수의 현재 동작을 보존하는 테스트를 작성해줘"라고 요청하면 됩니다.
3. **한 번에 하나씩 바꾸세요:** Plan 모드로 단계별 계획을 세우고, 각 단계 후에 테스트를 실행합니다. 한 번에 모든 것을 바꾸면 무엇이 문제를 일으켰는지 추적하기 어렵습니다.
4. **Before/After를 확인하세요:** `/diff` 명령이나 `git diff`로 변경 전후를 비교하면, 리팩토링이 의도대로 이루어졌는지 확인할 수 있습니다.
5. **보안 취약점을 최우선으로 수정하세요:** 하드코딩된 비밀번호, SQL 인젝션 같은 보안 문제는 구조 개선보다 먼저 처리해야 합니다. Claude Code는 `/security-review` 명령으로 보안 취약점을 자동으로 탐지할 수 있습니다.
6. **커밋 메시지에 "왜"를 담으세요:** Claude Code가 자동으로 생성하는 커밋 메시지는 변경 이유와 내용을 상세히 기록합니다. 팀원들이 나중에 이력을 추적할 때 큰 도움이 됩니다.



# Chapter 10: 버그 사냥과 디버깅

## 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다:

1. 에러 메시지와 로그를 Claude Code에게 제공하여 효율적으로 버그를 추적합니다
2. 재현, 원인 분석, 수정, 검증의 체계적 디버깅 워크플로를 수행합니다
3. 파이프라인을 활용한 자동 로그 분석 기법을 익힙니다

## 시나리오: "배포 후 갑자기 500 에러가 나기 시작했어요"

금요일 오후 4시, 새 기능을 배포한 직후 Slack 알림이 쏟아지기 시작합니다. "로그인이 안 돼요", "장바구니가 비었어요", "결제 페이지에서 500 에러가 나요." 전형적인 금요일 배포 사고입니다.

이 챕터에서는 Claude Code를 활용하여 이 위기를 체계적으로 해결하는 과정을 따라갑니다.

### 10.1 에러 메시지 기반 디버깅: 첫 번째 단서 잡기

가장 먼저 해야 할 일은 에러 메시지를 분석하는 것입니다. 사용자로부터 받은 에러 스크린샷에는 다음과 같은 메시지가 있습니다:

사용자: 배포 후 이런 에러가 발생합니다. 원인을 찾아주세요.

```
Error: Cannot read properties of undefined (reading 'id')
 at getUserProfile (/app/src/controllers/userController.js:47:28)
 at processTickets (/app/src/middleware/auth.js:23:15)
 at Layer.handle [as handle_request]
 (/app/node_modules/express/lib/router/layer.js:95:5)
 at next (/app/node_modules/express/lib/router/route.js:144:13)
 at /app/src/middleware/session.js:18:5
```

이 에러 메시지를 그대로 Claude Code에 붙여넣습니다. Claude Code는 스택 트레이스를 분석하여 관련 파일을 자동으로 찾아줍니다.

**Claude:** 스택 트레이스를 분석하겠습니다.

에러 발생 위치: `src/controllers/userController.js` 47번째 줄

호출 경로: `session.js` → `auth.js` → `userController.js`

관련 파일을 확인합니다...

## 분석 결과

**`userController.js:47`을 확인했습니다:**

```
// src/controllers/userController.js - 에러 발생 지점

async function getUserProfile(req, res) {
 const user = req.session.user; // 줄 46
 const profile = await db.getProfile(user.id); // 줄 47 ← 에러 발생!
 // ...
}
```

Claude: ``req.session.user``가 `undefined`인 상태에서 ``.id``에 접근하려 해서 에러가 발생합니다.

이 문제가 배포 후에 시작되었다면, 최근 변경 사항에서 세션 관련 코드가 바뀌었을 가능성이 높습니다. 최근 커밋을 확인하겠습니다.

> `git log --oneline -10` 을 실행해도 될까요? [Y/n]

```
a1b2c3d (HEAD) feat: 세션 스토어를 Redis로 마이그레이션
e4f5g6h fix: 로깅 포맷 개선
i7j8k9l feat: 사용자 프로필 캐싱 추가
```

가장 최근 커밋 "`세션 스토어를 Redis로 마이그레이션`"이 의심됩니다.  
해당 커밋의 변경 사항을 확인하겠습니다.

> `git diff HEAD~1 -- src/middleware/session.js` 을 실행해도 될까요? [Y/n]

**핵심 습관:** 에러가 발생하면 에러 메시지를 가공하지 말고 그대로 붙여넣으세요. Claude Code는 스택 트레이스에서 파일 경로, 줄 번호, 함수 이름을 파싱하여 자동으로 관련 코드를 찾아봅니다.

---

## 10.2 git diff로 근본 원인 파악

Claude Code가 최근 커밋의 변경 사항을 분석합니다:

**Claude:** 최근 커밋에서 세션 미들웨어가 다음과 같이 변경되었습니다:

**Before** — 기존 세션 미들웨어:

```
// src/middleware/session.js - 변경 전

import session from 'express-session';
import MemoryStore from 'memorystore';

const Store = MemoryStore(session);

app.use(session({
 secret: process.env.SESSION_SECRET,
 store: new Store({ checkPeriod: 86400000 }),
 resave: false,
 saveUninitialized: false,
 cookie: { secure: false }
}));
```

**After** — Redis로 변경된 코드:

```
// src/middleware/session.js - 변경 후

import session from 'express-session';
import RedisStore from 'connect-redis';
import { createClient } from 'redis';

const redisClient = createClient({ url: process.env.REDIS_URL });
redisClient.connect();

app.use(session({
 secret: process.env.SESSION_SECRET,
 store: new RedisStore({ client: redisClient }),
 resave: false,
 saveUninitialized: false,
 cookie: { secure: true } // ← 여기가 문제!
}));
```

Claude: ## 근본 원인 발견

두 가지 문제가 있습니다:

### ### 문제 1 (주요 원인): cookie.secure가 true로 변경됨

- 변경 전: `secure: false`
- 변경 후: `secure: true`
- 영향: `secure: true`이면 HTTPS에서만 쿠키가 전송됩니다
- 개발/스테이징 환경이 HTTP를 사용하고 있다면 세션 쿠키가 전혀 전송되지 않아 `req.session.user`가 항상 undefined가 됩니다

### ### 문제 2 (잠재적 원인): Redis 연결 에러 핸들링 부재

- `redisClient.connect()`의 에러를 처리하지 않아, Redis 서버에 연결할 수 없는 경우 프로세스가 크래시할 수 있습니다

수정하시겠습니까?

정확한 원인을 찾았습니다. 이제 수정합니다.

**사용자:** 수정해주세요. 환경에 따라 secure 옵션이 다르게 적용되도록 하고, Redis 연결 에러 핸들링도 추가해주세요.

**Before** — 문제의 코드:

```
const redisClient = createClient({ url: process.env.REDIS_URL });
redisClient.connect();

app.use(session({
 // ...
 cookie: { secure: true }
}));
```

**After** — 수정된 코드:

```
const redisClient = createClient({ url: process.env.REDIS_URL });

redisClient.on('error', (err) => {
 console.error('Redis 연결 에러:', err.message);
});

redisClient.on('connect', () => {
 console.log('Redis에 연결되었습니다. ');
});

await redisClient.connect();

const isProduction = process.env.NODE_ENV === 'production';

app.use(session({
 secret: process.env.SESSION_SECRET,
 store: new RedisStore({ client: redisClient }),
 resave: false,
 saveUninitialized: false,
 cookie: {
 secure: isProduction, // 프로덕션에서만 HTTPS 필수
 httpOnly: true, // XSS 방지
 maxAge: 24 * 60 * 60 * 1000, // 24시간
 sameSite: isProduction ? 'strict' : 'lax',
 }
}));
```

## 10.3 로그 파이프라인 분석: 실시간 감시

에러를 수정한 후에도 다른 문제가 숨어 있을 수 있습니다. Claude Code의 파이프라인 기능을 활용하여 로그를 분석합니다.

```
방법 1: 로그 파일을 직접 분석
$ cat /var/log/app/error.log | claude -p "이 로그에서 반복되는 에러 패턴을
 찾아 정리하고, 각 에러의 원인과 해결 방안을 제안해주세요"
```

Claude Code의 분석 결과:

Claude: 로그 분석 결과를 보고합니다.

## ## 에러 패턴 요약

### ### 패턴 1: 세션 관련 에러 (157건, 15:32~16:45)

- "Cannot read properties of undefined (reading 'id')"
- 원인: cookie.secure 설정 문제 (이미 수정됨)
- 상태:  수정 완료

### ### 패턴 2: Redis 타임아웃 (23건, 간헐적)

- "Redis connection timed out after 5000ms"
- 원인: Redis 서버의 maxmemory 설정이 낮아 간헐적 응답 지연
- 제안: Redis 메모리 제한을 늘리거나 maxmemory-policy를 설정

### ### 패턴 3: 데이터베이스 커넥션 풀 고갈 (8건, 16:20~16:25)

- "Error: too many connections"
- 원인: 세션 에러로 인한 반복 재시도가 DB 커넥션을 소진
- 제안: 커넥션 풀 최대치 조정 및 재시도 백오프 로직 추가

### ### 패턴 4: Deprecated API 경고 (345건)

- "DeprecationWarning: collection.ensureIndex is deprecated"
- 원인: MongoDB 드라이버 버전 업그레이드 필요
- 심각도: 낮음 (현재 기능에 영향 없음, 다음 스프린트에서 처리 권장)

실시간 로그 모니터링도 가능합니다:

```
방법 2: 실시간 로그 모니터링
$ tail -f /var/log/app/error.log | claude -p "이상 징후가 감지되면
 에러 유형, 발생 빈도, 추정 원인을 알려주세요"
```

## 10.4 재현 테스트 작성: 버그를 가두다

버그를 수정하기 전에, 먼저 버그를 재현하는 테스트를 작성합니다. 이 테스트가 있으면 수정 후에도 같은 버그가 다시 발생하지 않도록 보장할 수 있습니다.

사용자: 이 세션 버그를 재현하는 테스트를 작성해주세요.

HTTP 환경에서 `secure: true` 쿠키가 전송되지 않는 상황을 테스트해야 합니다.

```
// __tests__/session.test.js - 버그 재현 테스트

import request from 'supertest';
import { createApp } from '../src/app.js';

describe('세션 관리', () => {
 describe('HTTP 환경 (개발/스테이징)', () => {
 let app;

 beforeEach(() => {
 process.env.NODE_ENV = 'development';
 app = createApp();
 });

 it('HTTP에서 세션 쿠키가 정상적으로 설정됩니다', async () => {
 // 로그인
 const loginRes = await request(app)
 .post('/api/auth/login')
 .send({ email: 'test@example.com', password: 'password' });

 expect(loginRes.status).toBe(200);

 // 쿠키 추출
 const cookies = loginRes.headers['set-cookie'];
 expect(cookies).toBeDefined();

 // 세션 쿠키에 Secure 플래그가 없어야 함 (HTTP 환경)
 const sessionCookie = cookies.find(c => c.startsWith('connect.sid'));
 expect(sessionCookie).not.toContain('Secure');
 });

 it('로그인 후 보호된 엔드포인트에 접근할 수 있습니다', async () => {
 const agent = request.agent(app);

 // 로그인
 await agent
 .post('/api/auth/login')
 .send({ email: 'test@example.com', password: 'password' });

 // 프로필 조회 (세션 필요)
 const profileRes = await agent.get('/api/users/me');
 expect(profileRes.status).toBe(200);
 expect(profileRes.body.data.email).toBe('test@example.com');
 });

 it('세션 없이 보호된 엔드포인트에 접근하면 401을 반환합니다', async () => {
 const res = await request(app).get('/api/users/me');
 });
 });
});
```



```

 expect(res.status).toBe(401);
 });
});

describe('HTTPS 환경 (프로덕션)', () => {
 let app;

 beforeEach(() => {
 process.env.NODE_ENV = 'production';
 app = createApp();
 });

 it('프로덕션에서 세션 쿠키에 Secure 플래그가 설정됩니다', async () => {
 const loginRes = await request(app)
 .post('/api/auth/login')
 .send({ email: 'test@example.com', password: 'password' });

 const cookies = loginRes.headers['set-cookie'];
 const sessionCookie = cookies.find(c => c.startsWith('connect.sid'));
 expect(sessionCookie).toContain('Secure');
 });
});
});

```

Claude: 테스트를 실행합니다.

> npx jest \_\_tests\_\_/session.test.js --verbose 를 실행해도 될까요? [Y/n]

PASS \_\_tests\_\_/session.test.js

세션 관리

HTTP 환경 (개발/스테이징)

- ✓ HTTP에서 세션 쿠키가 정상적으로 설정됩니다 (45 ms)
- ✓ 로그인 후 보호된 엔드포인트에 접근할 수 있습니다 (32 ms)
- ✓ 세션 없이 보호된 엔드포인트에 접근하면 401을 반환합니다 (15 ms)

HTTPS 환경 (프로덕션)

- ✓ 프로덕션에서 세션 쿠키에 Secure 플래그가 설정됩니다 (28 ms)

Tests: 4 passed, 4 total

모든 테스트가 통과합니다. 이 테스트들이 향후 같은 유형의 버그를 방지하는 안전망 역할을 합니다.

## 10.5 성능 디버깅: 느린 API 응답 추적

에러 해결 후, 또 다른 문제가 보고됩니다. 특정 API 엔드포인트가 비정상적으로 느립니다.

사용자: **GET /api/products** 엔드포인트가 평균 5초 이상 걸립니다.  
원인을 찾아주세요. @src/controllers/productController.js

**Claude:** productController.js를 분석합니다.

문제를 발견했습니다:

```
// src/controllers/productController.js - 성능 문제가 있는 코드

async function getProducts(req, res) {
 const products = await db.query('SELECT * FROM products');

 // N+1 쿼리 문제!
 for (const product of products) {
 product.category = await db.query(
 'SELECT name FROM categories WHERE id = ?', [product.category_id]
);
 product.reviews = await db.query(
 'SELECT * FROM reviews WHERE product_id = ?', [product.id]
);
 product.images = await db.query(
 'SELECT url FROM images WHERE product_id = ?', [product.id]
);
 }

 res.json({ data: products });
}
```

Claude: ## 성능 문제 분석

### ### 근본 원인: N+1 쿼리 문제

- 상품 100개 조회 시: 1(상품) + 100(카테고리) + 100(리뷰) + 100(이미지) = **\*\*301번의 DB 쿼리\*\***
- 각 쿼리가 평균 15ms라면:  $301 \times 15\text{ms} = \textbf{**약 4.5초**}$

### ### 해결 방법

1. JOIN을 사용한 단일 쿼리
2. 또는 카테고리/리뷰/이미지를 각각 한 번의 IN 쿼리로 조회

수정하시겠습니까?

사용자: JOIN 쿼리로 수정해주세요.

**Before** — N+1 쿼리 (301번의 DB 호출):

```
const products = await db.query('SELECT * FROM products');
for (const product of products) {
 product.category = await db.query('SELECT name FROM categories WHERE id = ?',
 [product.category_id]);
 product.reviews = await db.query('SELECT * FROM reviews WHERE product_id = ?',
 [product.id]);
 product.images = await db.query('SELECT url FROM images WHERE product_id = ?',
 [product.id]);
}
```

**After** — 최적화된 쿼리 (3번의 DB 호출):

```

async function getProducts(req, res) {
 // 1. 상품 + 카테고리 조인 쿼리 (1번)
 const products = await db.query(`
 SELECT p.*, c.name as category_name
 FROM products p
 LEFT JOIN categories c ON p.category_id = c.id
 ORDER BY p.created_at DESC
 `);

 if (products.length === 0) {
 return res.json({ data: [] });
 }

 const productIds = products.map(p => p.id);

 // 2. 리뷰 일괄 조회 (1번)
 const reviews = await db.query(
 `SELECT * FROM reviews WHERE product_id IN (${productIds.map(() =>
 '?'`).join(',')})`,
 productIds
);

 // 3. 이미지 일괄 조회 (1번)
 const images = await db.query(
 `SELECT product_id, url FROM images WHERE product_id IN (${productIds.map(() =>
 '?'`).join(',')})`,
 productIds
);

 // 결과 조합
 const reviewsByProduct = groupBy(reviews, 'product_id');
 const imagesByProduct = groupBy(images, 'product_id');

 const result = products.map(product => ({
 ...product,
 reviews: reviewsByProduct[product.id] || [],
 images: imagesByProduct[product.id] || [],
 }));

 res.json({ data: result });
}

function groupBy(array, key) {
 return array.reduce((groups, item) => {
 const value = item[key];
 groups[value] = groups[value] || [];
 groups[value].push(item);
 }, {});
}

```

```
 return groups;
 }, {}));
}
```

Claude: 성능 비교:

- Before: 301 쿼리, ~4.5초
- After: 3 쿼리, ~50ms

약 90배 성능 향상입니다.

## 10.6 디버깅 워크플로 정리

이 챕터에서 사용한 디버깅 워크플로를 체계적으로 정리합니다:

디버깅 워크플로 (Claude Code 활용)

1. 에러 수집
  - └─ 에러 메시지 & 스택 트레이스를 그대로 붙여넣기
2. 원인 분석
  - └─ Claude Code가 스택 트레이스에서 파일/줄 번호 추적
  - └─ git log/diff로 최근 변경 사항 확인
  - └─ 관련 파일 @멘션으로 컨텍스트 제공
3. 재현 테스트 작성
  - └─ "이 버그를 재현하는 테스트를 작성해줘"
4. 수정
  - └─ Claude Code가 수정 코드 제안 → 검토 후 승인
5. 검증
  - └─ 재현 테스트 통과 확인
  - └─ 기존 테스트 전체 실행
  - └─ 로그 모니터링으로 재발 여부 확인
6. 방지
  - └─ 회귀 테스트 추가
  - └─ CLAUDE.md에 디버깅 인사이트 기록

## 10.7 Claude Code와 모니터링 도구 연동

MCP를 활용하면 Claude Code를 Sentry 같은 모니터링 도구와 직접 연동할 수 있습니다.

```
Sentry MCP 서버 연결
$ claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

연동 후에는 자연어로 에러를 추적할 수 있습니다:

사용자: 지난 24시간 동안 가장 많이 발생한 에러 5개를 보여주세요.

Claude: Sentry에서 지난 24시간간의 에러를 조회합니다.

1. `TypeError: Cannot read properties of undefined (reading 'id')` – 157건  
→ `src/controllers/userController.js:47` (이미 수정됨)
2. `RedisTimeoutError: Connection timed out` – 23건  
→ `src/middleware/session.js` (Redis 설정 조정 필요)
3. `MongoNetworkError: connection pool exhausted` – 8건  
→ `src/services/database.js` (커넥션 풀 설정 조정 필요)
4. `ValidationError: Invalid email format` – 5건  
→ `src/controllers/authController.js` (입력 검증 강화 필요)
5. `SyntaxError: Unexpected token in JSON` – 3건  
→ `src/middleware/bodyParser.js` (잘못된 JSON 요청 처리 필요)

사용자: 2번 Redis 타임아웃의 스택 트레이스를 보여주세요.

Claude: Sentry에서 해당 에러의 상세 정보를 조회합니다...

이처럼 MCP 연동을 통해 대시보드를 열지 않고도 터미널에서 바로 에러를 추적하고 코드를 수정할 수 있습니다.

## 10.8 디버깅 치트시트: 상황별 프롬프트 모음

자주 발생하는 디버깅 상황별로 효과적인 프롬프트를 정리합니다:

| 상황         | 프롬프트 예시                                                 |
|------------|---------------------------------------------------------|
| 에러 메시지 분석  | 이 에러의 원인을 찾아주세요: [에러 메시지 붙여넣기]                          |
| 스택 트레이스 추적 | 이 스택 트레이스에서 근본 원인이 되는 코드를 찾아주세요                         |
| 최근 변경 의심   | git diff HEAD~3으로 최근 변경 중 이 에러와 관련된 부분을 찾아주세요           |
| 로그 분석      | cat error.log   claude -p "반복되는 에러 패턴을 찾아주세요"           |
| 성능 문제      | @파일명 이 함수가 느린 원인을 분석하고 최적화해주세요                          |
| 재현 테스트     | 이 버그를 재현하는 테스트를 작성하고, 수정 후 통과하는지 확인해주세요                 |
| 환경 문제      | 이 코드가 로컬에서는 되는데 프로덕션에서 안 됩니다. 환경 차이 때문일 수 있는 원인을 분석해주세요 |
| 의존성 충돌     | package.json의 의존성 중 호환되지 않는 조합이 있는지 확인해주세요              |

## 10.9 실시간 모니터링 파이프라인 구축

일회성 로그 분석 외에도, Claude Code를 파이프라인에 통합하여 지속적인 모니터링 체계를 구축할 수 있습니다.

```
이상 패턴 감지 후 Slack 알림
$ tail -f /var/log/app/error.log | claude -p \
 "에러 빈도가 분당 10건을 초과하면 Slack me with the error pattern summary"

보안 위협 모니터링
$ tail -f /var/log/nginx/access.log | claude -p \
 "SQL 인젝션, XSS, 무차별 대입 시도 등 보안 위협 패턴을 감지하면 알려주세요"

배포 후 헬스체크
$ claude -p --max-turns 5 \
 "curl로 /api/health, /api/products, /api/auth/status를 차례로 호출하고
 모든 엔드포인트가 정상 응답하는지 확인해주세요"
```

이렇게 CLI 파이프라인을 활용하면 별도의 모니터링 도구를 설치하지 않고도 실시간 로그 분석이 가능합니다.

## 정리

이 챕터에서 우리는 배포 후 발생한 긴급 에러를 체계적으로 추적하고 해결하는 과정을 경험했습니다. 핵심 교훈은 다음과 같습니다:

1. **에러 메시지를 그대로 붙여넣으세요:** 가공하거나 요약하지 말고, 스택 트레이스 전체를 Claude Code에 제공합니다. Claude Code는 파일 경로와 줄 번호를 자동으로 파싱하여 관련 코드를 찾아봅니다.
2. **git 히스토리를 활용하세요:** "배포 후 발생한 에러"라면 `git log` 와 `git diff` 로 최근 변경 사항을 확인하는 것이 가장 빠른 근본 원인 추적 방법입니다. Claude Code는 이 과정을 자동으로 수행할 수 있습니다.
3. **재현 테스트를 먼저 작성하세요:** 버그를 수정하기 전에 해당 버그를 재현하는 테스트를 작성합니다. 이 테스트가 "빨간색에서 초록색으로" 바뀌면 수정이 성공한 것이고, 향후 같은 버그가 다시 발생하는 것을 방지합니다.
4. **파이프라인을 적극 활용하세요:** `cat logs.txt | claude -p "분석해줘"` 패턴은 로그 분석에서 강력한 도구입니다. 대량의 로그에서 패턴을 찾고, 빈도를 분석하고, 우선순위를 매기는 작업을 자동화할 수 있습니다.
5. **MCP로 모니터링 도구를 연동하세요:** Sentry, Datadog 같은 모니터링 도구를 MCP로 연결하면 터미널을 떠나지 않고도 에러를 추적하고 코드를 수정할 수 있습니다. 컨텍스트 전환 비용이 크게 줄어듭니다.
6. **디버깅 인사이트를 기록하세요:** 해결한 버그의 원인과 해결 방법을 CLAUDE.md나 Auto Memory에 기록해두면, 비슷한 문제가 다시 발생했을 때 Claude Code가 이전 경험을 참고하여 더 빠르게 해결할 수 있습니다.

**Part 3을 마치며:** 이 세 개의 챕터를 통해 우리는 Claude Code를 실제 개발 시나리오에 적용하는 방법을 배웠습니다. 빈 디렉터리에서 웹 앱을 만들고(Chapter 8), 레거시 코드를 체계적으로 개선하고(Chapter 9), 긴급한 버그를 추적하여 해결했습니다(Chapter 10). 이 경험들은 독립적인 기술이 아니라 서로 연결됩니다. 새 프로젝트를 만들 때는 처음부터 좋은 구조를 잡고, 레거시 코드를 만나면 단계적으로 개선하며, 문제가 발생하면 체계적으로 디버깅합니다. Claude Code는 이 모든 과정에서 여러분의 페어 프로그래밍 파트너가 됩니다.



# Chapter 11: 설정 심화

## 학습 목표

- 1. 4단계 설정 체계(Managed, User, Project, Local)의 우선순위를 이해하고 활용합니다
- 2. 권한 모드와 권한 규칙을 세밀하게 설정하여 안전한 사용 환경을 구축합니다
- 3. 샌드박스 모드를 설정하여 파일시스템과 네트워크를 격리합니다
- 4. 팀과 개인의 설정을 분리하여 효율적으로 관리합니다

## 11.1 설정 파일 체계 이해하기

Claude Code는 4단계 설정 체계를 제공합니다. 각 설정 파일은 서로 다른 범위(scope)에 적용되며, 명확한 우선순위에 따라 병합됩니다. 이 구조를 이해하면 개인 취향과 팀 표준을 충돌 없이 공존시킬 수 있습니다.

### 설정 범위와 파일 위치

| 범위             | 파일 위치                                     | 영향 범위         | 팀 공유 여부         |
|----------------|-------------------------------------------|---------------|-----------------|
| Managed (관리형)  | 시스템 레벨 <code>managed-settings.json</code> | 머신의 모든 사용자    | IT 관리자가 배포      |
| User (사용자)     | <code>~/.claude/settings.json</code>      | 내 모든 프로젝트     | 아니오 (개인 설정)     |
| Project (프로젝트) | <code>.claude/settings.json</code>        | 이 저장소의 모든 협업자 | 예 (git 커밋)      |
| Local (로컬)     | <code>.claude/settings.local.json</code>  | 이 저장소에서 나만    | 아니오 (gitignore) |

### 우선순위 (높은 순에서 낮은 순)

설정이 겹칠 때 다음 순서로 우선 적용됩니다:

- 1. **Managed** (최고 우선순위) — 어떤 설정으로도 재정의(override)할 수 없습니다. 조직의 보안 정책 등 반드시 지켜야 할 규칙에 사용됩니다.
- 2. **커맨드라인 인수** — `claude --model opus` 처럼 실행 시 지정한 값이 파일 설정보다 우선합니다.
- 3. **Local** — 프로젝트 내에서 나만 적용하는 설정입니다. Project 설정과 User 설정을 모두 재정의합니다.
- 4. **Project** — 팀 전체에 적용되는 프로젝트 공유 설정입니다. User 설정을 재정의합니다.

5. **User** (최저 우선순위) — 다른 설정이 없을 때 기본적으로 적용됩니다.

이 우선순위를 활용하면 다음과 같은 시나리오를 자연스럽게 처리할 수 있습니다:

- **팀 표준 (Project)**: `.claude/settings.json`에 린트 명령어 허용, `.env` 읽기 차단 규칙을 정의합니다.
- **개인 취향 (User)**: `~/.claude/settings.json`에 선호하는 모델과 출력 스타일을 설정합니다.
- **프로젝트별 예외 (Local)**: `.claude/settings.local.json`에 팀 설정을 일부 재정의합니다.
- **조직 보안 정책 (Managed)**: 관리형 설정으로 특정 도구 사용을 전면 차단합니다.

## 11.2 settings.json 구조 상세

settings.json 파일은 JSON 형식으로 작성하며, `$schema` 키를 통해 자동완성과 유효성 검사를 활용할 수 있습니다. 아래는 주요 설정 키를 기능별로 정리한 것입니다.

### 기본 구조

```
{
 "$schema": "https://json.schemastore.org/claude-code-settings.json",
 "model": "sonnet",
 "effortLevel": "medium",
 "language": "korean",
 "outputStyle": "Explanatory",
 "includeCoAuthoredBy": true,
 "showTurnDuration": true,
 "alwaysThinkingEnabled": false,
 "autoMemoryEnabled": true,
 "cleanupPeriodDays": 30,
 "autoUpdatesChannel": "stable"
}
```

## 주요 설정 키 상세

### 모델 및 성능 관련

| 키                                  | 타입       | 기본값                    | 설명                                                                                                           |
|------------------------------------|----------|------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>model</code>                 | string   | <code>"default"</code> | 기본 모델을 지정합니다. <code>"sonnet"</code> , <code>"opus"</code> , <code>"haiku"</code> , <code>"opusplan"</code> 등 |
| <code>effortLevel</code>           | string   | <code>"medium"</code>  | 적응형 추론의 노력 수준입니다. <code>"low"</code> , <code>"medium"</code> , <code>"high"</code>                           |
| <code>alwaysThinkingEnabled</code> | boolean  | <code>false</code>     | 확장 사고(extended thinking)를 기본 활성화합니다                                                                          |
| <code>availableModels</code>       | string[] | 전체                     | 사용 가능한 모델을 제한합니다                                                                                             |

### 출력 관련

| 키                                | 타입      | 기본값                        | 설명                            |
|----------------------------------|---------|----------------------------|-------------------------------|
| <code>language</code>            | string  | 시스템 언어                     | Claude의 응답 언어를 지정합니다          |
| <code>outputStyle</code>         | string  | <code>"Explanatory"</code> | 출력 스타일을 설정합니다                 |
| <code>showTurnDuration</code>    | boolean | <code>false</code>         | 각 턴의 소요 시간을 표시합니다             |
| <code>includeCoAuthoredBy</code> | boolean | <code>true</code>          | git 커밋에 Co-Authored-By를 포함합니다 |

### 세션 및 메모리 관련

| 키                               | 타입      | 기본값                   | 설명                                                            |
|---------------------------------|---------|-----------------------|---------------------------------------------------------------|
| <code>autoMemoryEnabled</code>  | boolean | <code>false</code>    | Auto Memory 기능을 활성화합니다                                        |
| <code>cleanupPeriodDays</code>  | number  | <code>30</code>       | 세션 자동 정리 기간(일)입니다                                             |
| <code>autoUpdatesChannel</code> | string  | <code>"stable"</code> | 자동 업데이트 채널입니다. <code>"stable"</code> 또는 <code>"latest"</code> |

## 환경 변수 주입

`env` 키를 사용하면 Claude Code 세션에 환경 변수를 주입할 수 있습니다. 이는 텔레메트리 설정이나 외부 서비스 연결에 유용합니다.

```
{
 "env": {
 "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
 "OTEL_METRICS_EXPORTER": "otlp",
 "NODE_ENV": "development"
 }
}
```

## 11.3 권한 모드 (Permission Mode)

권한 모드는 Claude Code가 파일을 수정하거나 명령을 실행할 때 사용자의 승인을 어느 수준까지 요구할지 결정합니다. 작업 성격과 신뢰 수준에 따라 적절한 모드를 선택해야 합니다.

### 권한 모드 종류

| 모드                | CLI 플래그                                          | 동작                         | 적합한 상황              |
|-------------------|--------------------------------------------------|----------------------------|---------------------|
| default           | <code>--permission-mode default</code>           | 매 작업마다 개별 승인 요청            | 처음 사용할 때, 민감한 작업    |
| plan              | <code>--permission-mode plan</code>              | 계획을 먼저 설명하고 승인 후 실행        | 대규모 변경, 코드 리뷰       |
| acceptEdits       | <code>--permission-mode acceptEdits</code>       | 파일 편집은 자동 승인, 명령 실행은 승인 요청 | 코드 작성에 집중할 때        |
| dontAsk           | <code>--permission-mode dontAsk</code>           | 대부분의 작업 자동 승인              | 신뢰할 수 있는 반복 작업      |
| bypassPermissions | <code>--permission-mode bypassPermissions</code> | 모든 권한 프롬프트 건너뛰기            | CI/CD 파이프라인 (주의 필요) |

**주의:** `bypassPermissions` 모드는 모든 보안 검증을 건너뛰므로, 반드시 자동화 환경에서만 사용해야 합니다. 대화형 세션에서는 사용하지 않는 것을 강력히 권장합니다.

## 권한 모드 설정 방법

```
CLI 플래그로 지정
claude --permission-mode plan

settings.json에서 기본값 설정
(VS Code 확장에서도 initialPermissionMode로 설정 가능)
```

## 11.4 권한 규칙 심화

권한 규칙(permission rules)은 특정 도구(tool)나 명령에 대해 허용(allow), 질문(ask), 거부(deny) 동작을 세밀하게 정의합니다.

### 권한 규칙 구조

```
{
 "permissions": {
 "allow": [
 "Bash(npm run lint)",
 "Bash(npm run test *)",
 "Bash(git log *)",
 "Bash(git diff *)",
 "Read(~/.zshrc)"
],
 "ask": [
 "Bash(git push *)",
 "Bash(git checkout *)"
],
 "deny": [
 "WebFetch",
 "Bash(curl *)",
 "Bash(rm -rf *)",
 "Read(./env)",
 "Read(./env.*)",
 "Read(./secrets/**)"
]
 }
}
```

## 규칙 평가 순서

권한 규칙은 다음 순서로 평가됩니다:

1. **deny** 규칙을 먼저 확인합니다. 매칭되면 즉시 차단합니다.
2. **ask** 규칙을 확인합니다. 매칭되면 사용자에게 승인을 요청합니다.
3. **allow** 규칙을 확인합니다. 매칭되면 자동 승인합니다.
4. 어떤 규칙에도 매칭되지 않으면 현재 권한 모드의 기본 동작을 따릅니다.

## 패턴 문법

| 패턴                                        | 설명          | 예시                                                       |
|-------------------------------------------|-------------|----------------------------------------------------------|
| <code>Bash</code>                         | 모든 Bash 명령어 | 모든 셸 명령 매칭                                               |
| <code>Bash(npm run *)</code>              | 특정 접두사의 명령어 | <code>npm run build</code> , <code>npm run test</code> 등 |
| <code>Read(./env)</code>                  | 특정 파일 읽기    | <code>.env</code> 파일 정확히 매칭                              |
| <code>Read(./env.*)</code>                | 패턴 매칭 읽기    | <code>.env.local</code> , <code>.env.production</code> 등 |
| <code>Read(/secrets/**)</code>            | 재귀 패턴       | <code>secrets/</code> 하위 모든 파일                           |
| <code>Edit(/**/*.ts)</code>               | 특정 확장자 편집   | 모든 TypeScript 파일 수정                                      |
| <code>WebFetch</code>                     | 웹 요청 전체     | 모든 HTTP 요청                                               |
| <code>WebFetch(domain:example.com)</code> | 특정 도메인 요청   | example.com으로의 요청만                                       |

## 실전 권한 설정 예시

### 프론트엔드 프로젝트

```
{
 "permissions": {
 "allow": [
 "Bash(npm run *)",
 "Bash(npx prettier *)",
 "Bash(npx eslint *)",
 "Bash(git status)",
 "Bash(git log *)",
 "Bash(git diff *)"
],
 "ask": [
 "Bash(git commit *)",
 "Bash(git push *)",
 "Bash(npm install *)"
],
 "deny": [
 "Bash(rm -rf *)",
 "Bash(curl *)",
 "Read(../.env)",
 "Read(../.env.*)",
 "Read(../secrets/**)"
]
 }
}
```

```
{
 "permissions": {
 "allow": [
 "Bash(python *)",
 "Bash(pip install *)",
 "Bash(jupyter *)",
 "Read"
],
 "deny": [
 "Bash(rm -rf *)",
 "Read(./credentials/**)",
 "Read(./env)",
 "WebFetch"
]
 }
}
```

## 11.5 샌드박스 설정

샌드박스(Sandbox)는 Claude Code의 파일시스템 접근과 네트워크 요청을 격리하는 보안 계층입니다. 민감한 프로젝트나 신뢰하기 어려운 작업에서 추가적인 안전장치를 제공합니다.



## 샌드박스 설정 구조

```
{
 "sandbox": {
 "enabled": true,
 "autoAllowBashIfSandboxed": true,
 "excludedCommands": ["docker", "kubectl"],
 "filesystem": {
 "allowWrite": [
 "//tmp/build",
 "~/ .kube"
],
 "denyRead": [
 "~/ .aws/credentials",
 "~/ .ssh/id_rsa"
]
 },
 "network": {
 "allowedDomains": [
 "github.com",
 "*.npmjs.org",
 "registry.npmjs.org"
],
 "allowLocalBinding": true
 }
 }
}
```

## 각 설정 키 설명

| 키                                      | 설명                         |
|----------------------------------------|----------------------------|
| <code>enabled</code>                   | 샌드박스 활성화 여부                |
| <code>autoAllowBashIfSandboxed</code>  | 샌드박스 내에서 Bash 명령을 자동 허용합니다 |
| <code>excludedCommands</code>          | 샌드박스에서 제외할 명령어(직접 실행 차단)   |
| <code>filesystem.allowWrite</code>     | 쓰기를 허용할 경로 목록              |
| <code>filesystem.denyRead</code>       | 읽기를 차단할 경로 목록              |
| <code>network.allowedDomains</code>    | 네트워크 접근을 허용할 도메인 목록        |
| <code>network.allowLocalBinding</code> | 로컬 포트 바인딩 허용 여부            |

샌드박스를 활성화하면 `autoAllowBashIfSandboxed` 옵션을 통해 격리된 환경 안에서는 Bash 명령을 매번 승인하지 않아도 됩니다. 이는 보안과 편의성의 균형을 맞추는 좋은 방법입니다.

## 11.6 팀 설정과 개인 설정 분리

효과적인 팀 운영을 위해서는 공유 설정과 개인 설정을 명확히 분리해야 합니다.

### 권장 파일 구조

```

your-project/
├── .claude/
│ ├── settings.json # 팀 공유 설정 (git 커밋)
│ ├── settings.local.json # 개인 로컬 설정 (gitignore)
│ ├── CLAUDE.md # 프로젝트 지시사항 (git 커밋)
│ └── rules/
│ ├── code-style.md # 코딩 스타일 규칙
│ ├── testing.md # 테스트 규칙
│ └── security.md # 보안 규칙
├── .mcp.json # 팀 공유 MCP 설정 (git 커밋)
└── .gitignore # settings.local.json 포함

```

## .gitignore 설정

```
Claude Code 개인 설정
.claude/settings.local.json
```

## 팀 공유 설정 ( `.claude/settings.json` )

```
{
 "$schema": "https://json.schemastore.org/claude-code-settings.json",
 "permissions": {
 "allow": [
 "Bash(npm run *)",
 "Bash(git log *)",
 "Bash(git diff *)"
],
 "deny": [
 "Read(./env)",
 "Read(./env.*)",
 "Read(./secrets/**)",
 "Bash(rm -rf *)"
]
 },
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
 }
]
 }
]
 }
}
```

## 개인 로컬 설정 ( `.claude/settings.local.json` )

```
{
 "model": "opus",
 "effortLevel": "high",
 "language": "korean",
 "showTurnDuration": true,
 "alwaysThinkingEnabled": true
}
```

## 11.7 조직 전체 관리형 설정 (Managed Settings)

대규모 조직에서는 Managed 설정을 통해 모든 개발자에게 일관된 보안 정책을 적용할 수 있습니다.

### Managed 설정 파일 위치

- macOS: `/Library/Application Support/ClaudeCode/managed-settings.json`
- Linux: `/etc/claude-code/managed-settings.json`

## Managed 설정 예시

```
{
 "permissions": {
 "deny": [
 "WebFetch",
 "Bash(curl *)",
 "Bash(wget *)",
 "Read(/etc/passwd)",
 "Read(~/.ssh/**)"
]
 },
 "sandbox": {
 "enabled": true,
 "network": {
 "allowedDomains": [
 "github.company.com",
 "*.internal.company.com",
 "registry.npmjs.org"
]
 }
 },
 "availableModels": ["sonnet", "opus"]
}
```

Managed 설정은 최고 우선순위를 가지므로, 개별 사용자나 프로젝트에서 이 설정을 재정의할 수 없습니다. 이는 조직의 보안 컴플라이언스를 보장하는 강력한 메커니즘입니다.

## | 11.8 설정 디버깅

설정이 기대와 다르게 동작할 때 다음 도구들로 현재 적용된 설정을 확인할 수 있습니다.

```
설정 인터페이스 열기
/config

설치 및 설정 전반 진단
/doctor

현재 권한 규칙 확인
/permissions

디버그 모드로 실행하여 상세 로그 확인
claude --debug "api,mcp"

현재 상태 종합 확인
/status
```

**/doctor** 명령은 설치 상태, 인증, 설정 파일 로드 여부, MCP 서버 연결 등을 종합적으로 진단하여 문제의 원인을 빠르게 파악하는 데 도움을 줍니다.

## | 정리

이 챕터에서는 Claude Code의 설정 체계를 심층적으로 살펴보았습니다.

- **4단계 설정 체계**(Managed, User, Project, Local)는 명확한 우선순위에 따라 병합되며, 조직 정책부터 개인 취향까지 계층적으로 관리할 수 있습니다.
- **settings.json**의 주요 키들을 이해하면 모델 선택, 출력 스타일, 메모리 관리 등을 세밀하게 제어할 수 있습니다.
- **권한 모드**는 작업의 성격에 따라 5단계로 설정할 수 있으며, **권한 규칙**은 deny > ask > allow 순서로 평가됩니다.
- **샌드박스**는 파일시스템과 네트워크를 격리하여 추가적인 보안 계층을 제공합니다.
- **팀 설정과 개인 설정을 분리**하여 git으로 공유할 내용과 개인만 사용할 내용을 명확히 구분해야 합니다.
- **Managed 설정**은 조직 전체에 재정의 불가능한 정책을 배포하는 데 사용됩니다.

# Chapter 12: MCP 서버로 능력 확장

## 학습 목표

1. MCP(Model Context Protocol)의 개념과 아키텍처를 이해합니다
2. 외부 서비스(GitHub, Sentry, 데이터베이스 등)를 MCP로 연결하고 활용합니다
3. 팀과 공유 가능한 MCP 설정을 구성합니다
4. Tool Search를 통한 컨텍스트 최적화 전략을 익힙니다

## 12.1 MCP란 무엇인가

### Model Context Protocol의 개념

MCP(Model Context Protocol)는 AI 도구와 외부 데이터 소스를 연결하기 위한 **오픈 소스 표준**입니다. MCP 서버를 통해 Claude Code는 기본적으로 갖추지 못한 능력 — 이슈 트래커 접근, 데이터베이스 쿼리, 모니터링 시스템 조회 등 — 을 확장할 수 있습니다.

MCP를 이해하기 위해 간단한 비유를 들어보겠습니다. Claude Code가 숙련된 개발자라면, MCP 서버는 그 개발자에게 주어지는 전문 도구입니다. 망치(Git 연동)와 드라이버(파일 편집)는 기본 탑재되어 있지만, 오실로스코프(Sentry 에러 모니터링)나 용접기(PostgreSQL 쿼리)는 MCP를 통해 추가합니다.

### MCP로 할 수 있는 것들

MCP 서버를 연결하면 자연어로 다음과 같은 작업이 가능해집니다:

- **이슈 트래커 연동:** "JIRA 이슈 ENG-4521에 설명된 기능을 구현하고 GitHub에 PR을 생성해줘"
- **모니터링 데이터 분석:** "Sentry에서 최근 24시간 동안 가장 많이 발생한 에러를 보여줘"
- **데이터베이스 쿼리:** "PostgreSQL에서 이번 달 총 매출을 조회해줘"
- **디자인 통합:** "Figma에서 새 디자인을 가져와 컴포넌트를 업데이트해줘"
- **커뮤니케이션:** "Slack에 배포 완료 메시지를 보내줘"

### MCP 아키텍처

MCP는 클라이언트-서버 구조로 동작합니다:

```
Claude Code (MCP 클라이언트)
 ↓ MCP 프로토콜 (JSON-RPC)
MCP 서버 (외부 서비스 어댑터)
 ↓ API 호출
외부 서비스 (GitHub, Sentry, DB 등)
```

Claude Code가 MCP 클라이언트 역할을 하며, 각 MCP 서버는 특정 외부 서비스에 대한 어댑터(adapter)로 동작합니다. Claude Code는 MCP 서버가 제공하는 도구(tools)를 자동으로 발견하고, 사용자의 요청에 따라 적절한 도구를 선택하여 호출합니다.

## 12.2 MCP 서버 설치 방법

MCP 서버는 세 가지 전송(transport) 방식으로 설치할 수 있습니다.

### 방법 1: HTTP 서버 (권장)

HTTP 방식은 가장 범용적이고 권장되는 방법입니다. 원격 서버에 HTTP POST 요청을 보내 통신합니다.

```
기본 구문
claude mcp add --transport http <이름> <URL>

예시: Notion 연결
claude mcp add --transport http notion https://mcp.notion.com/mcp

Bearer 토큰 인증이 필요한 경우
claude mcp add --transport http secure-api https://api.example.com/mcp \
 --header "Authorization: Bearer your-token-here"
```

많은 SaaS 서비스들이 HTTP MCP 엔드포인트를 공식적으로 제공하고 있어, 이 방식이 가장 간편합니다.

### 방법 2: SSE 서버

SSE(Server-Sent Events) 방식은 HTTP의 이전 버전입니다. 기존에 SSE로 제공되는 서버가 있다면 계속 사용할 수 있지만, 신규 연결에는 HTTP 방식을 권장합니다.

```
claude mcp add --transport sse asana https://mcp.asana.com/sse
```



### 방법 3: Stdio 서버 (로컬)

Stdio 방식은 로컬에서 프로세스를 직접 실행하여 표준 입출력(stdin/stdout)으로 통신합니다. 데이터베이스 연결이나 커스텀 도구에 적합합니다.

```
Airtable MCP 서버 (npx로 실행)
claude mcp add --transport stdio --env AIRTABLE_API_KEY=YOUR_KEY airtable \
 -- npx -y airtable-mcp-server

PostgreSQL 데이터베이스 연결
claude mcp add --transport stdio db \
 -- npx -y @bytebase/dbhub \
 --dsn "postgresql://readonly:pass@localhost:5432/mydb"
```

Stdio 방식은 서버 프로세스가 로컬에서 실행되므로, 네트워크 지연 없이 빠르게 동작합니다. 단, npx나 별도의 런타임이 필요할 수 있습니다.

## 12.3 MCP 관리 명령어

MCP 서버를 추가한 후에는 다양한 관리 명령어로 상태를 확인하고 관리할 수 있습니다.

```
설치된 MCP 서버 목록 보기
claude mcp list

특정 서버의 상세 정보 확인
claude mcp get github

서버 제거
claude mcp remove github

Claude Desktop에서 MCP 설정 가져오기
claude mcp add-from-claude-desktop

JSON 형식으로 서버 추가
claude mcp add-json weather-api
'{"type":"http","url":"https://api.weather.com/mcp"}'
```

대화형 모드에서는 `/mcp` 슬래시 명령어로 MCP 서버 연결 상태를 확인하고, OAuth 인증을 관리할 수 있습니다.

## 12.4 MCP 설치 범위 (Scope)

MCP 서버 설정은 세 가지 범위(scope)로 저장할 수 있으며, 용도에 따라 적절한 범위를 선택합니다.

| 범위         | 저장 위치                                 | 설명              | 사용 시나리오          |
|------------|---------------------------------------|-----------------|------------------|
| local (기본) | <code>~/.claude.json</code> 내 프로젝트 경로 | 이 프로젝트에서 나만 사용  | 개인 API 키가 필요한 서버 |
| project    | <code>.mcp.json</code> (소스 컨트롤)       | 팀 전체 공유         | 팀 공용 서비스 연결      |
| user       | <code>~/.claude.json</code>           | 모든 프로젝트에서 나만 사용 | 개인적으로 자주 쓰는 서버   |

# 범위를 지정하여 추가

```
claude mcp add --transport http stripe --scope user https://mcp.stripe.com
claude mcp add --transport http paypal --scope project https://mcp.paypal.com/mcp
```

## 12.5 팀 공유용 .mcp.json

팀원들과 MCP 설정을 공유하려면 프로젝트 루트에 `.mcp.json` 파일을 생성하고 git에 커밋합니다.

## 기본 구조

```
{
 "mcpServers": {
 "github": {
 "type": "http",
 "url": "https://api.githubcopilot.com/mcp/"
 },
 "sentry": {
 "type": "http",
 "url": "https://mcp.sentry.dev/mcp"
 },
 "db-readonly": {
 "command": "npx",
 "args": ["-y", "@bytebase/dbhub", "--dsn", "${DATABASE_READONLY_URL}"],
 "env": {}
 }
 }
}
```

## 환경 변수 확장

`.mcp.json`에서는 환경 변수 확장(expansion)을 지원합니다. 이를 통해 API 키 같은 민감한 정보를 파일에 직접 포함하지 않고, 각 개발자의 환경에서 동적으로 주입할 수 있습니다.

```
{
 "mcpServers": {
 "api-server": {
 "type": "http",
 "url": "${API_BASE_URL:-https://api.example.com}/mcp",
 "headers": {
 "Authorization": "Bearer ${API_KEY}"
 }
 }
 }
}
```

위 예시에서 `${API_BASE_URL:-https://api.example.com}`은 `API_BASE_URL` 환경 변수가 설정되어 있으면 그 값을, 없으면 기본값 `https://api.example.com`을 사용합니다.

## .mcp.json 사용 시 주의사항

- API 키나 토큰을 `.mcp.json`에 직접 하드코딩하지 마십시오. 반드시 환경 변수 확장을 사용합니다.
- `.mcp.json`에 정의된 MCP 서버는 팀원 모두에게 적용되므로, 팀에서 공통으로 사용하는 서비스만 포함합니다.
- 개인적으로 사용하는 MCP 서버는 `--scope local` 또는 `--scope user`로 별도 관리합니다.

## 12.6 실전 MCP 활용 예시

### GitHub 코드 리뷰

```
GitHub MCP 서버 연결
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
대화형 모드에서 /mcp로 OAuth 인증
```

연결 후 다음과 같이 활용합니다:

- > 내게 할당된 오픈 PR 목록을 보여줘
- > PR #456을 리뷰하고 개선 사항을 제안해줘
- > 방금 발견한 버그에 대해 새로운 이슈를 만들어줘

### Sentry 에러 모니터링

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

- > 최근 24시간 동안 가장 많이 발생한 에러를 보여줘
- > 에러 ID abc123의 스택 트레이스를 분석해줘
- > 이 에러의 원인을 파악하고 수정 코드를 작성해줘

## PostgreSQL 데이터베이스 쿼리

```
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub \
 --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"
```

```
> 이번 달 총 매출은 얼마야?
> orders 테이블의 스키마를 보여줘
> 최근 7일간 가입한 사용자 수를 일별로 보여줘
```

**보안 팁:** 데이터베이스 MCP 서버에는 반드시 읽기 전용(readonly) 계정을 사용하십시오. Claude Code가 임의의 쿼리를 실행할 수 있으므로, 쓰기 권한이 있으면 의도치 않은 데이터 변경이 발생할 수 있습니다.

## 12.7 Claude Code를 MCP 서버로 사용하기

Claude Code 자체를 MCP 서버로 실행하여 Claude Desktop 등 다른 MCP 클라이언트에서 사용할 수도 있습니다.

```
Claude Code를 MCP 서버 모드로 실행
claude mcp serve
```

Claude Desktop에서 연결하려면 다음과 같이 설정합니다:

```
{
 "mcpServers": {
 "claude-code": {
 "type": "stdio",
 "command": "claude",
 "args": ["mcp", "serve"],
 "env": {}
 }
 }
}
```

이렇게 하면 Claude Desktop에서 Claude Code의 파일 편집, 코드 실행 등의 도구를 활용할 수 있습니다.

---

## 12.8 Tool Search로 컨텍스트 최적화

MCP 서버를 많이 연결하면 도구(tool) 정의가 컨텍스트 윈도우의 상당 부분을 차지하게 됩니다. Tool Search는 이 문제를 해결하기 위해 도구 정의를 미리 로드하는 대신, 필요할 때 동적으로 검색하는 기능입니다.

### 동작 원리

- 등록된 MCP 도구 수가 일정 임계값을 넘으면 자동으로 활성화됩니다.
- Claude Code가 작업을 수행할 때 관련 도구를 키워드로 검색하여 사용합니다.
- 컨텍스트 사용량을 약 72,000 토큰에서 약 8,700 토큰으로 절감할 수 있습니다.

### 설정 방법

```
커스텀 임계값 설정 (도구 수의 5%를 초과하면 활성화)
ENABLE_TOOL_SEARCH=auto:5 claude

강제 활성화
ENABLE_TOOL_SEARCH=true claude

비활성화
ENABLE_TOOL_SEARCH=false claude
```

---

## 정리

이 챕터에서는 MCP(Model Context Protocol)를 통한 Claude Code의 능력 확장을 살펴보았습니다.

- **MCP**는 AI 도구와 외부 서비스를 연결하는 오픈 소스 표준으로, Claude Code의 기능을 사실상 무한하게 확장할 수 있습니다.
- **세 가지 전송 방식**(HTTP, SSE, Stdio)을 지원하며, HTTP가 가장 권장됩니다.
- MCP 서버는 **세 가지 범위**(local, project, user)로 설치할 수 있으며, 팀 공유는 `.mcp.json` 파일을 통해 이루어집니다.
- **실전 활용 사례**로 GitHub 코드 리뷰, Sentry 에러 모니터링, 데이터베이스 쿼리 등을 살펴보았습니다.
- **Tool Search**를 통해 다수의 MCP 도구가 컨텍스트를 과도하게 소비하는 문제를 해결할 수 있습니다.
- 보안을 위해 API 키는 환경 변수로 관리하고, 데이터베이스는 읽기 전용 계정을 사용하는 것이 중요합니다.

# Chapter 13: Hooks로 워크플로우 자동화

## 학습 목표

- 1. Hooks 시스템의 4가지 타입(command, http, prompt, agent)을 이해합니다
- 2. 라이프사이클 이벤트별로 적절한 Hook을 설계하고 구현합니다
- 3. 코드 품질과 보안을 자동으로 보장하는 Hook 파이프라인을 구축합니다
- 4. Hook 디버깅과 테스트 방법을 익힙니다

## 13.1 Hooks 시스템 개요

Hooks는 Claude Code의 라이프사이클 특정 시점에서 **자동으로 실행되는** 사용자 정의 동작입니다. "파일을 편집할 때마다 Prettier를 실행해줘", "위험한 명령어는 실행 전에 차단해줘"와 같은 규칙을 한 번 설정해두면, Claude Code가 매번 자동으로 지켜줍니다.

Hooks가 특별한 이유는 **결정론적 제어(deterministic control)** 를 제공하기 때문입니다. CLAUDE.md에 "항상 Prettier를 실행하세요"라고 적어두면 Claude가 가끔 잊을 수 있지만, Hook으로 설정하면 100% 실행됩니다.

### Hook의 4가지 타입

| 타입      | 설명                                                      | 사용 시나리오            |
|---------|---------------------------------------------------------|--------------------|
| command | 셸 명령어를 실행합니다. stdin으로 JSON 입력을 받고, exit code로 결과를 전달합니다 | 포매팅, 린팅, 파일 보호, 로깅 |
| http    | HTTP POST 요청으로 이벤트 JSON을 외부 엔드포인트에 전송합니다                | 외부 서비스 알림, 웹훅      |
| prompt  | Claude 모델(기본 Haiku)에 단일 턴 평가를 요청합니다                     | 코드 품질 평가, 완료 여부 확인 |
| agent   | 도구 접근이 가능한 서브에이전트를 생성합니다                                | 테스트 실행, 복잡한 검증     |

## 13.2 Hook 이벤트 종류

Claude Code는 다양한 라이프사이클 이벤트에서 Hook을 실행할 수 있습니다. 주요 이벤트를 시점별로 정리합니다.

### 세션 라이프사이클

| 이벤트                        | 발생 시점              |
|----------------------------|--------------------|
| <code>SessionStart</code>  | 세션이 시작되거나 재개될 때    |
| <code>SessionEnd</code>    | 세션이 종료될 때          |
| <code>Stop</code>          | Claude가 응답을 완료했을 때 |
| <code>TaskCompleted</code> | 작업이 완료로 표시되었을 때    |

### 도구 실행 라이프사이클

| 이벤트                             | 발생 시점                           |
|---------------------------------|---------------------------------|
| <code>PreToolUse</code>         | 도구(Bash, Edit, Write 등)가 실행되기 전 |
| <code>PostToolUse</code>        | 도구가 성공적으로 실행된 후                 |
| <code>PostToolUseFailure</code> | 도구 실행이 실패한 후                    |
| <code>PermissionRequest</code>  | 권한 대화상자가 표시될 때                  |

### 사용자 입력

| 이벤트                           | 발생 시점                    |
|-------------------------------|--------------------------|
| <code>UserPromptSubmit</code> | 사용자가 프롬프트를 제출했을 때 (처리 전) |



## 알림 및 기타

| 이벤트                | 발생 시점                        |
|--------------------|------------------------------|
| Notification       | 알림이 전송될 때                    |
| SubagentStart      | 서브에이전트가 생성될 때                |
| SubagentStop       | 서브에이전트가 완료될 때                |
| TeammateIdle       | 에이전트 팀 동료가 유휴 상태가 되려 할 때     |
| InstructionsLoaded | CLAUDE.md 또는 rules 파일이 로드될 때 |
| ConfigChange       | 설정 파일이 변경될 때                 |
| PreCompact         | 컨텍스트 압축 전                    |
| PostCompact        | 컨텍스트 압축 후                    |
| WorktreeCreate     | 워크트리가 생성될 때                  |
| WorktreeRemove     | 워크트리가 제거될 때                  |

## 13.3 Hook 설정 구조

Hook은 settings.json의 `hooks` 키에 정의합니다. 기본 구조는 다음과 같습니다:

```

{
 "hooks": {
 "<이벤트명>": [
 {
 "matcher": "<도구명 패턴>",
 "hooks": [
 {
 "type": "<hook 타입>",
 "command": "<실행할 명령어>",
 "timeout": 30
 }
]
 }
]
 }
}

```

## 구조 설명

- **이벤트명**: Hook이 트리거될 이벤트입니다 (예: `PreToolUse` , `PostToolUse` ).
- **matcher**: 선택적 필터입니다. 특정 도구(예: `"Edit|Write"` )에만 Hook을 적용할 때 사용합니다. 빈 문자열 `" "` 이면 모든 도구에 매칭됩니다.
- **hooks**: 실행할 Hook 동작의 배열입니다. 여러 Hook을 순서대로 실행할 수 있습니다.
- **type**: Hook의 타입입니다 ( `command` , `http` , `prompt` , `agent` ).
- **timeout**: 타임아웃(초)입니다. 기본값은 Hook 타입에 따라 다릅니다.

## | 13.4 command Hook 상세

command Hook은 가장 자주 사용되는 타입으로, 셸 명령어를 실행하여 결과를 전달합니다.

### 입출력 프로토콜

- **입력**: stdin으로 이벤트에 대한 JSON 데이터가 전달됩니다.
- **출력**: exit code로 결과를 전달합니다. stdout에서 JSON 구조를 파싱합니다.

## Exit Code 의미

| Exit Code | 의미     | 동작                                |
|-----------|--------|-----------------------------------|
| 0         | 성공     | stdout에서 JSON 제어 구조를 파싱합니다        |
| 2         | 차단 에러  | stderr의 메시지를 피드백으로 Claude에게 전달합니다 |
| 기타        | 비차단 에러 | stderr은 verbose 모드에서만 표시됩니다       |

## 예시: 파일 편집 후 자동 포매팅

파일이 편집될 때마다 Prettier를 자동으로 실행하는 Hook입니다:

```
{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
 }
]
 }
]
 }
}
```

이 Hook의 동작 과정:

- 1. Claude Code가 Edit 또는 Write 도구를 사용한 후 트리거됩니다.
- 2. stdin으로 전달된 JSON에서 `tool_input.file_path`를 추출합니다.
- 3. 해당 파일에 Prettier를 실행합니다.

## 예시: 보호 파일 편집 차단

중요한 파일을 실수로 수정하지 못하도록 차단하는 Hook입니다. 먼저 스크립트 파일을 생성합니다:

```
.claude/hooks/protect-files.sh :
```

```
#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

PROTECTED_PATTERNS=".env" "package-lock.json" ".git/"

for pattern in "${PROTECTED_PATTERNS[@]}; do
 if [["$FILE_PATH" == *"$pattern"*]]; then
 echo "Blocked: $FILE_PATH matches protected pattern '$pattern'" >&2
 exit 2
 fi
done

exit 0
```

이 스크립트를 PreToolUse Hook으로 등록합니다:

```
{
 "hooks": {
 "PreToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/protect-files.sh"
 }
]
 }
]
 }
}
```

`$CLAUDE_PROJECT_DIR` 은 Claude Code가 자동으로 제공하는 환경 변수로, 현재 프로젝트의 루트 디렉터리를 가리킵니다.

## 예시: 위험한 명령어 차단

`rm -rf` 같은 파괴적 명령어를 자동으로 차단하는 Hook입니다:

```
#!/bin/bash
COMMAND=$(jq -r '.tool_input.command')

if echo "$COMMAND" | grep -q 'rm -rf'; then
 jq -n '{
 hookSpecificOutput: {
 hookEventName: "PreToolUse",
 permissionDecision: "deny",
 permissionDecisionReason: "Destructive command blocked"
 }
 }'
else
 exit 0
fi
```

이 스크립트는 exit code 0으로 종료하면서 stdout으로 JSON을 출력합니다. `permissionDecision: "deny"` 를 포함한 JSON 구조를 반환하면 Claude Code가 해당 도구 실행을 거부합니다.

## 예시: 모든 Bash 명령어 로깅

Claude Code가 실행하는 모든 Bash 명령어를 로그 파일에 기록하는 Hook입니다:

```
{
 "hooks": {
 "PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "bash -c 'jq .tool_input.command >> ~/claude-bash-log.txt'",
 "async": true
 }
]
 }
]
 }
}
```

`async: true` 를 설정하면 Hook이 비동기로 실행되어 Claude Code의 작업 흐름을 차단하지 않습니다. 로깅처럼 결과를 기다릴 필요가 없는 작업에 적합합니다.

## | 13.5 http Hook 상세

http Hook은 이벤트 데이터를 HTTP POST 요청으로 외부 엔드포인트에 전송합니다. 웹훅(webhook) 방식으로 외부 서비스에 알림을 보내는 데 사용합니다.

```
{
 "hooks": {
 "Stop": [
 {
 "hooks": [
 {
 "type": "http",
 "url": "https://hooks.slack.com/services/T.../B.../...",
 "method": "POST",
 "headers": {
 "Content-Type": "application/json"
 }
 }
]
 }
]
 }
}
```

이 Hook은 Claude가 응답을 완료할 때마다 Slack 웹훅에 이벤트 정보를 전송합니다.

## | 13.6 prompt Hook 상세

prompt Hook은 Claude 모델(기본으로 Haiku)에 단일 턴 평가를 요청합니다. 코드 품질 검사, 완료 여부 확인 등에 유용합니다.

```
{
 "hooks": {
 "Stop": [
 {
 "hooks": [
 {
 "type": "prompt",
 "prompt": "Check if all tasks are complete. If not, respond with\n{\\\"ok\\\": false, \\\"reason\\\": \\\"what remains to be done\\\"}."
 }
]
 }
]
 }
}
```

이 Hook은 Claude가 응답을 마칠 때마다 Haiku 모델에게 "모든 작업이 완료되었는지" 평가를 요청합니다. 완료되지 않은 작업이 있으면 Claude가 이어서 작업합니다.

## prompt Hook의 장점

- 별도의 스크립트 파일 없이 설정 파일 내에서 바로 정의할 수 있습니다.
- Haiku 모델을 사용하므로 빠르고 비용 효율적입니다.
- 복잡한 판단이 필요한 검증(예: "코드가 프로젝트 스타일 가이드를 따르는가?")에 적합합니다.

## | 13.7 agent Hook 상세

agent Hook은 도구 접근이 가능한 서브 에이전트를 생성하여 복잡한 검증 작업을 수행합니다. prompt Hook과 달리 파일을 읽고, 명령을 실행하고, 다단계 추론이 가능합니다.

```

{
 "hooks": {
 "Stop": [
 {
 "hooks": [
 {
 "type": "agent",
 "prompt": "Verify that all unit tests pass. Run the test suite and check
the results. $ARGUMENTS",
 "timeout": 120
 }
]
 }
]
 }
}

```

이 Hook은 Claude가 작업을 마칠 때마다 서브에이전트를 생성하여 테스트 스위트를 실행하고, 모든 테스트가 통과하는지 확인합니다. 테스트 실패 시 Claude에게 피드백을 전달하여 수정을 유도합니다.

**주의:** agent Hook은 서브에이전트가 도구를 사용하므로 비용이 발생합니다. timeout을 적절히 설정하여 무한 실행을 방지하십시오.

## 13.8 실전 Hook 레시피 모음

### 데스크톱 알림 (macOS)

Claude Code가 사용자 주의를 필요로 할 때 macOS 알림을 표시합니다:



```
{
 "hooks": {
 "Notification": [
 {
 "matcher": "",
 "hooks": [
 {
 "type": "command",
 "command": "osascript -e 'display notification \"Claude Code needs your attention\" with title \"Claude Code\"'"
 }
]
 }
]
 }
}
```

## 압축 후 컨텍스트 재주입

컨텍스트 압축( `/compact` ) 후에 중요한 정보를 다시 주입합니다:

```
{
 "hooks": {
 "SessionStart": [
 {
 "matcher": "compact",
 "hooks": [
 {
 "type": "command",
 "command": "echo 'Reminder: use Bun, not npm. Run bun test before committing. Current sprint: auth refactor.'"
 }
]
 }
]
 }
}
```

## TypeScript 파일 편집 후 타입 체크

TypeScript 파일이 수정될 때마다 자동으로 타입 검사를 실행합니다:

```

{
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "FILE=$(jq -r '.tool_input.file_path'); if [[\"$FILE\" == *.ts || \"$FILE\" == *.tsx]]; then npx tsc --noEmit 2>&1 | head -20; fi"
 }
]
 }
]
 }
}

```

## 13.9 Hook 설정 위치와 디버깅

### 설정 위치

Hook은 다른 설정과 마찬가지로 4단계 범위로 설정할 수 있습니다:

| 위치                                       | 범위      | 팀 공유       |
|------------------------------------------|---------|------------|
| <code>~/.claude/settings.json</code>     | 모든 프로젝트 | 아니오        |
| <code>.claude/settings.json</code>       | 단일 프로젝트 | 예 (git 커밋) |
| <code>.claude/settings.local.json</code> | 단일 프로젝트 | 아니오        |
| Managed 정책 설정                            | 조직 전체   | 예 (관리자)    |

## 디버깅 방법

```
현재 설정된 Hook 목록 확인
/hooks

상세 로그 모드로 실행 (비차단 에러의 stderr 확인 가능)
claude --verbose

특정 카테고리 디버그
claude --debug "hooks"
```

Hook이 예상대로 동작하지 않을 때 확인할 체크리스트:

1. **JSON 문법 오류**: settings.json이 유효한 JSON인지 확인합니다.
2. **matcher 패턴**: 도구 이름과 정확히 일치하는지 확인합니다 (예: `"Edit|Write"`, `"Bash"`).
3. **스크립트 권한**: command Hook의 스크립트 파일에 실행 권한(`chmod +x`)이 있는지 확인합니다.
4. **Exit code**: exit code 2는 차단 에러, 0은 성공, 그 외는 비차단 에러입니다.
5. **jq 설치 여부**: JSON 파싱에 jq를 사용하는 경우 설치되어 있는지 확인합니다.

## 정리

이 챕터에서는 Hooks 시스템을 통한 Claude Code 워크플로우 자동화를 살펴보았습니다.

- **Hooks**는 Claude Code의 라이프사이클 이벤트에 자동으로 실행되는 동작을 정의하여, CLAUDE.md보다 확실한 **결정론적 제어**를 제공합니다.
- **4가지 Hook 타입** — command(셸 명령), http(웹훅), prompt(모델 평가), agent(서브에이전트) — 을 상황에 맞게 선택합니다.
- **주요 이벤트**로는 PreToolUse(도구 실행 전 차단/검증), PostToolUse(도구 실행 후 후처리), Stop(응답 완료 시 검증), Notification(알림 전달) 등이 있습니다.
- **실전 레시피**로 자동 포매팅, 보호 파일 차단, 위험 명령어 차단, 데스크톱 알림, Bash 로깅, 테스트 자동 실행 등을 구현할 수 있습니다.
- **Exit code 프로토콜**(0=성공, 2=차단, 기타=비차단)과 JSON 제어 구조를 통해 Hook과 Claude Code가 양방향으로 통신합니다.

# Chapter 14: IDE 연동과 팀 협업

## 학습 목표

1. VS Code와 JetBrains IDE에서 Claude Code를 연동하여 시각적 워크플로를 활용합니다
2. 데스크톱 앱과 웹 브라우저를 통한 추가적인 사용 환경을 이해합니다
3. 팀 프로젝트에서 Claude Code 설정을 표준화하고 공유하는 방법을 익힙니다
4. Git Worktree, 멀티 에이전트, CI/CD 통합 등 고급 협업 패턴을 학습합니다

## 14.1 VS Code 확장

### 설치 방법

VS Code에서 Claude Code 확장을 설치하는 방법은 두 가지입니다:

1. **Extensions 뷰에서 검색:** VS Code의 Extensions 패널에서 "Claude Code"를 검색하여 설치합니다.
2. **직접 설치 링크:** VS Code용( `vscode:extension/anthropic.claude-code` ) 또는 Cursor용( `cursor:extension/anthropic.claude-code` ) 링크를 통해 직접 설치합니다.

요구사항: VS Code 1.98.0 이상, Anthropic 계정

### 주요 기능

#### 인라인 diff

Claude Code가 파일을 수정하면, VS Code에서 원본과 수정본을 나란히 비교하는 diff 뷰를 자동으로 제공합니다. 각 변경 사항을 개별적으로 수락하거나 거부할 수 있어, 터미널에서보다 훨씬 직관적으로 코드 변경을 검토할 수 있습니다.

#### @-멘션으로 컨텍스트 공유

입력창에서 @ 기호를 사용하면 파일, 폴더, 줄 범위를 명시적으로 참조할 수 있습니다:

- `@auth.js` — 특정 파일 참조
- `@src/components/` — 폴더 전체 참조
- `@utils.ts:15-30` — 특정 줄 범위 참조

VS Code에서는 `Option+K` (Mac) / `Alt+K` 로 빠르게 @-멘션 참조를 삽입할 수 있습니다.

### 계획 검토

Plan 모드에서 Claude가 작성한 계획은 마크다운 문서로 VS Code에 표시됩니다. 인라인 코멘트를 통해 계획을 검토하고 수정 요청을 할 수 있습니다.

### 다중 대화

여러 탭이나 창에서 동시에 여러 Claude Code 대화를 진행할 수 있습니다. 이를 통해 한 대화에서는 기능 구현을, 다른 대화에서는 테스트 작성을 병렬로 수행할 수 있습니다.

### 체크포인트

VS Code 확장은 코드 변경에 대한 체크포인트를 자동으로 생성합니다. 원하지 않는 변경이 있으면 체크포인트로 되돌릴 수 있습니다.

### 원격 세션 재개

claude.ai 웹에서 시작한 세션을 VS Code에서 이어서 작업할 수 있습니다.

### VS Code 설정 옵션

| 설정                                 | 기본값                  | 설명                        |
|------------------------------------|----------------------|---------------------------|
| <code>selectedModel</code>         | <code>default</code> | 새 대화에서 사용할 모델             |
| <code>useTerminal</code>           | <code>false</code>   | 그래픽 패널 대신 터미널 모드 사용       |
| <code>initialPermissionMode</code> | <code>default</code> | 초기 권한 모드                  |
| <code>autosave</code>              | <code>true</code>    | 파일 자동 저장                  |
| <code>useCtrlEnterToSend</code>    | <code>false</code>   | Ctrl+Enter로 전송 (Enter 대신) |

## 주요 단축키

| 명령                           | Mac                        | Windows/Linux               |
|------------------------------|----------------------------|-----------------------------|
| Focus 전환 (에디터 ↔ Claude Code) | <code>Cmd+Esc</code>       | <code>Ctrl+Esc</code>       |
| 새 탭에서 열기                     | <code>Cmd+Shift+Esc</code> | <code>Ctrl+Shift+Esc</code> |
| 새 대화 시작                      | <code>Cmd+N</code>         | <code>Ctrl+N</code>         |
| @-멘션 참조 삽입                   | <code>Option+K</code>      | <code>Alt+K</code>          |

## 14.2 JetBrains 플러그인

Claude Code는 IntelliJ IDEA, PyCharm, WebStorm 등 모든 JetBrains IDE를 지원합니다.

### 설치

JetBrains Marketplace에서 "Claude Code"를 검색하여 설치합니다.

### 주요 기능

- **인터랙티브 diff 뷰:** VS Code와 마찬가지로 코드 변경 사항을 시각적으로 비교하고 선택적으로 수락할 수 있습니다.
- **선택 컨텍스트 공유:** 에디터에서 코드를 선택한 상태로 Claude Code에 질문하면, 선택된 코드가 자동으로 컨텍스트에 포함됩니다.
- **IDE 내 패널:** JetBrains IDE의 도구 윈도우에 Claude Code 패널이 통합됩니다.

JetBrains 플러그인은 터미널의 Claude Code와 동일한 설정 파일(settings.json, CLAUDE.md 등)을 공유하므로, 터미널에서 설정한 내용이 IDE에서도 그대로 적용됩니다.

## 14.3 데스크톱 앱과 웹 브라우저

### 데스크톱 앱

Claude Code 데스크톱 앱은 IDE나 터미널 외부에서 독립적으로 실행됩니다.

#### 주요 특징:

- diff를 시각적으로 검토할 수 있습니다

- 여러 세션을 나란히 실행할 수 있습니다
- 반복 작업을 스케줄링할 수 있습니다
- 클라우드 세션을 시작할 수 있습니다

다운로드:

- macOS: Intel 및 Apple Silicon 모두 지원
- Windows: x64 지원

웹 브라우저 (claude.ai/code)

claude.ai/code에서 로컬 설정 없이 웹 브라우저에서 Claude Code를 실행할 수 있습니다. 다음과 같은 시나리오에 특히 유용합니다:

- **장시간 작업:** 대규모 리팩토링이나 테스트 작성을 시작하고, 나중에 결과를 확인합니다.
- **병렬 작업:** 여러 작업을 동시에 실행하고 각각의 진행 상황을 모니터링합니다.
- **모바일 확인:** 이동 중에도 진행 중인 작업의 상태를 확인할 수 있습니다.

웹에서 시작한 세션은 `/desktop` 명령으로 데스크톱 앱에서, 또는 VS Code 확장의 원격 세션 재개 기능으로 로컬 환경에서 이어갈 수 있습니다.

14.4 팀 설정 표준화

팀에서 Claude Code를 효과적으로 사용하려면 공유 설정을 표준화하고, git으로 관리해야 합니다.

팀 공유 파일 목록

| 파일                                       | 용도                      | git 커밋          |
|------------------------------------------|-------------------------|-----------------|
| <code>.claude/settings.json</code>       | 팀 공통 설정 (권한 규칙, Hook 등) | 예               |
| <code>.claude/CLAUDE.md</code>           | 프로젝트 지시사항               | 예               |
| <code>.claude/rules/</code>              | 주제별 규칙 파일               | 예               |
| <code>.mcp.json</code>                   | 팀 공유 MCP 서버 설정          | 예               |
| <code>.claude/settings.local.json</code> | 개인 설정                   | 아니오 (gitignore) |

## 팀 온보딩 설정 예시

새로운 팀원이 프로젝트에 참여할 때 즉시 생산적으로 작업할 수 있도록, 다음과 같은 팀 설정을 준비합니다:

### `.claude/settings.json` (팀 공유)

```
{
 "$schema": "https://json.schemastore.org/claude-code-settings.json",
 "permissions": {
 "allow": [
 "Bash(npm run *)",
 "Bash(npx *)",
 "Bash(git log *)",
 "Bash(git diff *)",
 "Bash(git status)"
],
 "ask": [
 "Bash(git push *)",
 "Bash(git checkout *)",
 "Bash(npm install *)"
],
 "deny": [
 "Read(./env)",
 "Read(./env.*)",
 "Read(./secrets/**)",
 "Bash(rm -rf *)",
 "Bash(curl *)"
]
 },
 "hooks": {
 "PostToolUse": [
 {
 "matcher": "Edit|Write",
 "hooks": [
 {
 "type": "command",
 "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
 }
]
 }
]
 }
}
```



## `.claude/CLAUDE.md` (프로젝트 지시사항)

### # 프로젝트 규칙

#### ## 빌드 & 테스트

- `npm run build`로 빌드
- `npm test`로 전체 테스트 실행
- 커밋 전에 항상 `npm run lint` 실행

#### ## 코딩 표준

- TypeScript strict 모드 사용
- 2-space 들여쓰기
- 함수에 JSDoc 주석 포함
- API 핸들러는 `src/api/handlers/`에 위치

#### ## 아키텍처

- PostgreSQL 주 데이터베이스
- Redis 캐시 레이어
- REST API는 OpenAPI 3.0 스펙 준수

#### ## 참조

See @README.md for project overview.

See @package.json for available npm commands.

## `.claude/rules/security.md` (보안 규칙)

---  
paths:

- "src/api/\*\*/\*.ts"
- "**src/auth**/\*\*/\*.ts"

---

### # 보안 규칙

- 모든 API 엔드포인트에 입력 검증(input validation) 포함
- SQL 쿼리에 파라미터 바인딩 사용 (문자열 결합 금지)
- 비밀번호는 **bcrypt**로 해싱
- JWT 토큰 만료 시간은 1시간
- CORS 설정에서 허용 도메인 명시적 지정

## 14.5 Git Worktree 병렬 작업

Git Worktree를 활용하면 하나의 저장소에서 여러 브랜치를 동시에 체크아웃하여 병렬로 작업할 수 있습니다. Claude Code는 이를 `--worktree` (`-w`) 플래그로 직접 지원합니다.

### 기본 사용법

```
격리된 워크트리에서 기능 개발
claude -w feature-auth "인증 모듈을 구현해줘"

다른 터미널에서 동시에 다른 기능 개발
claude -w feature-payment "결제 모듈을 구현해줘"
```

각 워크트리는 독립된 디렉터리에 생성되므로, 두 작업이 서로의 파일을 간섭하지 않습니다.

### 워크트리의 장점

- **병렬 작업:** 여러 기능을 동시에 개발할 수 있습니다.
- **격리:** 각 워크트리는 독립적인 파일 시스템을 가집니다.
- **안전:** 한 작업에서의 실패가 다른 작업에 영향을 주지 않습니다.
- **효율:** 브랜치를 전환하지 않으므로 stash/unstash가 불필요합니다.

## 14.6 멀티 에이전트 팀

여러 Claude Code 에이전트를 동시에 실행하여 마치 개발팀처럼 협업시킬 수 있습니다. 각 에이전트는 독립적인 작업을 수행하며, Git을 통해 작업 결과를 통합합니다.

### 멀티 에이전트 시나리오

```
터미널 1: 백엔드 API 개발
claude -w backend -n "backend-dev" "REST API 엔드포인트를 구현해줘"

터미널 2: 프론트엔드 UI 개발
claude -w frontend -n "frontend-dev" "React 컴포넌트를 구현해줘"

터미널 3: 테스트 작성
claude -w tests -n "test-writer" "단위 테스트와 통합 테스트를 작성해줘"
```

## 에이전트 관리

```
에이전트 설정 관리
/agents
```

멀티 에이전트 패턴을 사용할 때는 각 에이전트가 작업하는 파일 범위가 겹치지 않도록 설계하는 것이 중요합니다. 동일한 파일을 여러 에이전트가 동시에 수정하면 머지 충돌이 발생할 수 있습니다.

## | 14.7 CI/CD 통합

Claude Code를 CI/CD 파이프라인에 통합하면 PR 리뷰, 코드 생성, 테스트 자동화 등을 자동화할 수 있습니다.

### GitHub Actions 통합

GitHub Actions에서 Claude Code를 실행하는 예시입니다:

```
.github/workflows/claude-review.yml
name: Claude Code Review
on:
 pull_request:
 types: [opened, synchronize]

jobs:
 review:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - name: Install Claude Code
 run: curl -fsSL https://claude.ai/install.sh | bash
 - name: Run Claude Review
 env:
 ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
 run: |
 git diff origin/main --name-only | \
 claude -p --permission-mode bypassPermissions \
 "review these changed files for bugs and security issues"
```

## Claude Code GitHub App

GitHub App을 설치하면 PR 코멘트에서 직접 Claude Code를 호출할 수 있습니다:

```
GitHub App 설치
/install-github-app
```

## 비대화형 모드 활용

CI/CD에서는 비대화형 모드( `-p` )와 함께 다음 플래그를 활용합니다:

```
예산과 턴 수를 제한하여 비용 관리
claude -p --permission-mode bypassPermissions \
 --max-budget-usd 2.00 \
 --max-turns 5 \
 "refactor the auth module and run tests"

JSON 형식으로 출력하여 후속 처리 용이
claude -p --output-format json \
 "list all TODO comments in src/"

구조화된 출력으로 파싱 용이한 결과 얻기
claude -p --json-schema '{"type":"object","properties":{"issues":{"type":"array"}}}' \
 "find potential bugs in the latest commit"
```

## 14.8 Chrome 연동과 원격 제어

### Chrome 연동

Claude Code는 Chrome 브라우저와 연동하여 웹 앱 테스트 및 브라우저 자동화를 수행할 수 있습니다.

```
Chrome 연동 설정
/chrome
```

설정 후 다음과 같이 활용합니다:

```
> @browser go to localhost:3000 and check the console for errors
> @browser take a screenshot of the login page
> @browser fill in the registration form and submit
```

## 원격 제어

claude.ai에서 시작한 작업을 로컬 터미널에서 이어가거나, 반대로 로컬 작업을 claude.ai에서 모니터링할 수 있습니다.

```
원격 제어 활성화
/remote-control

텔레포트 (원격 세션 연결)
claude --teleport
```

이 기능은 장시간 실행되는 작업을 시작한 후, 다른 기기에서 진행 상황을 확인하고 싶을 때 유용합니다.

## 정리

이 챕터에서는 IDE 연동과 팀 협업 전략을 살펴보았습니다.

- **VS Code 확장**은 인라인 diff, @-멘션, 다중 대화, 체크포인트 등 시각적 워크플로를 제공하며, **JetBrains 플러그인**은 모든 JetBrains IDE에서 유사한 기능을 지원합니다.
- **데스크톱 앱**과 **웹 브라우저**(claude.ai/code)를 통해 터미널이나 IDE 외부에서도 Claude Code를 사용할 수 있습니다.
- **팀 설정 표준화**는 `.claude/settings.json`, `.claude/CLAUDE.md`, `.claude/rules/`, `.mcp.json`을 git에 커밋하여 이루어집니다.
- **Git Worktree**를 활용하면 여러 기능을 독립된 환경에서 동시에 개발할 수 있으며, **멀티 에이전트** 패턴으로 여러 Claude 인스턴스를 병렬로 운영할 수 있습니다.
- **CI/CD 통합**을 통해 PR 리뷰, 코드 생성, 테스트 등을 자동화할 수 있으며, `--permission-mode bypassPermissions`와 `--max-budget-usd` 등의 플래그로 안전하게 운영합니다.
- **Chrome 연동**과 **원격 제어**로 브라우저 자동화 및 원격 세션 관리가 가능합니다.

## 부록

---

# 부록 A: 슬래시 명령어 전체 레퍼런스

## 세션 관리

| 명령어                                                  | 용도                                  | 별칭                                         |
|------------------------------------------------------|-------------------------------------|--------------------------------------------|
| <code>/clear</code>                                  | 대화 기록을 지우고 컨텍스트를 확보합니다              | <code>/reset</code> ,<br><code>/new</code> |
| <code>/compact</code><br><code>[instructions]</code> | 대화를 압축합니다. 선택적으로 포커스 지시를 추가할 수 있습니다 | —                                          |
| <code>/resume [session]</code>                       | 이전 세션을 이어갑니다                        | <code>/continue</code>                     |
| <code>/fork [name]</code>                            | 현재 대화의 분기점을 생성합니다                   | —                                          |
| <code>/exit</code>                                   | CLI를 종료합니다                          | <code>/quit</code>                         |
| <code>/rename [name]</code>                          | 세션 이름을 변경합니다                        | —                                          |

## 설정 및 정보

| 명령어                          | 용도                                      | 별칭                     |
|------------------------------|-----------------------------------------|------------------------|
| <code>/config</code>         | 설정 인터페이스를 엽니다                           | <code>/settings</code> |
| <code>/model [model]</code>  | AI 모델을 선택/변경합니다 (즉시 적용)                 | —                      |
| <code>/effort [level]</code> | 노력 수준을 설정합니다 (low/medium/high/max/auto) | —                      |
| <code>/help</code>           | 도움말 및 사용 가능한 명령어를 표시합니다                 | —                      |
| <code>/status</code>         | 버전, 모델, 계정, 연결 상태를 표시합니다                | —                      |
| <code>/cost</code>           | 토큰 사용량 통계를 표시합니다                        | —                      |
| <code>/usage</code>          | 플랜 사용 한도 및 속도 제한 상태를 표시합니다              | —                      |
| <code>/context</code>        | 현재 컨텍스트 사용량을 시각화합니다                     | —                      |
| <code>/doctor</code>         | 설치 및 설정을 진단합니다                          | —                      |
| <code>/version</code>        | 버전 정보를 표시합니다                            | —                      |

## 코드 작업

| 명령어                            | 용도                          | 별칭                       |
|--------------------------------|-----------------------------|--------------------------|
| <code>/init</code>             | CLAUDE.md 가이드로 프로젝트를 초기화합니다 | —                        |
| <code>/diff</code>             | 커밋되지 않은 변경 사항을 인터랙티브하게 봅니다  | —                        |
| <code>/plan</code>             | 프롬프트에서 바로 계획 모드에 진입합니다      | —                        |
| <code>/rewind</code>           | 대화 및/또는 코드를 이전 시점으로 되돌립니다   | <code>/checkpoint</code> |
| <code>/security-review</code>  | 현재 브랜치의 보안 취약점을 분석합니다       | —                        |
| <code>/pr-comments [PR]</code> | GitHub PR 코멘트를 가져옵니다        | —                        |



## 도구 및 연동

| 명령어                       | 용도                          | 별칭                          |
|---------------------------|-----------------------------|-----------------------------|
| <code>/mcp</code>         | MCP 서버 연결 및 OAuth 인증을 관리합니다 | —                           |
| <code>/hooks</code>       | 훅 설정을 봅니다                   | —                           |
| <code>/permissions</code> | 권한을 보거나 업데이트합니다             | <code>/allowed-tools</code> |
| <code>/memory</code>      | CLAUDE.md 메모리 파일을 편집합니다     | —                           |
| <code>/ide</code>         | IDE 연동을 관리합니다               | —                           |
| <code>/chrome</code>      | Claude in Chrome을 설정합니다     | —                           |
| <code>/skills</code>      | 사용 가능한 스킬 목록을 표시합니다         | —                           |
| <code>/plugin</code>      | 플러그인을 관리합니다                 | —                           |
| <code>/agents</code>      | 에이전트 설정을 관리합니다              | —                           |

## 기타

| 명령어                                | 용도                              | 별칭                |
|------------------------------------|---------------------------------|-------------------|
| <code>/copy</code>                 | 마지막 응답을 클립보드에 복사합니다             | —                 |
| <code>/export [filename]</code>    | 대화를 텍스트로 내보냅니다                  | —                 |
| <code>/feedback</code>             | 피드백을 제출합니다                      | <code>/bug</code> |
| <code>/fast [on off]</code>        | 빠른 모드를 전환합니다                    | —                 |
| <code>/login</code>                | Anthropic 계정에 로그인합니다            | —                 |
| <code>/logout</code>               | 로그아웃합니다                         | —                 |
| <code>/vim</code>                  | Vim / Normal 편집 모드를 전환합니다       | —                 |
| <code>/theme</code>                | 색상 테마를 변경합니다                    | —                 |
| <code>/color [color]</code>        | 프롬프트 바 색상을 설정합니다                | —                 |
| <code>/desktop</code>              | 데스크톱 앱에서 현재 세션을 이어갑니다           | —                 |
| <code>/remote-control</code>       | 원격 제어를 활성화합니다                   | <code>/rc</code>  |
| <code>/sandbox</code>              | 샌드박스 모드를 전환합니다                  | —                 |
| <code>/stats</code>                | 일일 사용량, 세션 히스토리, 모델 선호도를 시각화합니다 | —                 |
| <code>/insights</code>             | 세션 분석 리포트를 생성합니다                | —                 |
| <code>/release-notes</code>        | 전체 변경 로그를 봅니다                   | —                 |
| <code>/btw &lt;question&gt;</code> | 대화에 추가하지 않는 빠른 질문을 합니다          | —                 |
| <code>/stickers</code>             | Claude Code 스티커를 주문합니다          | —                 |
| <code>/tasks</code>                | 백그라운드 작업 목록 및 관리를 합니다           | —                 |
| <code>/keybindings</code>          | 키바인딩 설정 파일을 엽니다                 | —                 |
| <code>/install-github-app</code>   | GitHub Actions 앱을 설정합니다         | —                 |

| 명령어                                | 용도                    | 별칭 |
|------------------------------------|-----------------------|----|
| <code>/install-slack-app</code>    | Slack 앱을 설치합니다        | —  |
| <code>/add-dir &lt;path&gt;</code> | 현재 세션에 작업 디렉터리를 추가합니다 | —  |

---

# 부록 B: CLI 플래그 레퍼런스

## 실행 모드

| 플래그                                       | 설명                           | 기본 값 | 예시                                             |
|-------------------------------------------|------------------------------|------|------------------------------------------------|
| <code>--print</code> , <code>-p</code>    | 비대화형 모드 (출력 후 종료)            | —    | <code>claude -p "explain this function"</code> |
| <code>--continue</code> , <code>-c</code> | 최근 대화를 이어갑니다                 | —    | <code>claude -c</code>                         |
| <code>--resume</code> , <code>-r</code>   | 세션 ID 또는 이름으로 세션을 재개합니다      | —    | <code>claude -r "auth-refactor"</code>         |
| <code>--worktree</code> , <code>-w</code> | 격리된 git worktree에서 세션을 시작합니다 | —    | <code>claude -w feature-auth</code>            |
| <code>--name</code> , <code>-n</code>     | 세션 이름을 설정합니다                 | —    | <code>claude -n "my-feature"</code>            |

## 모델 및 성능

| 플래그                           | 설명                  | 기본값                  | 예시                                                   |
|-------------------------------|---------------------|----------------------|------------------------------------------------------|
| <code>--model</code>          | 사용할 모델을 지정합니다       | <code>default</code> | <code>claude --model opus</code>                     |
| <code>--effort</code>         | 노력 수준을 설정합니다        | <code>medium</code>  | <code>claude --effort high</code>                    |
| <code>--max-turns</code>      | 에이전트의 최대 턴 수를 제한합니다 | 무제한                  | <code>claude -p --max-turns 3 "query"</code>         |
| <code>--max-budget-usd</code> | 최대 비용을 제한합니다 (USD)  | 무제한                  | <code>claude -p --max-budget-usd 5.00 "query"</code> |

## 입출력

| 플래그                          | 설명                          | 기본값               | 예시                                                  |
|------------------------------|-----------------------------|-------------------|-----------------------------------------------------|
| <code>--output-format</code> | 출력 형식을 설정합니다                | <code>text</code> | <code>claude -p --output-format json "query"</code> |
| <code>--json-schema</code>   | JSON 스키마에 맞는 구조화된 출력을 요청합니다 | —                 | <code>claude -p --json-schema '...' "query"</code>  |
| <code>--verbose</code>       | 상세 로그를 출력합니다                | —                 | <code>claude --verbose</code>                       |
| <code>--debug</code>         | 디버그 모드를 활성화합니다              | —                 | <code>claude --debug "api,mcp"</code>               |

## 권한 및 보안

| 플래그                            | 설명            | 기본값                  | 예시                                               |
|--------------------------------|---------------|----------------------|--------------------------------------------------|
| <code>--permission-mode</code> | 권한 모드를 설정합니다  | <code>default</code> | <code>claude --permission-mode plan</code>       |
| <code>--allowedTools</code>    | 허용할 도구를 지정합니다 | 전체                   | <code>--allowedTools "Bash(git *)" "Read"</code> |
| <code>--disallowedTools</code> | 차단할 도구를 지정합니다 | 없음                   | <code>--disallowedTools "WebFetch"</code>        |

## 시스템 프롬프트

| 플래그                                      | 설명                      | 예시                                                                 |
|------------------------------------------|-------------------------|--------------------------------------------------------------------|
| <code>--system-prompt</code>             | 기본 시스템 프롬프트를 전체 교체합니다   | <code>claude --system-prompt "You are a Python expert"</code>      |
| <code>--system-prompt-file</code>        | 파일 내용으로 시스템 프롬프트를 교체합니다 | <code>claude --system-prompt-file ./prompts/review.txt</code>      |
| <code>--append-system-prompt</code>      | 기본 프롬프트에 내용을 추가합니다      | <code>claude --append-system-prompt "Always use TypeScript"</code> |
| <code>--append-system-prompt-file</code> | 파일 내용을 기본 프롬프트에 추가합니다   | <code>claude --append-system-prompt-file ./style-rules.txt</code>  |

## 기타

| 플래그                       | 설명                  | 예시                                           |
|---------------------------|---------------------|----------------------------------------------|
| <code>--add-dir</code>    | 추가 작업 디렉터리를 지정합니다   | <code>claude --add-dir ../apps ../lib</code> |
| <code>--mcp-config</code> | MCP 서버 설정 파일을 로드합니다 | <code>claude --mcp-config ./mcp.json</code>  |

## 부록 C: 트러블슈팅 가이드

---

### | 설치 문제

#### Node.js 버전 충돌

증상: `Error: Unsupported Node.js version` 또는 설치 시 오류 발생

해결법:

1. Node.js 18 이상이 설치되어 있는지 확인합니다:

```
node --version
```

2. nvm을 사용하는 경우 올바른 버전을 활성화합니다:

```
nvm install 18
nvm use 18
```

3. 네이티브 설치 방식을 사용하면 Node.js 의존성 없이 설치할 수 있습니다:

```
curl -fsSL https://claude.ai/install.sh | bash
```

#### 네트워크 프록시 환경

증상: 기업 프록시 환경에서 설치 또는 연결 실패

해결법:

1. 프록시 환경 변수를 설정합니다:

```
export HTTP_PROXY=http://proxy.company.com:8080
export HTTPS_PROXY=http://proxy.company.com:8080
```

2. npm 프록시를 설정합니다 (npm 설치 방식 사용 시):

```
npm config set proxy http://proxy.company.com:8080
npm config set https-proxy http://proxy.company.com:8080
```

## WSL 설정 문제

증상: Windows WSL에서 Claude Code가 정상 동작하지 않음

해결법:

1. WSL 2를 사용하고 있는지 확인합니다:

```
wsl --version
```

2. WSL 내에서 네이티브 설치를 수행합니다:

```
curl -fsSL https://claude.ai/install.sh | bash
```

3. Windows 네이티브 설치와 WSL 설치를 혼용하지 않습니다.

## | 인증 문제

### 로그인 실패

증상: **Authentication failed** 또는 로그인 화면이 열리지 않음

해결법:

1. 인증 상태를 확인합니다:

```
claude auth status --text
```

2. 로그아웃 후 다시 로그인합니다:

```
claude auth logout
claude auth login
```



3. SSO를 사용하는 경우 이메일을 명시합니다:

```
claude auth login --email user@company.com --sso
```

## 토큰 만료

**증상:** 세션 중 갑자기 인증 오류 발생

**해결법:**

1. `claude auth login` 으로 재인증합니다.
2. 지속적으로 발생하면 `claude --debug "api"` 로 상세 로그를 확인합니다.

## | 성능 문제

### 느린 응답

**증상:** Claude Code의 응답이 평소보다 현저히 느림

**해결법:**

1. 현재 모델과 노력 수준을 확인합니다:

```
/status
```

2. 더 빠른 모델로 전환합니다:

```
/model haiku
/effort low
```

3. 컨텍스트가 커졌을 수 있으므로 압축합니다:

```
/compact
```

4. `/cost` 로 현재 토큰 사용량을 확인합니다.

## 컨텍스트 초과

증상: `Context window exceeded` 또는 응답 품질 저하

해결법:

1. `/compact` 로 대화를 압축합니다:

```
/compact focus on the current task only
```

2. `/clear` 로 대화를 초기화하고 필요한 컨텍스트만 다시 제공합니다.
3. 1M 컨텍스트 모델을 사용합니다:

```
/model opus[1m]
```

## 토큰 소비 과다

증상: 예상보다 빠르게 토큰 한도에 도달

해결법:

1. CLAUDE.md를 200줄 이하로 최적화합니다.
2. 단순 작업에는 `haiku` 또는 낮은 노력 수준을 사용합니다.
3. `--max-budget-usd` 로 비용 상한을 설정합니다.
4. MCP 도구가 많으면 Tool Search를 활성화합니다.

## 권한 문제

### 파일 수정 차단

증상: Claude가 파일을 수정하려 할 때 계속 차단됨

해결법:

1. 현재 권한 규칙을 확인합니다:

```
/permissions
```

2. deny 규칙이 너무 넓게 설정되어 있지 않은지 확인합니다.
3. 필요한 경로를 allow에 추가합니다.

4. 권한 모드를 `acceptEdits` 로 변경하여 편집을 자동 승인합니다:

```
claude --permission-mode acceptEdits
```

## 명령 실행 거부

**증상:** Bash 명령어 실행이 지속적으로 거부됨

**해결법:**

1. 허용할 명령어 패턴을 settings.json의 allow에 추가합니다:

```
{
 "permissions": {
 "allow": ["Bash(npm run *)"]
 }
}
```

2. 샌드박스 설정에서 해당 명령어가 `excludedCommands` 에 포함되어 있지 않은지 확인합니다.

## MCP 연결 문제

### 서버 시작 실패

**증상:** MCP 서버가 연결되지 않거나 시작 시 오류 발생

**해결법:**

1. 서버 상태를 확인합니다:

```
claude mcp list
claude mcp get <서버이름>
```

2. Studio 서버의 경우 의존성이 설치되어 있는지 확인합니다:

```
npx -y <패키지이름> --help
```

3. 환경 변수가 올바르게 설정되어 있는지 확인합니다.

4. 서버를 제거하고 다시 추가합니다:

```
claude mcp remove <서버이름>
claude mcp add --transport http <서버이름> <URL>
```

## 인증 오류

증상: MCP 서버에 인증 실패

해결법:

1. `/mcp` 명령으로 OAuth 인증을 다시 수행합니다.
2. Bearer 토큰이 만료되지 않았는지 확인합니다.
3. 환경 변수로 토큰을 전달하는 경우 값이 올바른지 확인합니다.

## Hooks 문제

### Hook이 실행되지 않음

증상: 설정한 Hook이 트리거되지 않음

해결법:

1. `/hooks` 명령으로 현재 등록된 Hook을 확인합니다.
2. settings.json의 JSON 문법이 올바른지 확인합니다.
3. `matcher` 패턴이 도구 이름과 정확히 일치하는지 확인합니다 (예: `"Edit|Write"`, `"Bash"`).
4. `--verbose` 모드로 실행하여 Hook 관련 로그를 확인합니다.

### Exit code 오류

증상: Hook 스크립트가 의도치 않게 차단하거나 무시됨

해결법:

1. Exit code 규칙을 확인합니다:
  - `0`: 성공 (stdout에서 JSON 파싱)
  - `2`: 차단 에러 (stderr를 피드백으로 전달)
  - 기타: 비차단 에러

2. 스크립트를 직접 실행하여 exit code를 확인합니다:

```
echo '{"tool_input":{"file_path":"test.js"}}' | bash .claude/hooks/protect-
files.sh
echo $?
```

## JSON 파싱 실패

**증상:** Hook 스크립트의 JSON 처리에서 오류 발생

**해결법:**

1. `jq` 가 설치되어 있는지 확인합니다:

```
jq --version
```

2. stdin으로 전달되는 JSON 구조를 확인합니다 (verbose 모드 활용).

3. `jq -r` 대신 `jq -e` 를 사용하여 파싱 오류를 명시적으로 처리합니다.

## | 진단 도구 활용

### `/doctor` 명령

`/doctor` 는 Claude Code의 설치 상태, 인증, 설정, MCP 연결 등을 종합적으로 진단합니다. 문제가 발생하면 가장 먼저 실행해야 할 명령입니다.

### `--debug` 플래그

특정 영역의 상세 로그를 출력합니다:

```
API 통신 디버그
claude --debug "api"

MCP 관련 디버그
claude --debug "mcp"

Hooks 관련 디버그
claude --debug "hooks"

복수 영역 동시 디버그
claude --debug "api,mcp,hooks"
```

### **--verbose** 플래그

일반 실행에서 숨겨지는 비차단 에러 메시지를 표시합니다:

```
claude --verbose
```

## 부록 D: 유용한 CLAUDE.md 템플릿 모음

### D.1 웹 프론트엔드 프로젝트 (React + TypeScript)

#### # 프로젝트 규칙

##### ## 빌드 & 테스트

- `npm run dev`로 개발 서버 실행
- `npm run build`로 프로덕션 빌드
- `npm test`로 테스트 실행 (Vitest)
- `npm run lint`로 ESLint 실행
- 커밋 전에 항상 `npm run lint && npm test` 실행

##### ## 코딩 표준

- TypeScript strict 모드 사용
- 2-space 들여쓰기
- 함수형 컴포넌트와 React Hooks 사용 (클래스 컴포넌트 금지)
- 컴포넌트 파일명은 PascalCase (예: UserProfile.tsx)
- 유틸리티 함수는 camelCase (예: formatDate.ts)

##### ## 디렉터리 구조

- `src/components/` - React 컴포넌트
- `src/hooks/` - 커스텀 훅
- `src/utils/` - 유틸리티 함수
- `src/types/` - TypeScript 타입 정의
- `src/api/` - API 호출 함수
- `src/styles/` - 글로벌 스타일

##### ## 상태 관리

- 서버 상태: TanStack Query 사용
- 클라이언트 상태: Zustand 사용
- 폼 상태: React Hook Form 사용

##### ## 스타일링

- Tailwind CSS 사용
- 인라인 스타일 금지
- 반응형 디자인 mobile-first 접근

##### ## 참조

See @README.md for project overview.

See @package.json for available scripts.

## D.2 백엔드 API 프로젝트 (Node.js + Express)

### # 프로젝트 규칙

#### ## 빌드 & 테스트

- `npm run dev`로 개발 서버 실행 (nodemon)
- `npm run build`로 TypeScript 컴파일
- `npm test`로 테스트 실행 (Jest)
- `npm run test:integration`으로 통합 테스트 실행
- `npm run lint`로 ESLint 실행

#### ## API 설계 규칙

- REST API, OpenAPI 3.0 스펙 준수
- URL은 kebab-case 사용 (예: /api/user-profiles)
- 모든 엔드포인트에 입력 검증 포함 (Zod 사용)
- 표준 에러 응답 형식: { error: { code, message, details } }
- 인증: JWT Bearer 토큰
- 페이지네이션: cursor 기반

#### ## 디렉터리 구조

- `src/routes/` - Express 라우터
- `src/handlers/` - 요청 핸들러
- `src/services/` - 비즈니스 로직
- `src/models/` - 데이터 모델 (Prisma)
- `src/middleware/` - Express 미들웨어
- `src/utils/` - 유틸리티 함수

#### ## 데이터베이스

- PostgreSQL (Prisma ORM)
- 마이그레이션: `npx prisma migrate dev`
- 시드: `npx prisma db seed`

#### ## 보안

- SQL 쿼리에 파라미터 바인딩 사용
- 비밀번호는 bcrypt 해싱
- Rate limiting 적용
- CORS 허용 도메인 명시

#### ## 참조

See @README.md for project overview.

See @prisma/schema.prisma for database schema.



## D.3 Python 데이터 분석 프로젝트

### # 프로젝트 규칙

#### ## 환경 설정

- Python 3.11+ 사용
- `pip install -r requirements.txt`로 의존성 설치
- `python -m pytest`로 테스트 실행
- `ruff check .`로 린트 실행
- `ruff format .`로 포매팅

#### ## 코딩 표준

- PEP 8 준수
- Type hints 필수 사용
- docstring은 Google 스타일
- 4-space 들여쓰기

#### ## 디렉터리 구조

- `src/` - 소스 코드
- `notebooks/` - Jupyter 노트북
- `data/raw/` - 원본 데이터 (git 제외)
- `data/processed/` - 전처리된 데이터
- `tests/` - 테스트 코드
- `configs/` - 설정 파일

#### ## 데이터 처리 규칙

- pandas DataFrame 사용 시 메서드 체이닝 선호
- 대용량 데이터는 chunking 처리
- 데이터 경로는 하드코딩하지 않고 configs/ 참조

#### ## 시각화

- matplotlib + seaborn 사용
- 그래프에 반드시 제목, 축 레이블 포함
- 색상 팔레트: viridis 기본

#### ## 참조

See @README.md for project overview.

See @requirements.txt for dependencies.

## D.4 풀스택 모노레포 프로젝트

### # 프로젝트 규칙

#### ## 빌드 & 테스트

- `pnpm install`로 의존성 설치
- `pnpm run build`로 전체 빌드
- `pnpm run test`로 전체 테스트
- `pnpm run dev`로 개발 서버 시작 (Turborepo)

#### ## 모노레포 구조

- `apps/web/` - Next.js 프론트엔드
- `apps/api/` - Express 백엔드
- `apps/admin/` - 관리자 대시보드
- `packages/ui/` - 공유 UI 컴포넌트
- `packages/utils/` - 공유 유틸리티
- `packages/types/` - 공유 TypeScript 타입
- `packages/config/` - ESLint, TypeScript 공유 설정

#### ## 패키지 관리

- pnpm workspace 사용
- 패키지 간 의존성은 `workspace:\*` 프로토콜 사용
- 공유 타입은 반드시 packages/types/에 정의

#### ## 코딩 표준

- TypeScript strict 모드
- ESLint + Prettier 필수
- 커밋 메시지: Conventional Commits (feat:, fix:, docs: 등)
- PR 제목에 영향 받는 앱/패키지 명시

#### ## CI/CD

- GitHub Actions로 자동 빌드/테스트
- main 브랜치 직접 push 금지
- PR 머지 전 테스트 통과 필수

#### ## 참조

See @README.md for project overview.

See @pnpm-workspace.yaml for workspace config.

## D.5 인프라/DevOps 프로젝트 (Terraform + Docker)

### # 프로젝트 규칙

#### ## 명령어

- `terraform init`으로 초기화
- `terraform plan`으로 변경 계획 확인
- `terraform apply`로 인프라 적용
- `docker compose up -d`로 로컬 환경 시작
- `docker compose down`으로 환경 중지

#### ## Terraform 규칙

- 모든 리소스에 tags 포함 (environment, team, service)
- 변수는 variables.tf에 정의, 기본값은 terraform.tfvars에
- 상태 파일은 S3 + DynamoDB 원격 백엔드 사용
- 모듈은 modules/ 디렉터리에 구성

#### ## 디렉터리 구조

- `environments/` - 환경별 설정 (dev, staging, prod)
- `modules/` - 재사용 가능한 Terraform 모듈
- `docker/` - Dockerfile 및 compose 파일
- `scripts/` - 배포 및 운영 스크립트

#### ## 보안

- 비밀 값은 절대 코드에 하드코딩하지 않음
- AWS 시크릿은 Secrets Manager 사용
- IAM 정책은 최소 권한 원칙 적용
- 보안 그룹은 필요한 포트만 개방

#### ## Docker 규칙

- 멀티 스테이지 빌드 사용
- Alpine 기반 이미지 선호
- .dockerignore 파일 유지
- 헬스체크 포함

#### ## 참조

See @README.md for project overview.

See @environments/dev/main.tf for infrastructure layout.

## D.6 최소 범용 템플릿

프로젝트 유형에 관계없이 사용할 수 있는 최소한의 템플릿입니다:

## # 프로젝트 규칙

### ## 빌드 & 테스트

- 빌드: [빌드 명령어]
- 테스트: [테스트 명령어]
- 린트: [린트 명령어]

### ## 코딩 표준

- [들여쓰기 규칙]
- [네이밍 컨벤션]
- [기타 팀 규칙]

### ## 핵심 아키텍처

- [사용 중인 프레임워크/라이브러리]
- [데이터베이스/외부 서비스]
- [주요 디렉터리 구조]

### ## 참조

See @README.md for project overview.

**팁:** `/init` 명령을 실행하면 Claude가 코드베이스를 분석하여 프로젝트에 맞는 CLAUDE.md를 자동으로 생성합니다. 위 템플릿을 기반으로 자동 생성된 내용을 보강하는 것이 가장 효율적인 방법입니다.

이 책의 14개 챕터를 통해 우리는 긴 여정을 함께했습니다. AI 코딩 도구의 역사를 살펴보는 것으로 시작하여, Claude Code를 설치하고, 첫 대화를 나누고, 핵심 기능을 마스터하고, 실전 프로젝트에 적용하며, 팀 전체의 개발 워크플로를 혁신하는 데까지 이르렀습니다.

하지만 이 책은 여정의 끝이 아니라, 시작점입니다.

## 다음 단계

### 1. 일상에 적용하세요

가장 중요한 다음 단계는 여러분의 실제 프로젝트에서 Claude Code를 사용하는 것입니다. 이 책에서 배운 기법들을 하나씩 적용해보세요:

- 프로젝트에 CLAUDE.md를 작성하고 `/init` 으로 시작하세요
- 매일 하는 반복 작업 하나를 Claude Code로 자동화해보세요

- 코드 리뷰에 Claude Code를 활용해보세요
- 디버깅할 때 에러 메시지를 그대로 붙여넣는 습관을 들이세요

## 2. 팀에 도입하세요

혼자 사용하는 것에서 팀 전체로 확장하면 생산성 향상이 배가됩니다:

- `.claude/settings.json` 과 `.claude/rules/` 를 git에 커밋하여 팀 표준을 세우세요
- `.mcp.json` 으로 팀 공용 MCP 서버를 공유하세요
- CI/CD 파이프라인에 Claude Code를 통합하세요
- 팀 온보딩 가이드에 Claude Code 설정을 포함하세요

## 3. 최신 동향을 따라가세요

Claude Code는 빠르게 진화하고 있습니다. 새로운 기능과 개선 사항을 놓치지 마세요:

- 공식 문서: docs.anthropic.com에서 최신 기능과 API 변경 사항을 확인하세요
- 릴리스 노트: `/release-notes` 명령으로 변경 로그를 확인하세요
- 자동 업데이트: `claude update` 로 항상 최신 버전을 유지하세요

## 4. 커뮤니티에 참여하세요

Claude Code 사용자 커뮤니티에서 다른 개발자들과 경험을 나누세요:

- 자신만의 CLAUDE.md 템플릿, Hook 레시피, MCP 설정을 공유하세요
- 다른 개발자들의 활용 사례에서 영감을 얻으세요
- 버그를 발견하면 `/feedback` 명령으로 피드백을 보내세요

## | 마지막으로

에이전틱 코딩은 개발자의 역할을 없애는 것이 아니라, **확장**하는 것입니다. AI가 반복적인 코딩, 보일러플레이트 작성, 테스트 생성 같은 작업을 대신해주면, 개발자는 아키텍처 설계, 사용자 경험 고민, 비즈니스 문제 해결 같은 더 높은 수준의 사고에 집중할 수 있습니다.

Claude Code는 여러분의 능력을 대체하는 것이 아니라, 여러분이 가진 전문성을 최대한 발휘할 수 있도록 돕는 도구입니다. 마치 숙련된 동료 개발자가 옆에 앉아 함께 코딩하는 것처럼, Claude Code와의 협업을 통해 더 좋은 코드를 더 빠르게 작성하시길 바랍니다.

터미널을 열고, `claude` 를 입력하세요. 새로운 개발 경험이 여러분을 기다리고 있습니다.

---

이 책의 내용은 Claude Code의 발전과 함께 계속 업데이트됩니다.